Deep Learning & Generative AI

Beginner study guide: plain-language notes + runnable experiments with outputs

Part 1: notes (01 history · 02 AI/ML/DL/GenAI & the neuron · 03 perceptron · 04 activation functions · 05 normalization & regularization · 06 loss functions · 07 how an MLP works · 08 weight initialization · 09 text → tokens → vectors)
Part 2: notebooks (each lecture's claims tested in code)

Plus the cheat sheet. Runnable notebooks are in the **notebooks/** folder next to this PDF.

01 · History of deep learning (lecture of 04-08-2026)

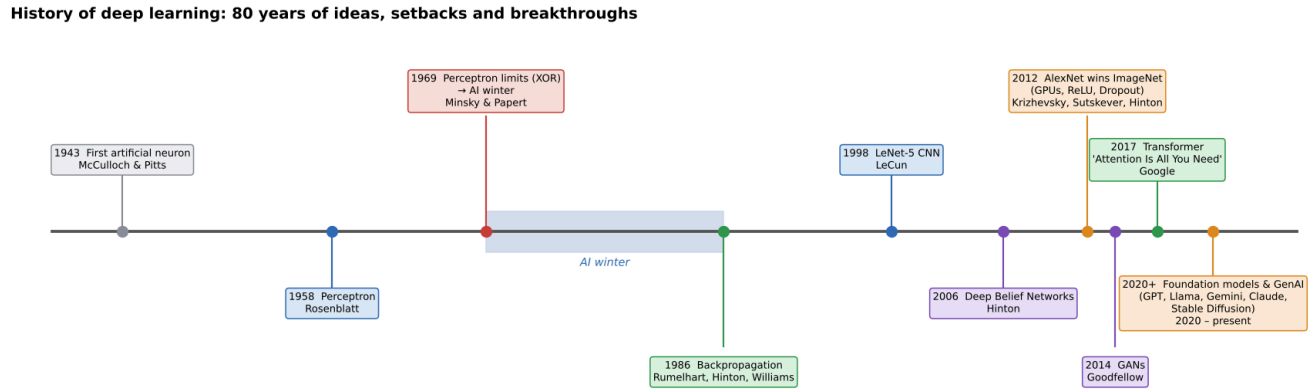
Notebook: `notebooks/02_neurons_1943_to_1986.ipynb` runs the first four milestones (1943 → 1986) as code.

---

1. What is deep learning?

> **Deep learning** is a branch of machine learning that uses **artificial neural networks with multiple hidden layers** to learn complex patterns from data.

It became famous only in the last decade or so, but its roots go back to the **1940s**. The field moved through **ideas, setbacks and breakthroughs** before becoming the engine of today's AI.



---

2. The ten milestones (learn the year, the people and the one-line contribution)

Year	Milestone	Who	Why it matters
1943	first **artificial neuron**	Warren **McCulloch** & Walter **Pitts**	first mathematical model of a biological neuron; showed that neurons with binary inputs and outputs can do **logic** (AND, OR, NOT); brain functions can be represented **computationally**
1958	**Perceptron**	Frank **Rosenblatt**	the first **trainable** neural network: it learns its weights from examples. Single layer, supervised, binary classification. **Limitation: only linearly separable problems**
1969	***Perceptrons*** (the book)	Marvin **Minsky** & Seymour **Papert**	showed the limits of single-layer networks (e.g. **XOR**) → funding fell, research moved to rule-based **expert systems** → the **AI winter**

Year	Milestone	Who	Why it matters
1986	**Backpropagation** popularised	David **Rumelhart**, Geoffrey **Hinton**, Ronald **Williams**	trains **multi-layer** networks by sending the error **backwards**, so deeper networks can learn complex patterns. Still **the** standard training algorithm
1998	**LeNet-5** (CNN)	Yann **LeCun**	one of the first successful **convolutional** networks: handwritten digits, bank cheques, postcodes. Showed networks can handle **images**
2006	**Deep Belief Networks**	Geoffrey **Hinton**	**layer-wise pre-training** made deeper networks practical. Marks the **start of modern deep learning**
2012	**AlexNet** wins **ImageNet**	Alex **Krizhevsky**, Ilya **Sutskever**, Geoffrey **Hinton**	trained on **GPUs**, used **ReLU** and **Dropout**, and crushed traditional computer-vision methods. Sparked wide adoption in academia and industry
2014	**GANs**	Ian **Goodfellow**	**generator vs discriminator**: image generation, face synthesis, super-resolution, data augmentation. Opened up **generative AI**
2017	**Transformer**	researchers at **Google** ("**Attention Is All You Need**")	built on the **attention** mechanism, replaced recurrent networks for language: translation, chatbots, summarisation, code generation
2020–now	**Foundation models & Generative AI**	many organisations	**LLMs**, multimodal AI, **diffusion models**, AI assistants: GPT, Llama, Gemini, Claude, Stable Diffusion

Memory trick: three "Hinton" years (1986 backprop, 2006 DBNs, 2012 AlexNet). He's often called a "godfather of deep learning".

Two stories worth telling in an exam answer

- **The AI winter (1969 → mid-1980s).** A single perceptron draws one straight line, so it can't learn **XOR** (the 1s are on one diagonal, the 0s on the other). Minsky and Papert's book made this famous, funding for neural networks dried up, and AI research turned to hand-written rules (expert systems). **Backpropagation (1986)** ended it: multi-layer networks *can* learn XOR, and now they could be **trained**. Notebook 02 reproduces the whole story in code.
- **The 2012 turning point.** The ideas (CNNs, backprop) already existed. What changed was **big data** (ImageNet's 1.2 million labelled images), **GPUs** (fast parallel training) and **better tricks** (ReLU, Dropout). AlexNet's win convinced the world.

---

3. Why deep learning succeeded (the five enabling factors)

Factor	Contribution
large datasets	more data → better training (ImageNet, web text)

Factor	Contribution
GPUs and TPUs	massively parallel hardware made training deep networks fast enough
improved algorithms	backpropagation, **ReLU**, **Batch Normalization**, **Dropout**, **attention** (chapters 04–05)
open-source frameworks	TensorFlow, **PyTorch**, Keras made building networks accessible
cloud computing	scalable compute to train very large models

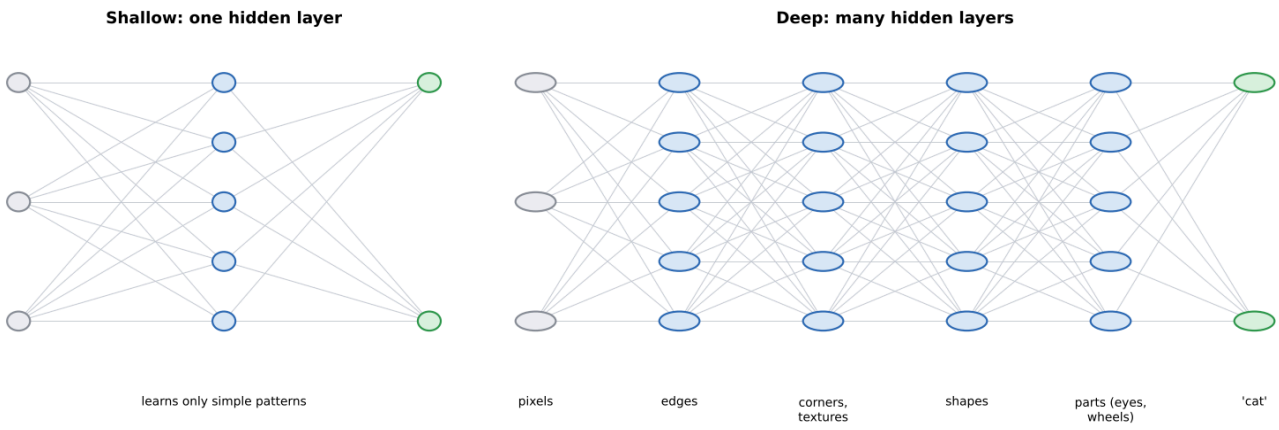
4. How the models evolved

artificial neuron → perceptron → multi-layer perceptron (MLP) → CNN → RNN → LSTM → Transformer → foundation models and GenAI

- **MLP:** hidden layers (solves XOR).
- **CNN:** images, with local filters.
- **RNN / LSTM:** sequences, with memory (the LSTM remembers longer).
- **Transformer:** attention, parallel training, the basis of LLMs.

---

5. Why is it called "deep" learning?



> "Deep" refers to the **number of hidden layers** between input and output: the **depth** of the network. It does **not** mean deep thinking, deep intelligence or deep mathematics.

Each layer builds on the previous one (hierarchical feature learning). The lecture's example, recognising a cat or an object:

Layer	Learns
input	raw pixels
layer 1	edges
layer 2	corners and textures
layer 3	shapes
layer 4	object parts (eyes, nose, wheels)

Layer	Learns
layer 5	the complete object

The lecture's face-recognition version: pixels → edges and lines → eyes, nose, mouth → face shape → identity.

The key point: nobody programs these features. **The network learns them automatically** from data. That's the big difference from classical machine learning, where a human designs the features (chapter 02, and notebook 01 shows it).

A shallow network (one hidden layer) learns only simple features. **A deep network** builds increasingly **abstract** representations, which is why deep learning leads in computer vision, speech recognition, natural-language processing and generative AI.

Key takeaways (from the lecture)

- Deep learning started with the **artificial neuron in 1943**.
- The **perceptron (1958)** made networks trainable but was limited to simple (linearly separable) problems.
- **Backpropagation (1986)** made multi-layer networks trainable.
- **CNNs (1998)** transformed computer vision, and **AlexNet (2012)** showed the power of deep networks on GPUs.
- **Transformers (2017)** revolutionised NLP and underpin modern **LLMs**.
- Today deep learning powers generative AI, autonomous systems, healthcare, finance, robotics and science.

Check yourself

1. Who built the first artificial neuron, and when? What could it do?
2. What did Rosenblatt's perceptron add, and what was its limitation?
3. What caused the AI winter, and what ended it?
4. What three ingredients did AlexNet combine in 2012?
5. Which 2017 paper introduced the Transformer, and what mechanism is it built on?
6. What does "deep" in deep learning refer to? What does it NOT mean?
7. Name the five factors that enabled deep learning's success.
8. Put in order: Transformer, LSTM, perceptron, CNN, MLP, artificial neuron, RNN.

Answers

1. McCulloch and Pitts, 1943. A mathematical model of a neuron that performed logical operations with binary inputs and outputs.
2. Learning: it was the first trainable neural network, adjusting its weights from examples. It could only solve linearly separable problems (not XOR).
3. Minsky and Papert's *Perceptrons* (1969) exposed the limits of single-layer networks, and funding moved to rule-based AI. Backpropagation (1986) made multi-layer networks trainable.
4. GPUs, ReLU and Dropout (plus the big ImageNet dataset).
5. "Attention Is All You Need" (Google). The attention mechanism.

6. The number of hidden layers (the depth). Not deep thinking, deep intelligence or deep mathematics.
7. Large datasets, GPUs and TPUs, improved algorithms, open-source frameworks, cloud computing.
8. Artificial neuron → perceptron → MLP → CNN → RNN → LSTM → Transformer.

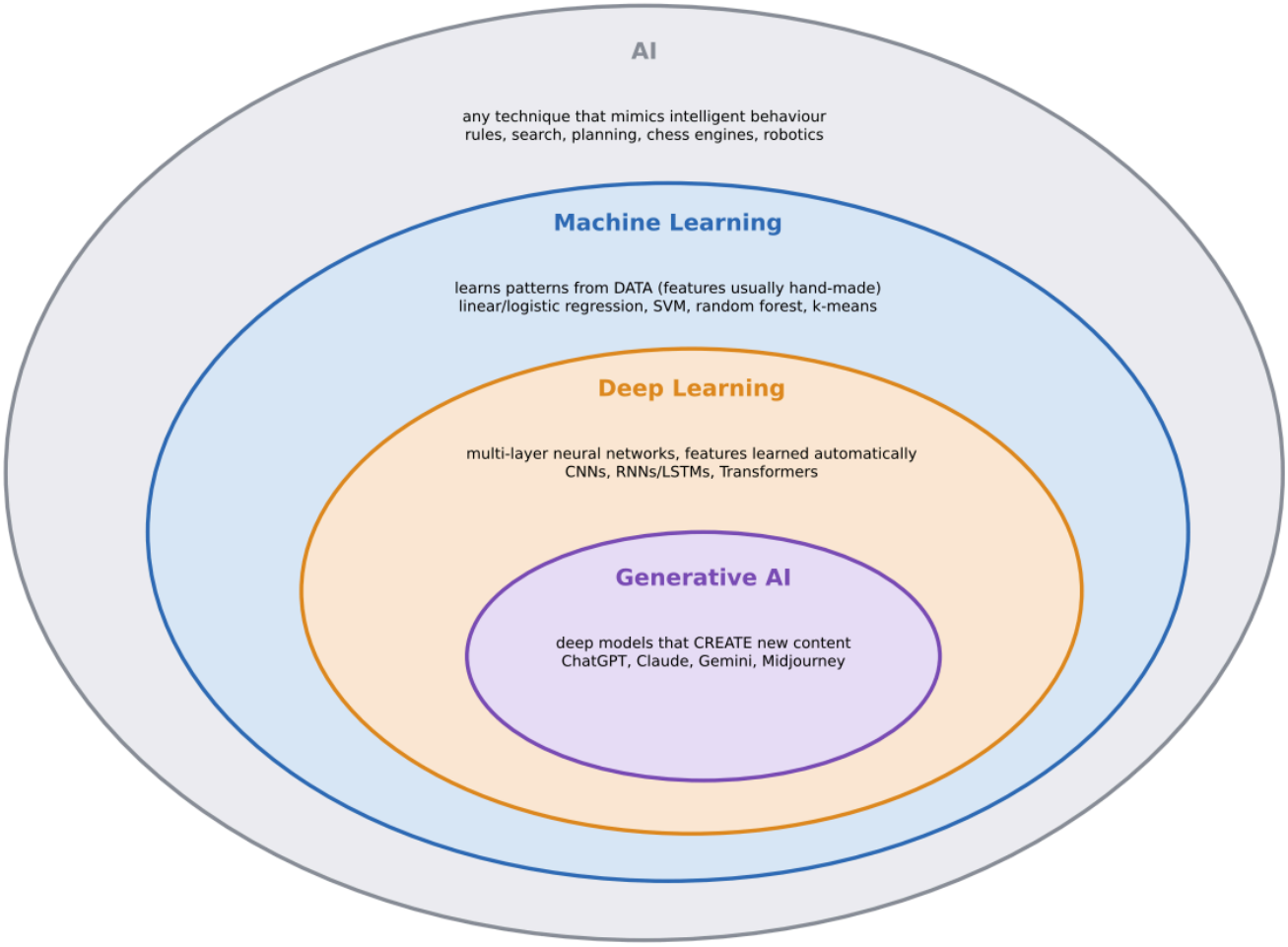
02 · AI vs ML vs DL vs GenAI, and the biological neuron analogy (lecture of 05-08-2026)

Notebooks: - `notebooks/01_ai_ml_dl_genai_in_code.ipynb` : one problem solved at all four levels; - `notebooks/02_neurons_1943_to_1986.ipynb` : the neuron, from biology to code.

Part A · AI vs ML vs DL vs GenAI

1. The hierarchy: nested circles

AI ⊃ ML ⊃ DL ⊃ GenAI: each circle is a special case of the one around it



ML is a subset of AI, DL is a subset of ML, GenAI is a subset (and an application) of DL. Each inner circle keeps the properties of the one around it and adds its own defining constraint.

Level	Defining idea	Examples
Artificial Intelligence (AI)	any technique that lets a machine **mimic intelligent behaviour**	rule-based expert systems, search algorithms, robotics, chess engines
Machine Learning (ML)	AI that **learns patterns from data** instead of following hand-written rules	random forest, SVM, logistic regression
Deep Learning (DL)	ML using **multi-layer neural networks** that **learn features automatically** from raw data	CNNs, RNNs, Transformers

Level	Defining idea	Examples
Generative AI (GenAI)	DL models trained to **generate new content** (text, images, audio, code), not just classify or predict	ChatGPT, Claude, Gemini, Midjourney, DALL-E

2. Each level in more detail

AI: the broadest term

- Any system doing tasks that normally need human intelligence: reasoning, planning, perception, language, decision-making.
- **Not defined by a technique.** It includes 1960s–80s **symbolic, rule-based** systems (if-then medical-diagnosis rules), **search and planning** (pathfinding, early chess programs), all of ML, and **robotics and control**.
- Best seen as a **goal / field of study**: building machines that behave intelligently. ML, DL and GenAI are **strategies** for reaching that goal.

ML: learning from data

- Instead of a person writing the logic, the algorithm gets **examples** and a learning procedure and **infers** the logic.
- It learns an input → output mapping (or finds structure in unlabelled data), and usually **improves with more data**.
- It **often still needs a human to engineer the features** (which input variables the model sees).
- Three paradigms: **supervised, unsupervised, reinforcement** learning.

Algorithm	Typical use
linear / logistic regression	a number or a yes/no from tabular data
decision trees / random forests	classification and regression on business data
SVM	classification with a clear margin
k-means	grouping unlabelled data (customer segments)

DL: deep neural networks

- **Many layers** ("deep"), each turning its input into a **more abstract representation**: edges → shapes and textures → whole objects or faces. This **hierarchical feature extraction** is the mechanical difference from classical ML.
- **Automatic feature extraction** from raw data, with no manual feature engineering.
- Needs **much more data and compute** (GPUs).
- **Excels at unstructured data:** images, audio, video, text.
- Architectures: **CNNs** (images), **RNNs/LSTMs/GRUs** (sequences), **Transformers** (language and beyond).

GenAI: creating new content

- A classifier says "cat or dog". A generative model **draws a new cat that never existed**.
- Trained, often **self-supervised**, on **massive** data to learn the **distribution** of the data, not just a decision boundary.
- Produces new text, images, audio, video and code.
- Mostly **Transformer**-based (LLMs), plus **diffusion models** for images and video.
- What separates it from older generative methods is mainly **scale**: data, parameters, compute.

System	Generates
ChatGPT, Claude, Gemini	text: conversation, writing, code, analysis
Midjourney, DALL-E	images from text
Runway	video generation and editing
ElevenLabs	synthetic speech, voice cloning

3. The comparison table (exam favourite)

	AI	ML	DL	GenAI
scope	broadest	subset of AI	subset of ML	subset of DL
core mechanism	rules, search, learning…	statistical patterns from examples	multi-layer networks, automatic features	large networks that model and reproduce the data distribution
feature engineering	n/a / varies	**usually manual**	**automatic**	automatic, at massive scale
typical data needs	varies	**moderate** (hundreds to thousands)	**large** (100k+)	**massive** (web-scale, billions of tokens or images)
typical output	any intelligent behaviour	prediction / classification	prediction, classification, detection	**newly generated content**
example	chess program	spam filter	CNN finding tumours on X-rays	LLM writing an essay

4. The three distinguishing factors

1. **Data.** The amount needed rises sharply from ML → DL → GenAI.
2. Classical ML: often fine with thousands of rows of tabular data.
3. DL: needs large labelled sets (or augmentation) to avoid **overfitting** its many parameters.
4. GenAI: **self-supervised** learning on web-scale, mostly unlabelled data.
5. **Feature engineering.**
6. ML: a human selects and builds features (e.g. "average transaction in the last 30 days" for fraud detection).
7. DL: learned automatically by the layers.
8. GenAI: learned at huge scale, producing rich **embeddings** that transfer to many tasks.
9. **Architecture.**

10. ML: simple, well-understood models (trees, SVMs, linear functions).
11. DL: **artificial neural networks** (MLPs, CNNs, RNNs/LSTMs/GRUs).
12. GenAI: overwhelmingly **Transformers** (self-attention relates all parts of the input at once), plus **diffusion models**.

5. Real systems mapped (shows the nesting)

System	AI	ML	DL	GenAI
1980s chess engine with hand-coded rules	✓	✗	✗	✗
Naive Bayes spam filter	✓	✓	✗	✗
CNN photo tagger ("beach", "mountain")	✓	✓	✓	✗
ChatGPT writing a poem	✓	✓	✓	✓

Every GenAI system is also DL, ML and AI, **but not the other way round**.

6. Common misconceptions

- **"AI and ML are the same."** ML is one way to build AI, and many AI systems (rule-based) don't learn at all.
- **"DL is just ML with more layers."** Technically true, but the real shift is **automatic feature learning**, which changes the whole workflow.
- **"GenAI is a different technology from DL."** No: it is deep learning (mostly Transformers) applied and scaled to content generation.
- **"Bigger is always better."** A well-tuned classical model can beat a big deep network on **small, structured** data.

Why it matters in practice: - a few thousand rows of business data → **classical ML**; - lots of labelled images → **DL (CNNs)**; - producing new content (a summary, code, an image) → **GenAI**.

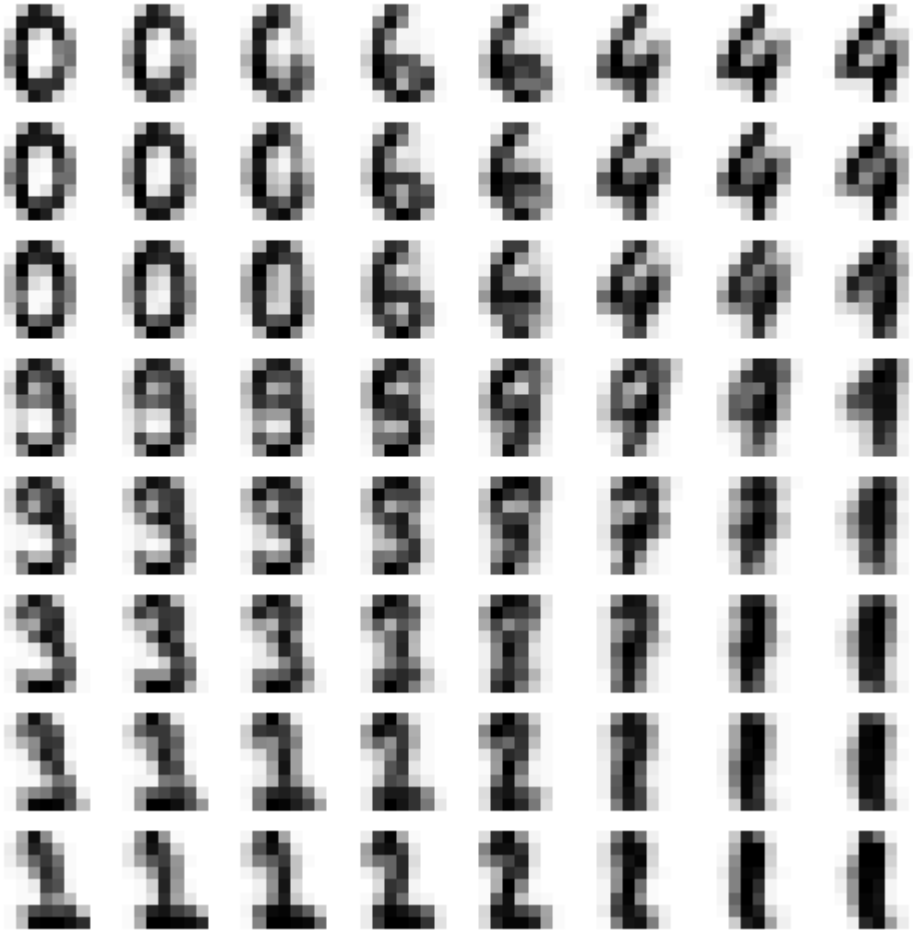
Picking the right category is often the most important early decision in an AI project.

What notebook 01 measured (same digits, four levels)

Level	Approach	Result
AI	hand-written if-then rules	97% on **0 vs 1 only**; rules for all 10 digits are impractical
ML	logistic regression on 8 **hand-engineered** features	74% on all 10 digits; limited by the features chosen
ML	the same model on 64 raw pixels	97%
DL	2-hidden-layer network on raw pixels	98%, with features it invented itself
GenAI	a tiny generative network (VAE)	**draws new digits** that aren't copies of training images

Honest note: on these tiny, clean 8×8 digits, the deep network only tied with logistic regression on raw pixels at every training-set size. DL's big advantage appears on large, complex data (photos, speech, text).

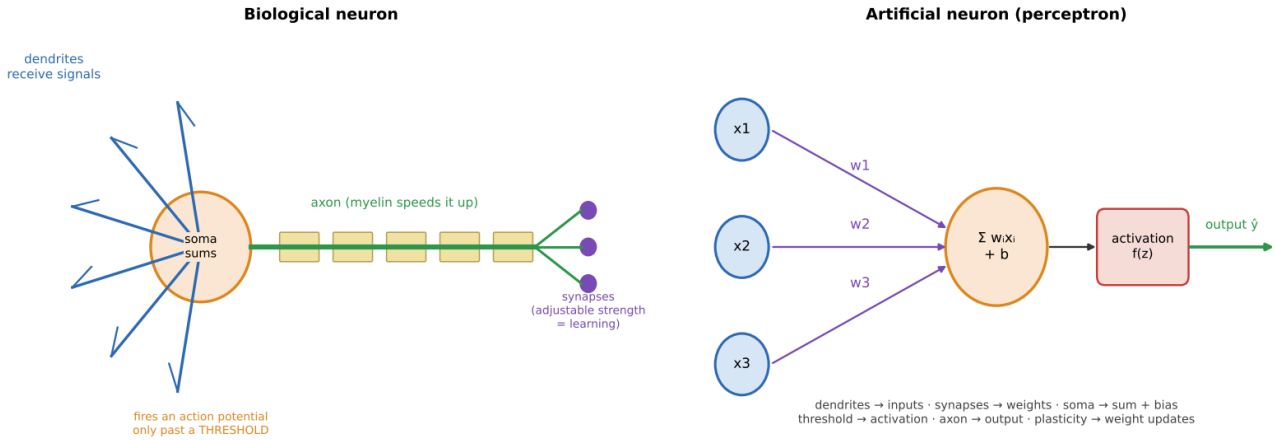
64 NEW digits generated from a grid of 2-number codes (none of them is a copy of a training image)



Part B · The biological neuron analogy

7. Why the brain?

Artificial neural networks were **directly inspired by biological neurons**. The **McCulloch–Pitts neuron (1943)** and the **perceptron (1958)** were attempts to model, mathematically, how a neuron receives, processes and passes on information. Modern DL has moved far from biology, but its vocabulary still maps onto it.



8. The biological neuron (four parts)

The human brain has about **86 billion neurons**, each connected to thousands of others.

Part	What it does
dendrites	branch-like **inputs**: receive signals from other neurons (thousands of them)
soma (cell body)	**integrates (sums)** the incoming signals. If the total passes a **threshold**, it fires an **action potential**, otherwise nothing: **all-or-nothing**
axon	long cable carrying the output signal to other neurons (under 1 mm to over 1 m long). **Myelin** insulation speeds it up
synapse	the junction with the next neuron. The signal crosses a tiny gap (**synaptic cleft**) chemically, via **neurotransmitters**. Its **strength can change** with experience (**synaptic plasticity**), the biological basis of **learning and memory**

Signal path: dendrites receive → soma sums (weighted by synapse strengths) → fires if over the threshold → action potential travels down the axon → converted to chemicals at the synapse → influences the next neuron. Synapse strengths change over time, which is how the network learns.

9. The artificial neuron (perceptron)

A **drastic mathematical simplification** that keeps the essential computation: 1. **Inputs** x_1, x_2, x_3… are the feature values (pixels, word embeddings). They play the role of the **dendrites**. 2. **Weights** w_1, w_2, w_3… set the importance of each input, like **synapse strength**. Training

adjusts them (backpropagation + gradient descent), the analogue of **plasticity**. 3. **Summation:** $z = \Sigma\, w_i x_i + \mathbf{b}$ (the bias shifts the sum), like the **soma**. 4. **Activation function** f(z): the **threshold** idea, adding non-linearity (step, sigmoid, ReLU, tanh). **Without it, any number of layers collapses into one linear function** (chapter 04 proves this in code).

10. The mapping table (learn it)

Biological	Artificial	Role
dendrites	inputs x_1, x_2, …	receive signals / feature values
synapse (strength)	weights w_1, w_2, …	importance of each signal
soma	$\Sigma\, w_i x_i + b$	integrate into one value
firing threshold / action potential	activation function	whether, and how strongly, to "fire"
axon	output to the next layer	carry the result forward
synaptic plasticity	weight updates (backpropagation)	learn from experience / data

From one neuron to a network

Brain	Artificial neural network
billions of neurons in regions and circuits (visual cortex, motor cortex)	layers: input, hidden, output
signals pass through many layers of neurons	**forward propagation** through the layers
learning = strengthening/weakening synapses	learning = **backpropagation** weight updates
regions specialise (vision, language)	architectures specialise (CNNs for images, RNNs/Transformers for sequences)

11. Similarities and where the analogy breaks

Similar: - both take **many weighted inputs**, **sum** them, and apply a **threshold / non-linearity**; - both **adjust connection strengths** with experience; - both get intelligence only from **many connected units**, never from one.

Different (don't treat ANNs as brain models):

	Biological	Artificial
signal	electrochemical (ion channels, neurotransmitters)	numbers (floating point)
timing	asynchronous, continuous-time spikes	synchronous, discrete steps
learning	biochemical plasticity, not fully understood	an exact procedure (backprop + gradient descent)
complexity	thousands of synapses, many neuron types, glial support	weighted sum + activation
energy	**~20 watts** for the whole brain	training big models uses enormous energy

	Biological	Artificial
scale	~86 billion neurons, **~100 trillion synapses**	even the largest models have fewer parameters than the brain has synapses

The artificial neuron is **inspired by**, but a **drastic simplification** of, the real one, **by design**: the goal is engineering performance, not biological accuracy.

12. Why the analogy is still useful

- **Many layers:** the brain processes information hierarchically across regions, and deep networks build abstraction layer by layer.
- **Learned weights:** synapses adapt with experience, and weights are what training adjusts.
- **Activation functions:** neurons fire only past a threshold, and networks need non-linearity.
- **More connections → more complex behaviour**, up to a point.

Connecting the two halves: DL (Part A) is *defined* by artificial neural networks, which are built from the neurons in Part B. Networks of millions or billions of simple neurons need a correspondingly huge amount of data to tune every connection, which is why DL and GenAI need so much more data and compute than classical ML.

Check yourself

1. Draw the hierarchy. Where does ChatGPT sit? A rule-based chess program?
2. Name the three factors that distinguish ML, DL and GenAI.
3. Who chooses the features in classical ML? In DL?
4. Give two misconceptions about AI/ML/DL/GenAI and correct them.
5. Map each part: dendrites, synapse, soma, threshold, axon, plasticity.
6. Why must an artificial neuron have an activation function?
7. Give three differences between biological and artificial neurons.
8. Which kind of model would you pick for: a 3,000-row sales table? 1 million labelled X-rays? Writing product descriptions?

Answers

1. AI ⊃ ML ⊃ DL ⊃ GenAI. ChatGPT is in all four. The chess program is AI only (no learning from data).
2. Data requirements, feature engineering (manual vs automatic), architecture.
3. A human (manual feature engineering). The network itself (automatic).
4. "AI = ML" (no: rule-based AI doesn't learn). "GenAI is a different technology" (no: it's scaled deep learning). Also "bigger is always better" (no: classical ML can win on small structured data).
5. Inputs, weights, weighted sum + bias, activation function, output, weight updates (backpropagation).
6. Without non-linearity, stacked layers collapse into one linear function and can't learn complex patterns. It also mirrors the neuron's firing threshold.
7. Any three of: electrochemical vs numerical signals; asynchronous vs synchronous timing; poorly understood biochemical learning vs exact backpropagation; huge vs simple

complexity; 20 W vs huge energy use; ~100 trillion synapses vs fewer parameters.

8. Classical ML. DL (a CNN). GenAI (an LLM).

03 · The perceptron (lecture of 06-08-2026)

Notebook: `notebooks/03_perceptron_from_scratch.ipynb` (both worked examples recomputed, a perceptron built from scratch, learning-rate and XOR experiments)

---

1. What is a perceptron?

> The **simplest unit of a neural network**: a single artificial neuron that takes numeric inputs, combines them with a **weighted sum**, applies an **activation function**, and outputs a **binary** (0/1) decision.

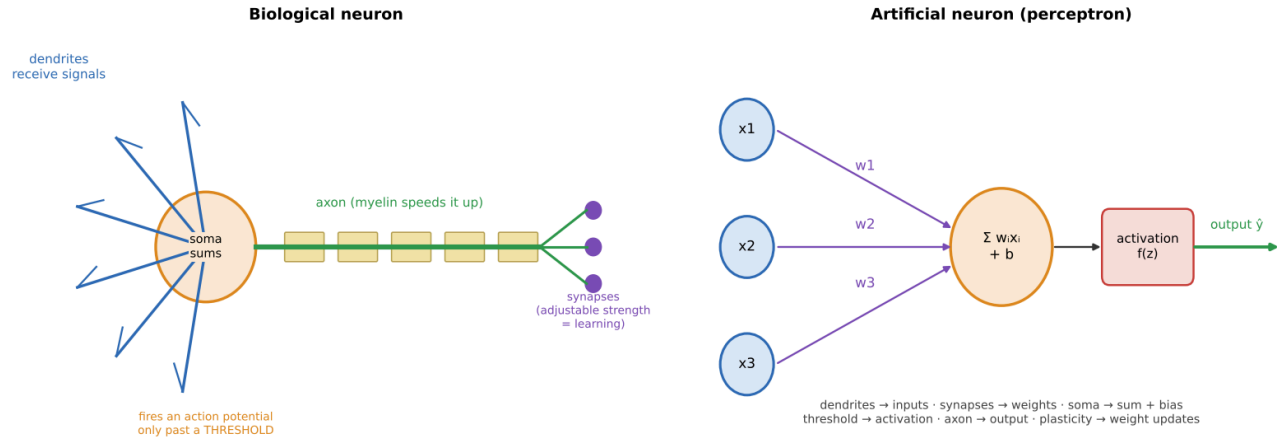
The lecture's running example: predict whether a student will **Pass (1) or Fail (0)** from **x1 = hours of study** and **x2 = hours of practice**.

Symbol	Meaning
x1, x2	inputs (study hours, practice hours)
w1, w2	weights: importance of each input
b	bias: shifts the threshold
z	net input (weighted sum): z = w1·x1 + w2·x2 + b
f(z)	activation function (the **step** function)
ŷ (y_hat)	predicted output (0 = Fail, 1 = Pass)
y	actual (true) answer
η (eta)	learning rate: size of each correction

2. Architecture

The lecture draws three conceptual layers: **input** (x1, x2) → the **neuron** (weighted sum, then activation) → **output** (ŷ). Inside the neuron there are two stages: ① compute z, ② apply f(z).

> Careful with words: the lecture labels the neuron's column "hidden layer". In standard terminology a single perceptron has **no** hidden layer: its neuron *is* the output unit. "Hidden layers" are what multi-layer perceptrons add (section 7). Use the lecture's wording in the lecture's exam, and know the standard meaning.



3. Forward propagation

Information flowing **left to right**: inputs → weighted sum → activation → prediction.

$z = \Sigma\, w_i x_i + b$, $\hat{y} = f(z)$

Worked example (lecture 3.2), recomputed in the notebook

Given x1 = 2, x2 = 3, w1 = 0.5, w2 = 0.8, b = −2: 1. Weighted products: x1·w1 = 2 × 0.5 = **1.0** and x2·w2 = 3 × 0.8 = **2.4**. 2. Net input: z = 1.0 + 2.4 − 2.0 = **1.4**. 3. Step activation: f(1.4) = **1**, since 1.4 ≥ 0. 4. **$\hat{y}$ = 1 → Pass.**

4. The activation: Heaviside step function

f(z) = 1 if z ≥ 0, f(z) = 0 if z < 0

It turns the continuous z into a crisp yes/no. Note that **z = 0 counts as 1** in the lecture's definition. (Chapter 04: the step function's slope is 0 everywhere, which is why modern networks use smooth activations.)

5. Learning: the perceptron update rule

Forward propagation only **predicts**. Learning compares $\hat{y}$ with the truth y and corrects the weights:

$w_i(\text{new}) = w_i(\text{old}) + \eta\cdot(y - \hat{y})\cdot x_i$ $b(\text{new}) = b(\text{old}) + \eta\cdot(y - \hat{y})$

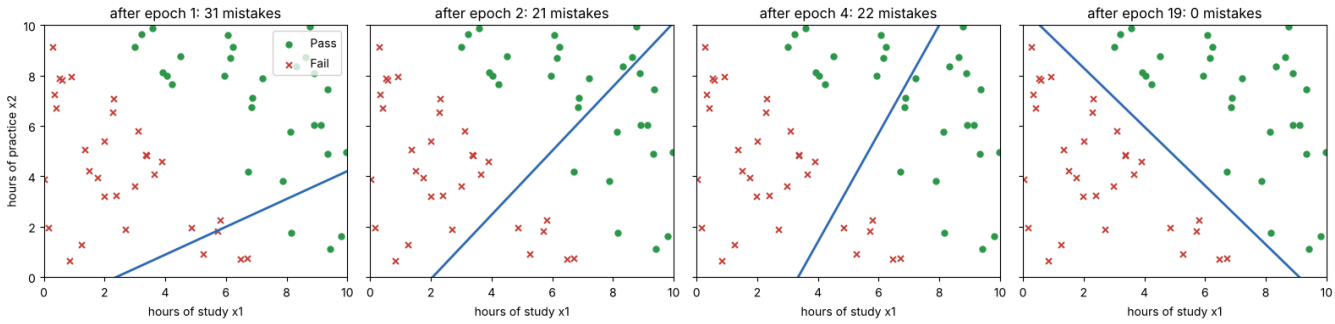
The error $(y - \hat{y})$ can only be:

$y - \hat{y}$	Meaning	Effect
0	correct	no change
+1	said Fail, was Pass (**under-predicted**)	weights move **up**, in proportion to each input
−1	said Pass, was Fail (**over-predicted**)	weights move **down**

Worked example (lecture 5.2), recomputed in the notebook

Given x1 = 2, x2 = 3, w1 = 0.2, w2 = −0.4, b = −1.0, y = 1, ŷ = 0, η = 0.1: 1. Error = 1 − 0 = **+1** (under-prediction). 2. w1 = 0.2 + 0.1·(1)·(2) = **0.4**. 3. w2 = −0.4 + 0.1·(1)·(3) = **−0.1**. 4. b = −1.0 + 0.1·(1) = **−0.9**.

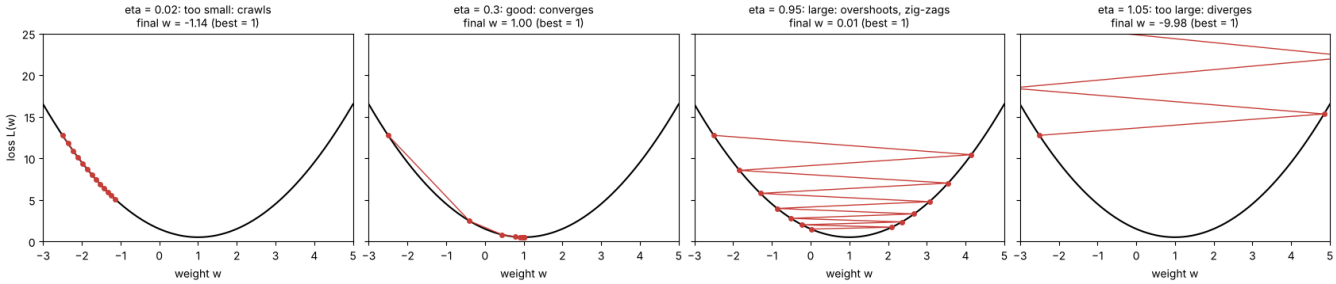
Notice (from the notebook): - Each weight moved **in proportion to its input** (w2 moved more, because x2 = 3 > x1 = 2). - After this one update z went from −1.8 to −0.4. That's **closer, but still wrong** (ŷ is still 0). It takes several passes over the data.



Notebook: the decision line moves as the perceptron learns on 60 students, reaching 0 mistakes after 19 epochs.

6. Gradient descent and the learning rate

Repeating updates over many examples is like **walking downhill on a loss landscape**: each step moves the weights in the direction that reduces the error, until reaching the minimum.



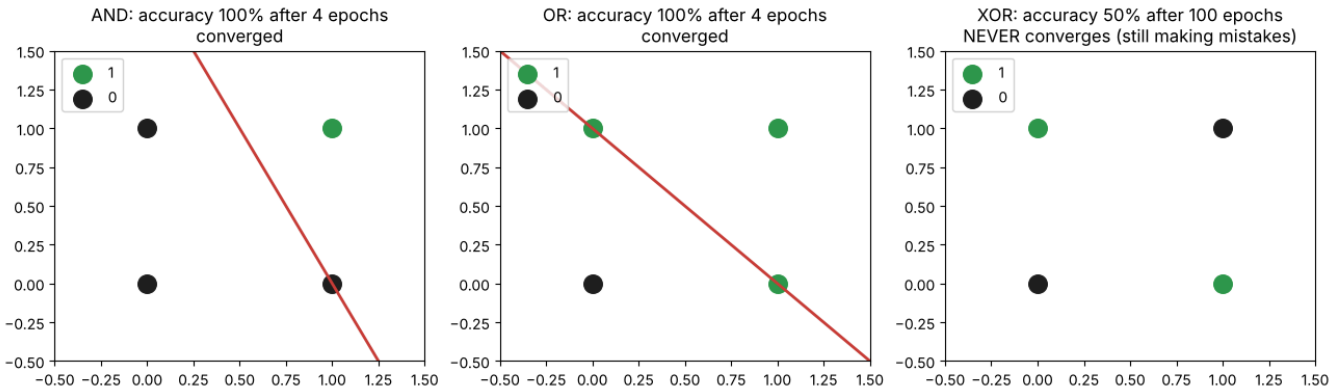
Learning rate	What happens
too small (η → 0)	**very slow**: thousands of tiny steps
good	converges quickly
too large (η ≫ 0)	**overshoots** the minimum, and the loss **bounces** or **diverges**
local vs global minimum	a well-tuned η helps reach the true (global) minimum instead of getting stuck in a shallow local one

> **A surprise the notebook found:** for the classic perceptron starting from **zero weights**, every η (0.001 to 10) took **exactly the same 19 epochs**. The step function only looks at the **sign** of z, and scaling all the weights and the bias by η doesn't change any sign. From non-zero starting weights, η did matter (3 to 19 epochs). The trade-off in the table is really about **gradient descent on a smooth loss**, which all modern networks use, and the notebook's loss-curve plot shows it exactly.

7. Linear separability: what one perceptron can and can't learn

A perceptron draws **one straight line** (w1·x1 + w2·x2 + b = 0). So it can only learn **linearly separable** functions:

x1	x2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

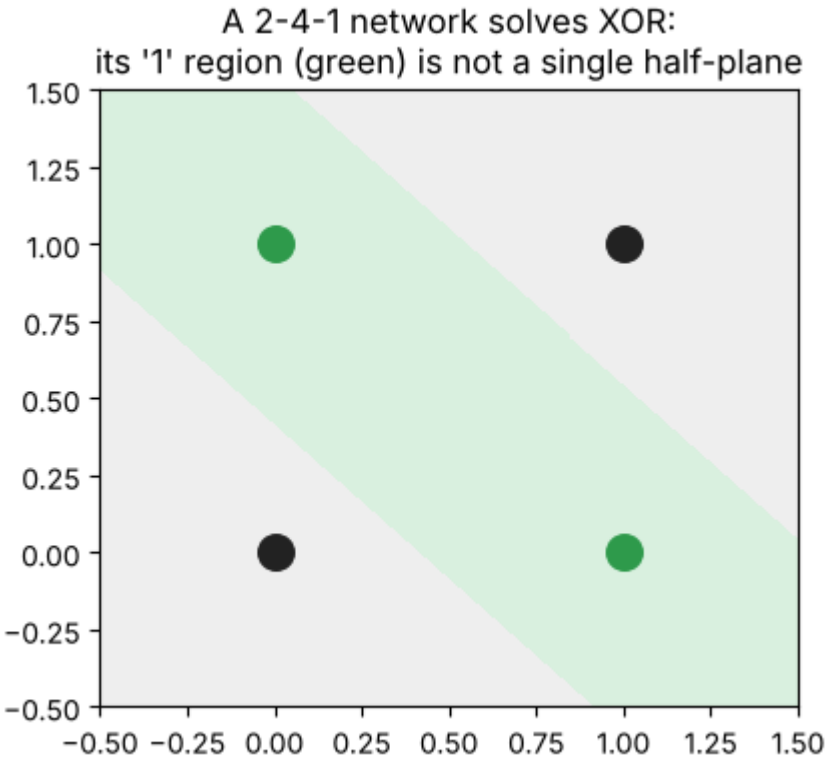


- **AND and OR:** one line separates the 1s from the 0s, and the perceptron learns them (4 epochs each in the notebook).
- **XOR:** the 1s sit on one diagonal and the 0s on the other, so **no straight line works**. The notebook tried **200,000 random lines**, and none beat **75%** (3 of 4 points). This is **Minsky & Papert's 1969** result, which helped start the AI winter (chapter 01).

  Zero margin, seen in the notebook: the learned AND line passes **exactly through the point (1, 0)**. z there is −0.000000000000000028, a floating-point rounding hair. The perceptron stops as soon as it makes no mistakes, however close a point sits to the line. Methods that **maximise the margin** (SVMs) exist to fix this.

The fix: hidden layers (multi-layer perceptron)

XOR = (x1 OR x2) AND NOT (x1 AND x2). Two hidden neurons (OR, NAND) and one output neuron (AND) solve it. The notebook does this with hand-set weights, then **learns** it with a 2-4-1 network trained by backpropagation (outputs 0.003, 0.997, 0.997, 0.003).



Stacking neurons lets the network **bend** its decision boundary into curves and regions, which is why deep networks exist. (Gradient training needs a **smooth** activation such as sigmoid instead of the step, see chapter 04.)

---

8. Summary (the lecture's revision box)

1. **Forward propagation:** $z = \Sigma\, w_i x_i + b$, then $\hat{y} = f(z)$.
2. **Step activation:** 1 if $z \geq 0$, else 0.
3. **Error:** $(y - \hat{y}) \in \{-1, 0, +1\}$.
4. **Update rule:** $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$ and $b \leftarrow b + \eta(y - \hat{y})$.
5. **Gradient descent:** repeated updates walk the weights downhill towards the minimum loss.
6. **Core limitation:** a single perceptron only solves **linearly separable** problems (AND, OR), **not XOR**.

Practice problems (with answers)

1. $x = (1, 4)$, $w = (0.5, -0.5)$, $b = 1$. Compute z and $\hat{y}$.
2. A perceptron predicts $\hat{y} = 1$ for $x = (2, 1)$ but $y = 0$, with $\eta = 0.5$, $w = (0.3, 0.6)$, $b = 0$. Give the new w and b.
3. With $w = (1, 1)$, what bias makes the perceptron compute AND? OR?
4. Why does the update rule never change the weights for a correctly classified example?
5. Explain in two sentences why XOR needs a hidden layer.
6. Your training loss bounces up and down and grows. What do you change?
7. Is the perceptron supervised or unsupervised? Why?

Answers

1. $z = 0.5 \cdot 1 + (-0.5) \cdot 4 + 1 = 0.5 - 2 + 1 = -0.5 \rightarrow \hat{y} = 0$.
2. Error $= 0 - 1 = -1$. $w_1 = 0.3 + 0.5 \cdot (-1) \cdot 2 = -0.7$. $w_2 = 0.6 + 0.5 \cdot (-1) \cdot 1 = 0.1$. $b = 0 + 0.5 \cdot (-1) = -0.5$.
3. AND: b between -2 and -1, e.g. $b = -1.5$ (only $1 + 1 = 2$ crosses zero). OR: b between -1 and 0, e.g. $b = -0.5$.
4. The error $(y - \hat{y})$ is 0, so the update $\eta \cdot 0 \cdot x$ is 0.
5. A single perceptron draws one straight line, and XOR's classes lie on opposite diagonals, so no line separates them. A hidden layer combines several lines (e.g. OR and NAND) into a non-linear region.
6. Lower the learning rate (the steps are overshooting the minimum).
7. Supervised: every training example comes with the correct label y, which the update rule uses.

04 · Activation functions (lectures of 07-08, 11-08 and 13-08-2026)

Part 1 (07-08): what an activation function is, why we need one, Step, Linear. **Part 2 (11-08):** Sigmoid, tanh, ReLU, Leaky ReLU, PReLU, vanishing gradients vs dying ReLU, which to use where. **Part 3 (13-08):** ELU, GELU, Swish/SiLU, Softmax, choosing output and hidden activations, ReLU sparsity (section 6).

Notebook: `notebooks/04_activation_functions.ipynb` (plots of every function and its slope, the linear-collapse proof, vanishing and dying gradients measured, an honest comparison; section 6 covers Part 3)

---

1. What is an activation function?

> A mathematical function applied to a neuron's **weighted sum**, turning it into the output passed to the next layer.

$z = \Sigma\, w_i x_i + b \rightarrow \text{output} = f(z)$

Inputs → weighted sum + bias → activation function → output

2. Why we need one

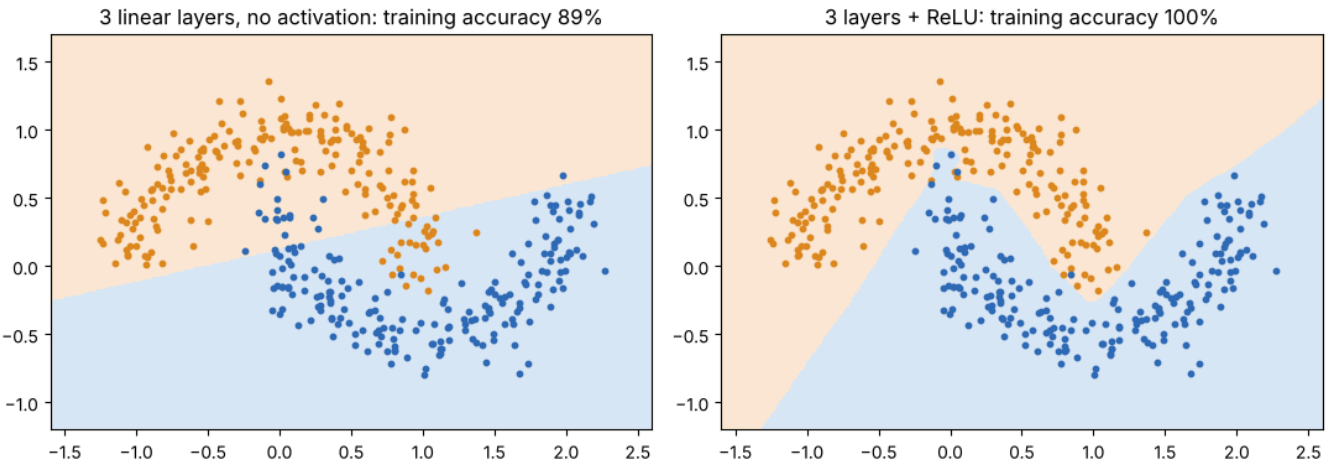
- **Non-linearity:** real problems are rarely linear, and non-linear activations let networks learn complex relationships.
- **Depth that means something:** stacking non-linear layers builds increasingly complex representations.
- **Useful decision boundaries:** curved, complex classification regions.
- **Controlled output ranges** (e.g. probabilities between 0 and 1).

Why linear layers alone are not enough

If every layer is linear, the whole network is **still one linear function**:

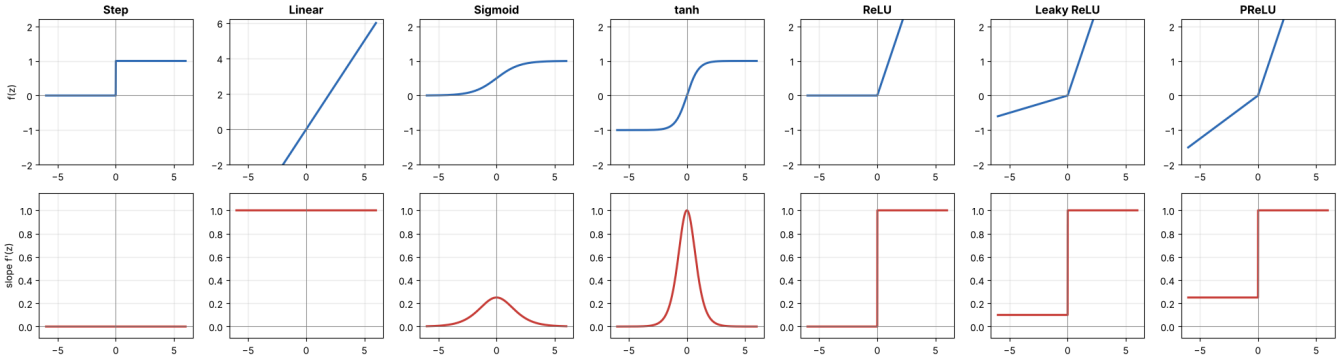
$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$

so adding more linear layers adds **no** power. The notebook **proves** it: a 4-layer linear network gives identical outputs to one collapsed layer, and a "deep" linear network on the two-moons data could only draw a straight line, while the same network with ReLU separated the moons.



---

3. The seven functions



Top row: the function. Bottom row: its slope (derivative), which is what backpropagation multiplies through the layers.

Step (Part 1)

- **f(z) = 1 if z ≥ 0, else 0.** A hard binary threshold (z = 2.5 → 1, z = −1.2 → 0).
- **Pros:** simple, a clear binary decision, historically important (the perceptron).
- **Cons: not differentiable at 0**, and its **derivative is 0 almost everywhere**, so it gives **no useful gradient** for backpropagation.
- **Use:** explaining the classic perceptron and hard binary decisions. **Not** used in modern deep networks.

Linear (Part 1)

- **f(z) = z.** Output proportional to input, unbounded.
- **Pros:** simple, cheap, easy to differentiate (slope 1).
- **Cons: no non-linearity**, so a network made only of linear layers stays linear. Unsuitable for hidden layers.
- **Use:** the **output layer for regression** (predicting an unbounded continuous number, e.g. a price).

	Step	Linear
output	discrete (0/1)	continuous

	Step	Linear
shape	threshold	straight line
differentiable	no (at 0)	yes
non-linear	yes	**no**
hidden layers	generally not used	generally not used
typical use	classical perceptron	regression output

Sigmoid (Part 2)

- **$\sigma(z) = 1 / (1 + e^{-z})$**, with derivative **$\sigma(z)\cdot(1 - \sigma(z))$**.
- Range **(0, 1)**, an S-shape. σ(0) = 0.5.
- Smooth, but **not zero-centred**. It **saturates** near 0 and 1, and its **maximum slope is only 0.25**.
- **Pros:** probability-like output, smooth, good for binary decisions and **gates** (LSTMs).
- **Cons: vanishing gradients** when saturated; not zero-centred; a poor default for deep hidden layers; the exponential is costlier than piecewise functions.
- **Use:** the **output layer of binary classifiers**.

tanh (Part 2)

- **$\tanh(z) = (e^{z} - e^{-z})/(e^{z} + e^{-z})$**, with derivative **$1 - \tanh^2(z)$**.
- Range **(−1, 1)**, **zero-centred** (tanh(0) = 0), the key difference from sigmoid. Largest slope (1) at 0, and it **saturates** near ±1.
- **Pros:** zero-centred outputs, bounded hidden states, historically important in RNNs.
- **Cons:** vanishing gradients when saturated, so large inputs slow learning. Often replaced by ReLU-family activations in deep feed-forward networks.
- **Use: RNN hidden states, LSTM/GRU state computations**, anywhere bounded zero-centred values are wanted.

ReLU, Rectified Linear Unit (Part 2)

- **f(z) = max(0, z).** Range **[0, ∞)**, piecewise linear. Slope **1** for z > 0 and **0** for z < 0.
- Negative inputs give exactly 0, so activations are **sparse**.
- **Pros: fast**; the strong default for deep hidden layers; **no saturation on the positive side**; sparsity can help.
- **Cons: dying ReLU** (a neuron stuck on the negative side stops learning); zero gradient for negatives; not zero-centred; unbounded positive output.
- **Use: hidden layers in CNNs and most feed-forward networks**.

Leaky ReLU (Part 2)

- **f(z) = z if z > 0, else α·z** (small fixed α, e.g. 0.01). Slope 1 or α.
- **Pros:** gradient still flows for negative inputs, **less prone to dying neurons**, simple and cheap.
- **Cons:** you must **choose α**, which may not suit every dataset; still not zero-centred.
- **Use:** hidden layers when ReLU leaves too many neurons inactive.

PReLU, Parametric ReLU (Part 2)

- **f(z) = z if z > 0, else a·z**, where **a is learned** during training (one per layer, or one per channel/unit).
- **Pros: learns the negative slope from the data**, reduces dying-neuron risk, more flexible than a fixed α.
- **Cons: extra trainable parameters**, can increase overfitting on small data, more complex, benefits depend on the architecture and data.
- **Use:** deep CNNs, and experiments where an adaptive slope may help.

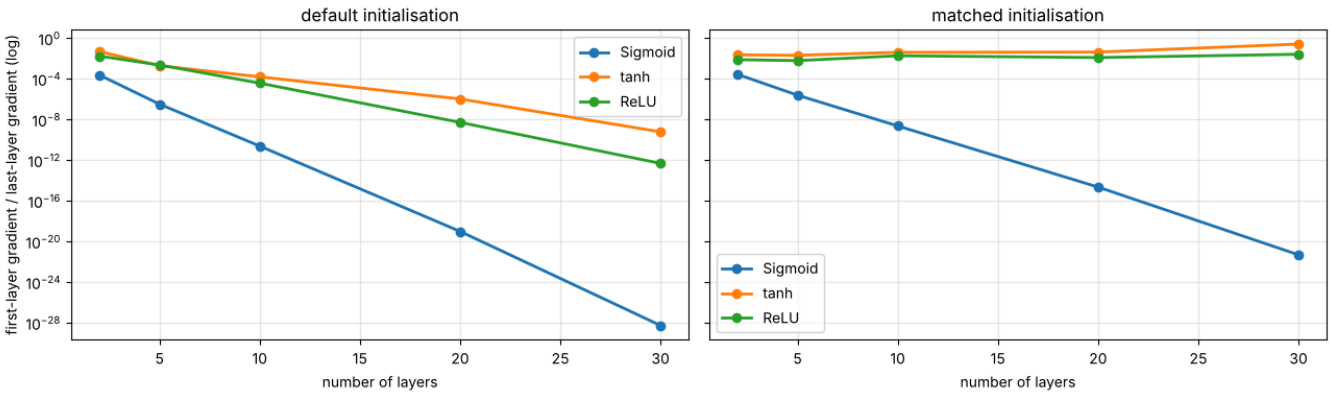
	ReLU	Leaky ReLU	PReLU
negative input →	0	α·z	a·z
negative-side gradient	0	fixed α	learned a
dying-neuron risk	higher	lower	lower
trainable slope	no	no	**yes**
core idea	rectify	leak a little	**learn** the leak

---

4. Vanishing gradient vs dying ReLU (don't mix them up)

	Vanishing gradient	Dying ReLU
cause	derivatives are **small** (sigmoid ≤ 0.25, saturated sigmoid/tanh ≈ 0) and get **multiplied** over many layers	the derivative is **exactly 0** for negative inputs
who suffers	sigmoid, tanh (in saturation)	ReLU
effect	**early layers** learn extremely slowly	a neuron whose input stays negative **never updates again**
remedies	ReLU-family activations, good initialisation, BatchNorm	Leaky ReLU, PReLU (non-zero negative slope), sensible learning rates

What the notebook measured



- **Vanishing gradient.** In a 30-layer **sigmoid** network the gradient reaching layer 1 was about **10^{-22}** of the last layer's, even with the best initialisation, so those layers can't learn. **tanh and ReLU kept a steady gradient only with matching initialisation**: Xavier for tanh, **He** for ReLU. With PyTorch's default starting weights, they also lost most

of the signal. So "ReLU fixes vanishing gradients" really means "ReLU **with He initialisation**".

- **Dying ReLU, one neuron.** With its bias pushed to −2, a ReLU neuron's gradient was **exactly 0** for 2,000 steps (dead forever). The Leaky ReLU neuron recovered and fitted the data.
- **Dying ReLU, a whole network** with 77% of its neurons knocked out (biases −0.35):
- ReLU stayed at chance (10%).
- Leaky ReLU with α = 0.01 *and* with α = 0.1 **also** stayed stuck in 60 epochs: the small leak through three layers was too weak for plain SGD.
- **PReLU learned its slope and recovered to 97.8%.**
- Leaky ReLU **reduces** the risk; it doesn't remove it.
- **PReLU's learned slopes** ended at 1.79, 1.34 and 2.06, from a start of 0.25. Nothing forces them to stay small.
- **Head-to-head** (6 hidden layers on digits, Adam, 3 seeds):
- sigmoid **27%**, tanh 95.2%, ReLU 91.0%, Leaky ReLU 93.1%, PReLU 94.6%, all with default initialisation;
- **ReLU + He initialisation 96.9% (best)**.

---

5. Which activation where (the lecture's table)

Situation	Recommended	Reason
binary classification **output**	**Sigmoid**	output between 0 and 1
regression **output**	**Linear**	unbounded continuous value
typical **hidden layers**	**ReLU**	fast, strong baseline (use He initialisation)
dying-ReLU concern	**Leaky ReLU**	non-zero negative gradient
adaptive negative slope wanted	**PReLU**	slope is learned
bounded, zero-centred hidden state	**tanh**	range −1 to +1
explaining the classic perceptron	**Step**	hard 0/1 threshold

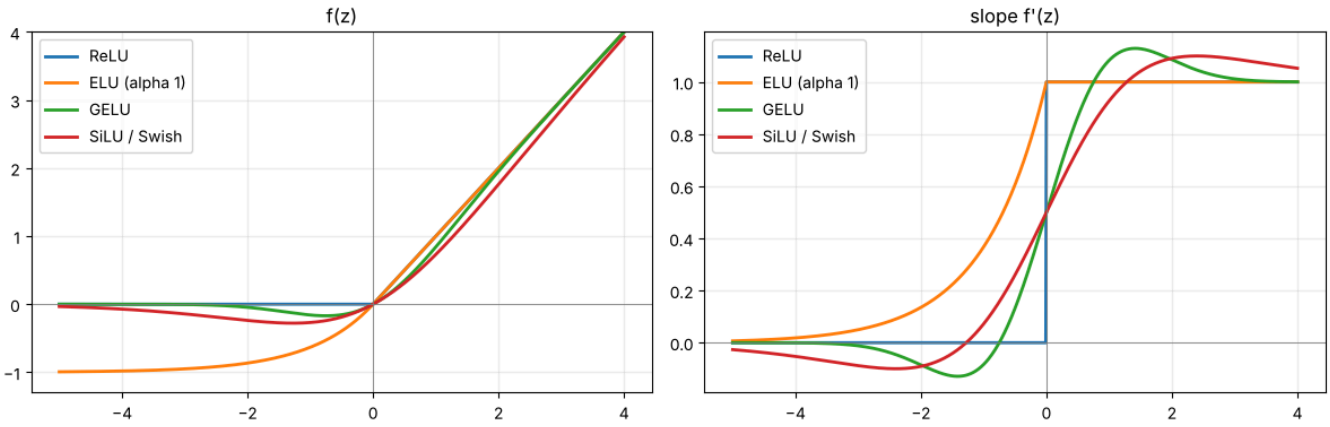
Function	Range	Zero-centred	Strength	Limitation	Typical use
Sigmoid	(0, 1)	no	probability-like output	saturation	binary output
tanh	(−1, 1)	**yes**	zero-centred	saturation	RNNs / bounded states
ReLU	[0, ∞)	no	fast and sparse	dying neurons	hidden layers
Leaky ReLU	(−∞, ∞)	no	negative gradient	fixed α	hidden layers
PReLU	(−∞, ∞)	no	learned negative slope	extra parameter	CNNs

Summary (lecture): - Sigmoid is mainly an **output** activation for binary predictions. - tanh is zero-centred, good for bounded hidden states, but saturates. - ReLU is the strong default for hidden layers and gives sparse activations. - Leaky ReLU keeps a small negative gradient against dying ReLU. - PReLU learns the negative slope, at the cost of extra parameters. -

Choose by layer, task and optimisation behaviour, and validate empirically. The notebook's results show why that last point matters.

---

6. Part 3 (13-08): ELU, GELU, Swish/SiLU and Softmax



Notebook: the three smooth ReLU relatives (left) and their slopes (right).

ELU: Exponential Linear Unit

ELU(z) = z if z > 0, else α·(eᶻ − 1) (α = 1 by default) - Linear positive side (like ReLU); a **smooth exponential** negative side that levels off at **−α**. In the notebook, ELU(−10) = −0.99995. - **Pros:** smooth; can output negative values, so the average output is closer to 0; its slope never hits exactly 0. - **Cons:** the exponential costs more than ReLU's max; **not always better** in practice. - **Use:** a smooth ReLU alternative when negative outputs help.

GELU: Gaussian Error Linear Unit

GELU(z) = z·Φ(z), where Φ is the probability that a standard normal variable is below z. - In plain words: **z is multiplied by a "how much to let through" factor between 0 and 1** that rises smoothly with z. Large positive z passes almost unchanged; large negative z is blocked; near 0 it's partial. That's the lecture's "**softly gates** inputs rather than hard-thresholding them". - Dips slightly below 0 (lowest value **−0.170 at z ≈ −0.75**, measured). - **The activation used in Transformer feed-forward blocks** (BERT, GPT). - **Cons:** more expensive and more mathematically complex than ReLU.

Swish / SiLU

SiLU(z) = z·σ(z) (the same idea as GELU, gating with the sigmoid instead of Φ). - Smooth and **non-monotonic**: it goes down to **−0.278 at z ≈ −1.28** and then back up towards 0, a small negative region. - Used in modern architectures (e.g. EfficientNet; LLaMA's SwiGLU blocks). - **Not guaranteed** to beat simpler choices.

z	−3	−1	−0.5	0	0.5	1	3
ReLU	0	0	0	0	0.5	1	3
ELU	−0.950	−0.632	−0.393	0	0.5	1	3
GELU	−0.004	−0.159	−0.154	0	0.346	0.841	2.996

z	−3	−1	−0.5	0	0.5	1	3
SiLU	−0.142	−0.269	−0.189	0	0.311	0.731	2.858

(values computed in the notebook)

Honest comparison (same 6-layer digits benchmark as Part 2, default initialisation, 3 seeds): ELU **95.1%**, GELU **93.4%**, SiLU **92.5%**, against ReLU 91.0%, tanh 95.2% and **ReLU + He initialisation 96.9%**. The newer functions beat plain ReLU here, but none beat ReLU with proper initialisation. **Newer is not automatically better**, which is the lecture's own closing rule.

Softmax: turning scores into a probability distribution

$p_i = e^{z_i} / \Sigma_j e^{z_j}$, applied to the **whole vector** of class scores (logits), not to each neuron separately.

The lecture's example, recomputed:

Class	Logit	e^z	Probability
cat	2.5	12.182	**0.723**
dog	1.2	3.320	**0.197**
horse	0.3	1.350	**0.080**
		sum 16.852	**1.000**

> **Handout correction:** the lecture's table says horse ≈ **0.07**. The correct value is **0.080** (0.0801 rounds to 0.08), and with 0.73 + 0.20 + 0.07 the probabilities wouldn't even sum to 1.

- Outputs are between 0 and 1 and **sum to 1**. Pairs naturally with **categorical cross-entropy** (chapter 06).
- **Not for hidden layers. Not for multi-label** problems: there, use one **independent sigmoid per label**. Notebook demo: for scores (3, 2.8, −2) on a photo with both a cat and a dog, softmax gives 0.548 / 0.449 / 0.004 (cat and dog forced to split), while sigmoids give 0.953 / 0.943 / 0.119 (both "yes").
- **Numerical stability:** e^1000 overflows. Naive softmax of (1000, 999, 998) gives `nan`; subtracting the maximum first gives (0.6652, 0.2447, 0.09), which is the correct answer, since the factor e^{−max} cancels. Built-ins (`torch.softmax`, `F.cross_entropy`) already do this.

ReLU sparsity

ReLU sets every negative input to **exactly 0**:

input	−3	−1	0.5	2	−4	1
ReLU	0	0	0.5	2	0	1

That's 3 zeros out of 6. In a real untrained ReLU layer on the digits, **51%** of the activations were exactly 0. Only some neurons "fire" for a given input, which is cheap and tends to give

cleaner features.

Why the shape matters (recap)

Backpropagation multiplies by the activation's slope. **Sigmoid/tanh saturate** (slope → 0 at both ends). **ReLU** doesn't saturate on the positive side, but its slope is **exactly 0** on the negative side. The smooth variants (ELU/GELU/SiLU) keep a small non-zero slope over part of the negative side.

Choosing the output activation

Task	Output	Why
binary classification	**sigmoid**	one probability in (0, 1)
single-label multi-class	**softmax**	one distribution over classes
multi-label	**sigmoid per label**	independent yes/no
unrestricted regression	**linear / identity**	any real number
positive-only regression	often **softplus** $\log(1 + e^z)$	guarantees > 0
bounded regression	**scaled sigmoid / tanh**	stays in a known interval

Choosing a hidden-layer activation

Situation	Start with	Why
general deep network	**ReLU** (+ He initialisation)	fast, simple, strong baseline
ReLU neurons are dying	**Leaky ReLU**	keeps a negative-side gradient
want the negative slope learned	**PReLU**	the slope is trainable
Transformer-style model	**GELU**	the architecture's standard
modern architecture that uses SiLU	**Swish / SiLU**	smooth and effective
bounded, zero-centred state	**tanh**	range −1 to +1

Rules of thumb (lecture): start with ReLU; binary output → sigmoid; single-label multi-class → softmax; unrestricted regression → linear; if ReLU neurons die → Leaky ReLU/PReLU and inspect the activations; tanh when a bounded zero-centred state is useful; GELU/SiLU when the architecture calls for them; **don't pick an activation because it's newer: validate it.**

Check yourself

1. Compute $\sigma(0)$, $\tanh(0)$, $\mathrm{ReLU}(-3)$, $\mathrm{LeakyReLU}(-3)$ with $\alpha = 0.01$.
2. What is the maximum derivative of sigmoid? Of tanh? Where do they occur?
3. Why can't you train a network of step functions with backpropagation?
4. Why is a network of only linear layers no better than one layer?
5. Explain the difference between the vanishing-gradient problem and dying ReLU.
6. Leaky ReLU vs PReLU: what is fixed and what is learned?
7. Choose activations for: hidden layers of a CNN; the output of a spam detector; the output of a temperature predictor; an LSTM's hidden state.

8. Which activation is zero-centred, sigmoid or tanh? Why does that matter?
9. Compute ELU(−1) and SiLU(0) and GELU(0).
10. Softmax of logits (2, 0): what are the probabilities?
11. An image can be tagged "beach", "sunset" and "people" at once. Softmax or sigmoids at the output? Why?
12. Why is naive softmax of (1000, 999, 998) `nan`, and how do you fix it?
13. Choose an output activation for: house price; a probability-of-default model; the number of visitors tomorrow (must be > 0).

Answers

1. 0.5, 0, 0, −0.03.
2. Sigmoid: 0.25 at z = 0. tanh: 1 at z = 0.
3. Its derivative is 0 everywhere except the jump, where it's undefined, so there's no gradient signal.
4. A composition of linear functions is linear: $W_2(W_1x + b_1) + b_2 = Wx + b$.
5. Vanishing: slopes are small but non-zero (saturating sigmoid/tanh) and shrink when multiplied across layers. Dying ReLU: the slope is exactly 0 because the neuron's input is always negative, so it never updates.
6. Leaky ReLU: the slope α is a fixed hyperparameter. PReLU: the slope a is a trainable parameter.
7. ReLU (Leaky ReLU or PReLU if neurons die); sigmoid; linear; tanh.
8. tanh. Zero-centred outputs keep the next layer's inputs balanced around 0, which helps optimisation (sigmoid outputs are always positive).
9. ELU(−1) = $e^{-1} - 1 = -0.632$. SiLU(0) = 0·0.5 = 0. GELU(0) = 0·0.5 = 0.
10. $e^2 = 7.389$; 7.389 / 8.389 = **0.881** and 1 / 8.389 = **0.119**.
11. Independent sigmoids: it's multi-label, so the labels shouldn't compete for a total of 1.
12. e^1000 overflows to infinity, and ∞/∞ = nan. Subtract the maximum logit first (the answer is unchanged).
13. Linear; sigmoid; softplus (or another positive transform).

05 · Normalization and regularization (lecture of 04-09-2026)

Notebook: `notebooks/05_normalization_and_regularization.ipynb` (scaling vs gradient descent, leakage, BatchNorm and LayerNorm by hand, every regulariser tested on an overfitting network)

---

1. Two different problems

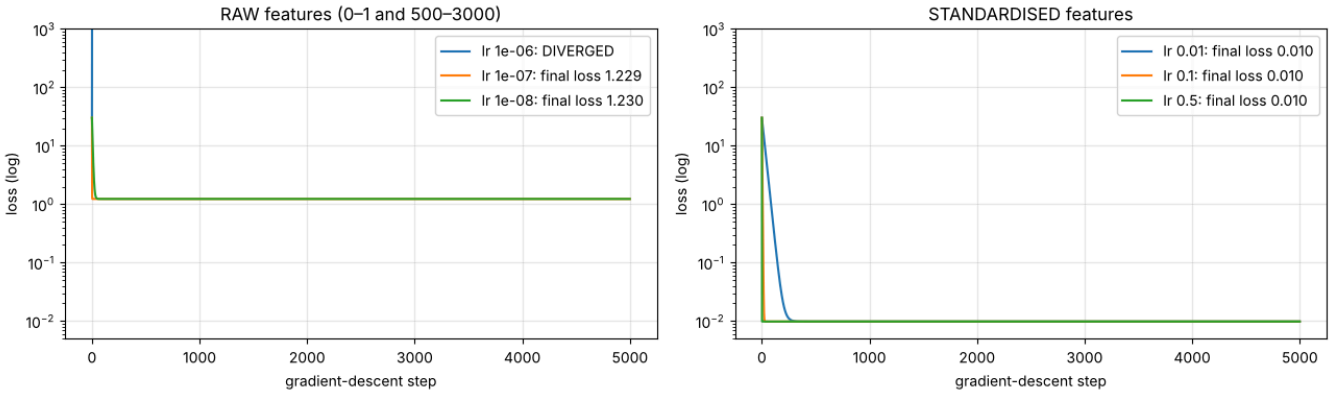
Concept	Main purpose	Examples
Normalization	control the **scale/distribution** of data and activations → **stable, efficient optimisation**	min-max scaling, standardisation, BatchNorm, LayerNorm
Regularization	control **model complexity**, reduce **overfitting** → **better generalisation** to unseen data	L1, L2, dropout, early stopping, augmentation, weight decay

They solve different problems and are **complementary**: a model can be normalised **and** regularised at the same time.

Part A · Normalization

2. Why normalisation is needed

If one feature lies in 0–1 and another in the millions, their gradients are on very different scales, and gradient descent struggles. Scaling makes the problem easy to navigate.



Notebook result: predicting a price from a 0–1 rating and a 500–3,000 floor area. - **Raw features:** learning rate 10^{-6} **diverged**; 10^{-7} and 10^{-8} **stalled** at loss 1.23 after 5,000 steps (the small feature barely learns at such tiny rates). - **Standardised features:** the same model reached the noise floor (loss 0.0098) in 199, 19, or even 1 step(s) at learning rates 0.01, 0.1, 0.5.

2.1 Standardisation (z-score)

$x' = (x - \mu) / \sigma$, with μ and σ from the **training** data. - centres the feature at 0, with standard deviation ≈ 1 on the training set; - a strong default for continuous numerical features.

2.2 Min-max scaling

$x' = (x - \text{min}) / (\text{max} - \text{min})$ → about [0, 1]. - Useful when a bounded range is wanted. - **Sensitive to outliers**, because the extremes set the range. Notebook: with one 1,000k salary among 99 normal ones, min-max squashed all 99 normal values into **5%** of the range.

2.3 Image normalisation

8-bit pixels **0–255 → 0–1** (divide by 255), so the optimiser works with manageable numbers.

2.4 Avoid data leakage

> Compute μ, σ (or min/max) from the **training set only**, then apply **those same values** to validation and test data.

Never compute normalisation statistics on the test set and then report test performance as if it were unseen. Information has leaked, and the score is optimistic.

3. Normalization layers (inside the network)

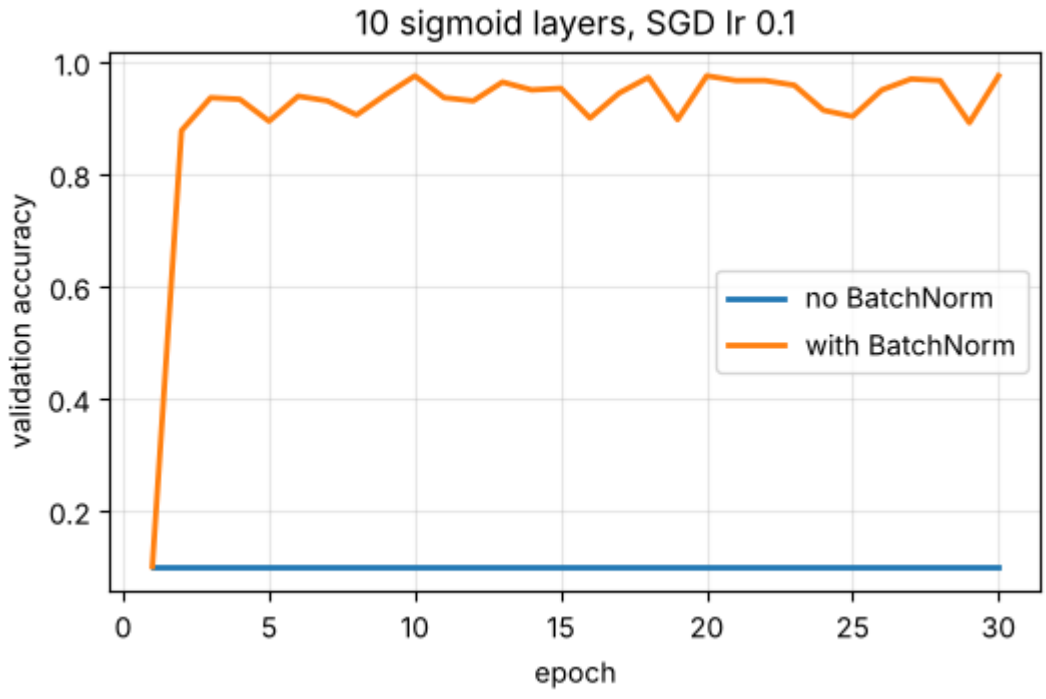
Method	Main idea	Common use
Batch Normalization	mini-batch statistics for each feature/channel	CNNs, many feed-forward networks
Layer Normalization	normalises the features **within each sample**	Transformers, sequence models
Instance Normalization	spatial features per sample/channel	style transfer, image generation
Group Normalization	groups of channels	CNNs with **small batches**

4. Batch Normalization

For each feature, over a mini-batch of size m: 1. **mean** $\mu_B = (1/m)\ \Sigma\ x_i$ 2. **variance** $\sigma^2_B = (1/m)\ \Sigma\ (x_i - \mu_B)^2$ 3. **normalise** $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)}$ 4. **scale and shift** $y_i = \gamma \cdot \hat{x}_i + \beta$, where **γ and β are learned**

Key points: - **Training:** uses **batch statistics**. - **Inference:** uses **moving (running) averages** collected during training. - **ε** prevents division by zero or by a tiny number. - **γ, β** keep the layer able to learn whatever scale and offset is useful (it can even undo the normalisation).

Notebook: - The hand computation matched `nn.BatchNorm1d` exactly. - In train mode, a "batch" of identical rows normalised to all zeros (variance 0). In eval mode the running statistics were used instead, which is why `model.eval()` matters. - A 10-layer **sigmoid** network learned **nothing** in 30 epochs without BatchNorm (10% = chance), but reached **89% after 5 epochs** and **97.5% after 30** with BatchNorm after every layer.



5. Layer Normalization

The same formulas, but the mean and variance are computed **across the H features of one sample**, then γ and β are applied.

BatchNorm	LayerNorm
relies on **batch** statistics during training	works **independently for each sample**
can be hurt by **very small** batches	handles small or variable batch sizes naturally
behaves differently in train and eval	identical in train and eval
common in **CNNs**	common in **Transformers**

Part B · Regularization

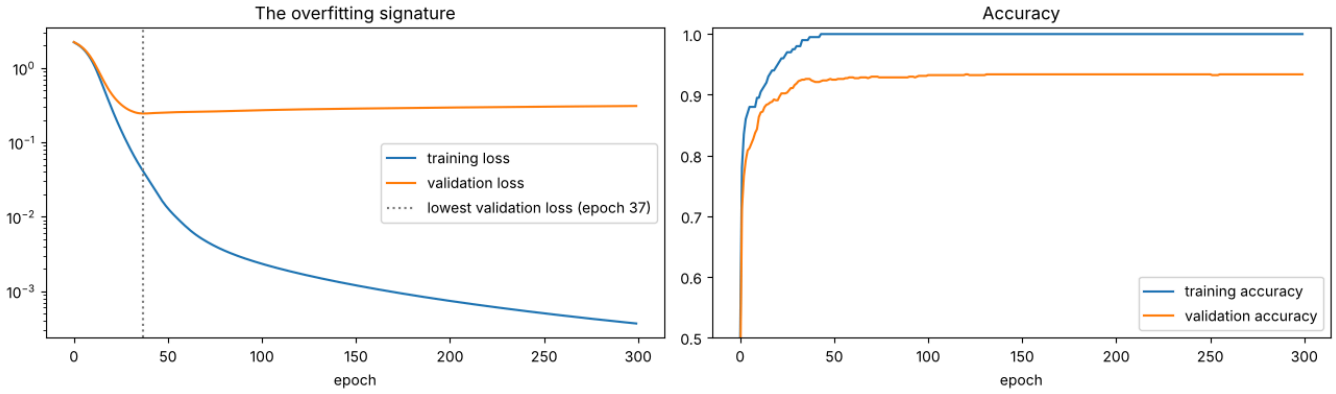
6. What is regularization?

> Methods that stop a network from **fitting noise or memorising** the training set, so it performs better on **unseen** data.

Condition	Training	Validation/Test
underfitting	poor	poor
good generalisation	good	good
overfitting	**very good**	**worsening / poor**

7. How to detect overfitting

> **Training loss keeps falling while validation loss stops improving or starts rising.**



Notebook: a 300,000-weight network on just 200 digit images. - **Training:** 100% accuracy (memorised). - **Validation loss:** bottomed out at 0.245 around epoch 37, then rose to 0.309 by epoch 300. - **Validation accuracy** just plateaued (93.4%), so **compare losses, not just accuracy**. Establish a clear **baseline** before adding regularisation.

8. L1 regularization

Loss = original loss + λ Σ $|w_i|$. The (sub)gradient adds λ·sign(w), a constant push towards zero. - Main effect: **sparsity** (many weights become exactly or nearly zero), so it's useful when you want feature selection. - Too large a λ removes too much capacity → **underfitting**.

9. L2 regularization

Loss = original loss + (λ/2) Σ w_i^2. The gradient adds **λ·w**, so larger weights shrink more. - Shrinks weights **smoothly** without forcing many to exactly zero. - The general-purpose default.

	L1	L2
penalty	$\|w\|$	$w^2/2$

	L1	L2
main effect	**sparsity**	**weight shrinkage**
exactly-zero weights	**more likely**	less likely
smooth at zero	no	yes
typical purpose	sparse models, feature selection	general weight control

Notebook: L1 ($\lambda = 10^{-4}$) left **88%** of the weights near zero, against 24% for L2 and 2% with no regularisation.

10. Elastic Net

Loss + $\lambda_1 \Sigma |w| + (\lambda_2/2) \Sigma w^2$: a compromise between L1's sparsity and L2's smooth shrinkage.

11. Dropout

Randomly **disable a fraction p** of activations on each training step. - The network can't depend on particular neurons, so it learns **distributed** representations. - **Inverted dropout:** the survivors are scaled by $1/(1 - p)$ during training, so nothing needs changing at test time. - **Disabled during inference** (`model.eval()`). - It is **stochastic** (a different mask each update). - **Too much dropout → underfitting.**

12. Early stopping

Stop training when **validation** performance stops improving, and keep the parameters from the **best validation epoch**. - **Patience** allows temporary ups and downs before stopping. - Use the **validation** set to decide, **never** the test set repeatedly.

Notebook: with patience 20, training stopped around epochs 56–70 instead of 300 (about 5× cheaper), with a similar test accuracy (93.2% vs 93.5%).

13. Data augmentation

Create **realistic variations** of the training examples: small rotations, crops, translations, flips, brightness changes. **The transformation must keep the label correct.** - More effective diversity, less overfitting, especially useful for small datasets. - **Wrong transformations create mislabelled or unrealistic examples.** Flipping a "6" can make a "9", for instance. In the notebook, 1-pixel shifts on tiny 8×8 digits (12% of the width, with wrap-around) **hurt** test accuracy (91.8% vs 93.5%): a live example of this warning.

14. Weight decay

Shrink the parameters directly at each step. - **SGD:** $w \leftarrow w - \eta(\nabla L + \lambda w) = (1 - \eta\lambda)\cdot w - \eta\nabla L$. That is "shrink, then step", which for plain SGD is equivalent to L2. - **Adam:** adding L2 to the loss is *not* the same as true weight decay, because Adam rescales each gradient. **Decoupled** weight decay applies the shrink directly, and that is the motivation for **AdamW**.

Notebook: - **Adam with** `weight_decay = 0.01` behaved like the L2 penalty: 24% of weights near zero. - **AdamW with the same 0.01** barely changed anything: each step only shrinks w by lr × wd = 0.001%. **Same number, different mechanism.** AdamW is normally used with larger values (e.g. 0.05–0.1).

All regularisers on one problem (notebook, 3 seeds, test accuracy)

Setting	Test accuracy	Weights ≈ 0
none	93.5%	2%
L2 ($\lambda = 10^{-3}$)	**94.1%**	24%
L1 ($\lambda = 10^{-4}$)	93.7%	**88%**
dropout p = 0.5	**93.9%**	2%
augmentation (1-px shift)	91.8%	2%
Adam weight_decay 0.01	93.7%	24%
AdamW weight_decay 0.01	93.5%	2%
early stopping	93.2%	—

Honest reading: with only 200 training images, gains are about half a point, the size of the seed noise. Regularisation **helps a little**; it's not magic. **More data** is the strongest cure for overfitting.

---

15. Normalization vs regularization

Question	Normalization	Regularization
primary objective	stable scale / optimisation	better generalisation
examples	standardisation, BatchNorm, LayerNorm	L1, L2, dropout, early stopping, augmentation, weight decay
prevents overfitting?	**not necessarily**	**yes, by design**
can be combined?	yes	yes

16. Practical selection guide (lecture)

Problem	Starting point	Reason
continuous features on very different scales	**standardisation**	comparable feature scales
image pixels 0–255	rescale to 0–1 (or the model's preprocessing)	manageable values
CNN with ordinary batch sizes	**BatchNorm**	effective activation normalisation
Transformer / sequence model	**LayerNorm**	independent of batch statistics

Problem	Starting point	Reason
overfitting in a dense network	**L2, dropout, early stopping**	control model complexity
need sparse weights	**L1**	many small or zero weights

Check yourself

1. Normalization vs regularization: one sentence each on what problem it solves.
2. Standardise x = 70 if the training mean is 50 and std is 10. Min-max scale x = 70 if min = 20, max = 120.
3. Why is min-max scaling risky with outliers?
4. What is data leakage in normalisation, and how do you avoid it?
5. Write the four BatchNorm steps. What are γ and β for? What changes at inference?
6. BatchNorm vs LayerNorm: what does each average over, and where is each used?
7. How do you recognise overfitting from training and validation curves?
8. L1 vs L2: penalty, effect on weights, typical use.
9. What does dropout do during training, and during inference?
10. Why does AdamW exist?

Answers

1. Normalization makes optimisation stable and efficient by controlling scale. Regularization reduces overfitting so the model generalises.
2. $(70 - 50)/10 = 2.0$. $(70 - 20)/(120 - 20) = 0.5$.
3. The extreme value sets the range, squashing all the normal values into a tiny interval.
4. Using statistics computed with test (or validation) data. Fit the scaler on training data only, then apply it to the others.
5. Batch mean → batch variance → normalise with ε → scale by γ and shift by β (learned, so the layer can choose its own scale and offset). At inference the running averages replace the batch statistics.
6. BatchNorm averages over the batch (per feature/channel) and is used in CNNs. LayerNorm averages over the features (per sample) and is used in Transformers.
7. Training loss keeps falling while validation loss stops improving or rises.
8. L1: $\lambda\Sigma|w|$, drives weights to zero (sparsity), used for feature selection. L2: $(\lambda/2)\Sigma w^2$, shrinks weights smoothly, the general default.
9. Training: randomly zeroes a fraction p of activations (scaling the rest by $1/(1 - p)$). Inference: off, so all neurons are used.
10. With adaptive optimisers like Adam, L2-in-the-loss is not true weight decay. AdamW decouples the decay and applies it directly to the weights.

06 · Loss functions (lecture of 18-08-2026)

Notebook: `notebooks/06_loss_functions.ipynb` (every worked example recomputed, MSE vs MAE with an outlier, why classification uses cross-entropy, sparse vs one-hot labels, PyTorch's versions and the double-softmax trap)

---

1. What is a loss function?

> A **loss function** tells a neural network **how wrong** its prediction is compared with the actual target. Training adjusts the weights and biases to make the loss **smaller**, using an optimizer such as SGD or Adam.

input → network → prediction ŷ → compare with the truth y → loss → optimizer updates the weights → repeat

Key idea: different problems need different losses, because "error" means different things: - **regression** (predict a number): how far off is the number? - **binary classification** (yes/no): how well does the probability match the answer? - **multi-class classification** (one of K classes): how much probability went to the correct class?

---

2. MSE: Mean Squared Error (regression)

$MSE = (1/n) \cdot \Sigma (y_i - \hat{y}_i)^2$

Here n = number of examples, y_i = actual value, $\hat{y}_i$ = predicted value, and $y_i - \hat{y}_i$ = the error. **The error is squared.**

Regression examples: house price (₹65,00,000), temperature (28.5 °C), salary, taxi fare, units sold.

Worked example (recomputed in the notebook)

Actual	Predicted	Error	Squared
100	90	10	100
200	220	−20	400
300	270	30	900

MSE = (100 + 400 + 900) / 3 = 466.67 (PyTorch's `F.mse_loss` agrees).

Why square the error?

Take errors of 2 and 10. The second is **5×** bigger, but after squaring (4 vs 100) it counts **25×** more. **MSE heavily penalises large errors.**

Advantages: punishes big mistakes strongly; smooth and differentiable (easy for gradient descent); simple; a good default for regression, especially when the noise is roughly Gaussian

(bell-shaped).

Main disadvantage: sensitive to outliers. With errors 2, 3, 4, 2, 3 and one error of 100, the notebook computed **MSE = 1673.7**, and the single 100 is **99.6%** of it. One bad point dominates.

Use MSE when: the target is continuous, large errors are especially costly, and there are no severe outliers.

---

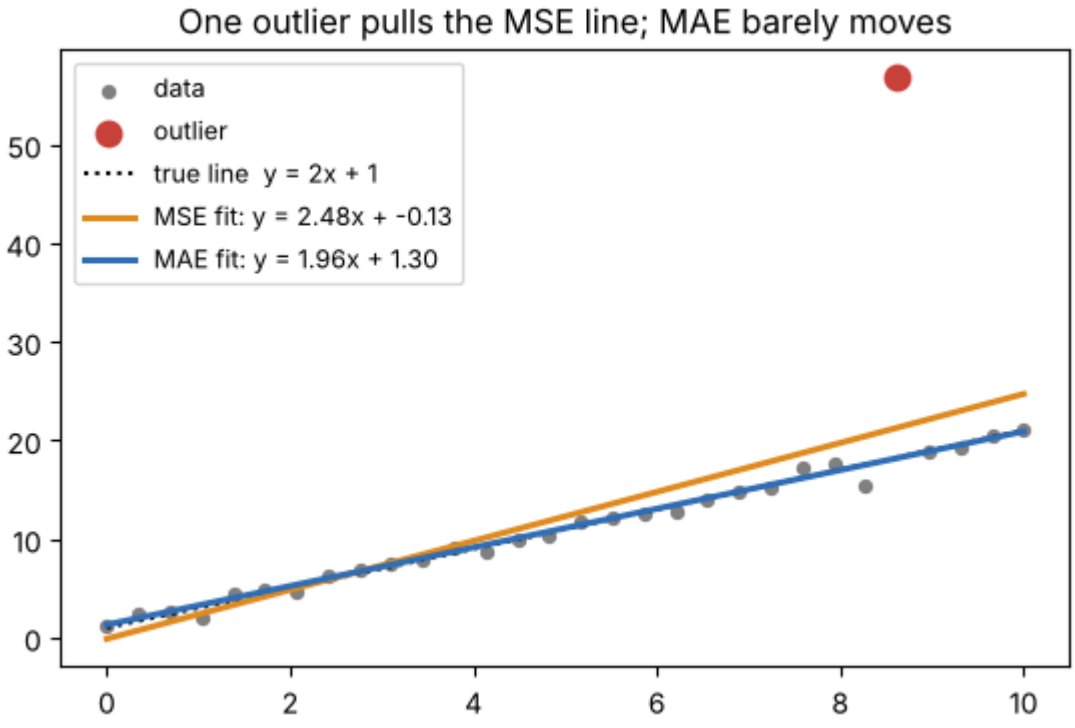
3. MAE: Mean Absolute Error (regression)

MAE = (1/n) · Σ |y_i − $\hat{y}_i$|

Same example: absolute errors 10, 20, 30 → **MAE = 60 / 3 = 20**.

	small error 2	**large error 10**	**outlier 100**
MSE contribution	4	100	10,000
MAE contribution	2	10	100

MAE grows **linearly** with the error, so it's **more robust to outliers**. For the errors 2, 3, 4, 2, 3, 100 it gives **MAE = 19.0**.



Notebook: fitting a line to data with one outlier. The MSE line is pulled towards the outlier; the MAE line stays close to the true line y = 2x + 1.

Advantages: less sensitive to outliers; in the **same units as the target** ("on average the prediction is off by ₹X"); treats all errors proportionally.

Disadvantages: |x| has a **sharp corner at 0**, so it isn't differentiable exactly there, and its gradient is always ±1 (it doesn't shrink near the answer). Optimizers cope, but behave

differently than on smooth MSE. It also doesn't prioritise very large errors.

	MSE	MAE
error calculation	squared	absolute
large errors	strongly penalised	penalised proportionally
outlier sensitivity	high	lower
smoothness	smooth	not differentiable at 0
units	squared target units	same units as the target
typical use	standard regression	robust regression

Rule: no big outliers and large errors are costly → **MSE**. Outliers present and you want robustness → **MAE**. *(Beyond the lecture: **Huber loss** is MSE for small errors and MAE for large ones, a popular compromise.)*

---

4. Binary cross-entropy, BCE (two classes)

Examples: spam / not spam, fraud / not fraud, disease / no disease, pass / fail. The target is **0 or 1**; the network ends with **one sigmoid** output p = the probability of class 1.

BCE = −[y·log(p) + (1 − y)·log(1 − p)] (averaged over the n examples)

If y = 1, the second term vanishes and **BCE = −log(p)**. Computed in the notebook:

Actual	Predicted p	Loss	Lecture's word
1	0.99	**0.010**	very small
1	0.90	**0.105**	small
1	0.60	**0.511**	moderate
1	0.10	**2.303**	very large
1	0.01	**4.605**	extremely large

BCE heavily punishes confident wrong predictions.

Reading a sigmoid output: 0.02 and 0.25 → probably class 0; 0.51 and 0.95 → probably class 1 (threshold 0.5).

Why not use MSE for classification? (measured)

The lecture says "classification is about probabilities and likelihood, not numerical distance". The notebook shows the practical consequence. Take y = 1 and a network that is **confidently wrong** (logit z = −5, so p = 0.007), and compare how hard each loss pushes the logit:

logit z	p	gradient with MSE	gradient with BCE
−5	0.007	**−0.0132**	**−0.9933**
−2	0.119	−0.1850	−0.8808
0	0.500	−0.2500	−0.5000

logit z	p	gradient with MSE	gradient with BCE
+2	0.881	−0.0250	−0.1192

With MSE, the sigmoid is saturated, so the push almost **vanishes exactly when the model is most wrong**. With BCE the gradient on the logit is simply **p − y**: the more wrong, the harder the push.

---

5. Categorical cross-entropy, CCE (K classes, one correct)

Handwritten digits 0–9: K = 10. The network ends with **10 outputs + softmax**, which gives a probability for each class, summing to 1 (chapter 04, section 6). The lecture's example output puts **0.82** on class 7, so the prediction is 7.

CCE = $-\Sigma_k\ y_k \cdot \log(p_k)$

With a **one-hot** label (7 → [0,0,0,0,0,0,0,1,0,0]), every term except the correct class is 0, so **CCE = −log(p_correct)**: - p(7) = 0.82 → **0.198** (small); - p(7) = 0.01 → **4.605** (confidently wrong, heavily punished).

CCE asks: how much probability did I give the correct class?

---

6. Sparse categorical cross-entropy

Same loss, different label format. This is where beginners get confused.

	Categorical CE	Sparse categorical CE
label for class 2 of 4	one-hot [0, 0, 1, 0]	the integer **2**
output layer	softmax	softmax
underlying idea	cross-entropy	cross-entropy

> **"Sparse" does NOT mean sparse predictions.** It means the **label** is stored as a class index instead of a one-hot vector.

MNIST's `y_train[0] = 7` is an integer, so use `sparse_categorical_crossentropy` (Keras). If you one-hot encode the labels first, use `categorical_crossentropy`. The notebook checks this: integer labels, one-hot labels and the formula by hand all give exactly **2.5375** on the same logits.

PyTorch equivalents (not in the lecture's Keras examples)

Keras	PyTorch	output layer
`mse` / `mae`	`nn.MSELoss` / `nn.L1Loss`	linear
`binary_crossentropy`	`nn.BCEWithLogitsLoss` (on raw scores, preferred) or `nn.BCELoss` (on sigmoid outputs)	sigmoid

Keras	PyTorch	output layer
`categorical_crossentropy` / `sparse_categorical_crossentropy`	`nn.CrossEntropyLoss` (accepts integer **or** one-hot/probability targets)	**none: feed the raw logits**

> ⚠️ **The most common PyTorch bug:** `nn.CrossEntropyLoss` **already applies (log-)softmax**. If your model ends in `nn.Softmax` too, softmax is applied twice. Notebook: for logits (2, 0, −2) with class 0 correct, the correct loss is **0.1429**, while the double-softmax version gives **0.6417** (wrong, with weak gradients). In PyTorch, end classification models with a plain `nn.Linear`.

---

7. The complete picture

Problem	Loss	Output layer	Target format
regression	**MSE**	linear	continuous
regression with outliers	**MAE**	linear	continuous
binary classification	**binary CE**	sigmoid	0 / 1
multi-class	**categorical CE**	softmax	one-hot
multi-class	**sparse categorical CE**	softmax	integer
multi-label (several true)	binary CE per label	one sigmoid per label	0/1 vector

Decision tree: predicting a number? → MSE (or MAE if outliers). Predicting a class? Two classes → BCE + sigmoid. More than two → softmax + CCE (integer labels → the sparse version).

What each loss is asking: - **MSE:** "How far is my number from the real number?" - **MAE:** "How much numerical error did I make, without over-weighting large errors?" - **BCE:** "How well does my probability match the correct yes/no class?" - **CCE:** "How much probability did I assign to the correct class?" - **Sparse CCE:** "The same, but my correct class is given as an integer."

Check yourself

1. Actual (10, 20), predicted (12, 14). Compute MSE and MAE.
2. Why is MSE sensitive to outliers? Why is MAE less so?
3. Which loss is in the same units as the target?
4. y = 1 and p = 0.5. What is the BCE?
5. y = 0 and p = 0.9. What is the BCE? Is that good or bad?
6. Why does sigmoid + MSE learn slowly when the model is confidently wrong?
7. A 4-class model outputs (0.1, 0.6, 0.2, 0.1) and the true class is 2 (the 0.2). What is the CCE?
8. Your labels are integers 0–9. Categorical or sparse categorical cross-entropy (Keras)?
9. Your PyTorch model ends with `nn.Softmax(dim=1)` and you train with `nn.CrossEntropyLoss`. What's wrong?

10. Choose loss + output layer for: taxi fare; email spam; handwritten digit; a photo's tags (several allowed).

Answers

1. Errors −2 and 6. MSE = (4 + 36) / 2 = **20**. MAE = (2 + 6) / 2 = **4**.
2. MSE squares errors, so one huge error dominates the sum. MAE grows only linearly with the error.
3. MAE (MSE is in squared units).
4. −ln 0.5 = **0.693**.
5. −ln(1 − 0.9) = −ln 0.1 = **2.303**: bad (confident and wrong).
6. The sigmoid is saturated, so its slope is ≈ 0, and MSE's gradient (which includes that slope) nearly vanishes. BCE's gradient on the logit is p − y, which stays large.
7. −ln 0.2 = **1.609**.
8. Sparse categorical.
9. Softmax is applied twice (CrossEntropyLoss does its own). Remove the Softmax layer and feed raw logits.
10. MSE (or MAE) + linear; BCE + sigmoid; (sparse) CCE + softmax; BCE + one sigmoid per tag.

07 · How an MLP / ANN works (lecture of 20-08-2026)

Notebook: `notebooks/07_mlp_from_scratch.ipynb` (the lecture's small examples, a 2-layer MLP in plain NumPy using the lecture's backpropagation formulas, two independent proofs that the formulas are right, training on digits with steps/batches/epochs, underfitting vs overfitting, what the hidden neurons learn)

---

1. The big picture in one line

An MLP learns by repeating:

input → weighted sum → activation → … → prediction → loss → backpropagation (gradients) → update the weights → repeat

until the predictions are good enough.

---

2. ANN, neuron, MLP

- **ANN (Artificial Neural Network):** a computational model loosely inspired by biological neurons (chapter 02).
- **A neuron:** takes inputs, multiplies each by a weight, adds a bias, applies an activation: **$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b = w \cdot x + b$, then $a = f(z)$**
- **MLP (Multilayer Perceptron):** an ANN with **one or more hidden layers**. Every neuron in one layer connects to every neuron in the next, so it's also called a **fully connected** (dense) network.

The three parts: 1. **Input layer:** receives the features. It does **no learning**; it just passes the values on. 2. **Hidden layers:** learn patterns and representations. 3. **Output layer:** produces the final prediction.

Example: predicting whether a customer will leave (churn) from features such as age, income and usage: inputs → hidden layers → one sigmoid output "probability of churn".

One neuron, worked (recomputed in the notebook)

Inputs $x = (2, 3, 1)$, weights $w = (0.5, 0.2, -0.4)$, bias $b = 0.1$:

$z = 2(0.5) + 3(0.2) + 1(-0.4) + 0.1 = 1.0 + 0.6 - 0.4 + 0.1 =$ **1.3** → $\mathrm{ReLU}(1.3) =$ **1.3**.

Weights, bias and activations: why each is needed

- **Weights = importance.** A large weight on x_1 and a small one on x_2 means x_1 has much more influence. Weights are **learned**.
- **Bias = shift.** Like $y = m \cdot x + c$: the weight is the slope (m), the bias the shift (c). Without bias, z is forced to 0 whenever all the inputs are 0.

- **Activations = non-linearity.** Without them, layer after layer collapses into **one** linear layer: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$. The proof is in code in notebook 04. Common choices: **ReLU** in hidden layers, **sigmoid** for a binary output, **softmax** for multi-class (outputs add to 1, e.g. 0.1 / 0.7 / 0.2 = "class 2 at 70%"), tanh for bounded states.

---

3. The complete working, step by step

Steps 1–4: forward propagation

1. **Input:** the feature vector x enters the input layer.
2. **Weighted sum:** the first hidden layer computes $\mathbf{Z^{(1)} = X \cdot W^{(1)} + b^{(1)}}$, one z per neuron (the **pre-activations**).
3. **Activation:** $\mathbf{A^{(1)} = f(Z^{(1)})}$. E.g. ReLU on Z = [−2, 3] gives **[0, 3]**: the first neuron is off, the second passes 3.
4. **Next layer:** $A^{(1)}$ becomes the input of layer 2: $Z^{(2)} = A^{(1)}W^{(2)} + b^{(2)}$, $A^{(2)} = f(Z^{(2)})$, and so on through every hidden layer.

Step 5: output layer, depending on the problem

Problem	Output layer	Loss (chapter 06)
regression (house price)	linear	MSE / MAE
binary (churn yes/no)	sigmoid	binary CE
multi-class, one-hot labels	softmax	categorical CE
multi-class, integer labels	softmax	sparse categorical CE
multi-label	one sigmoid per label	binary CE

For multi-class, the predicted class is the one with the **largest probability**.

Step 6: loss

Compare the prediction with the target. Lecture example: $\mathbf{Y = [1, 0]}$, $\mathbf{\hat{Y} = [0.7, 0.3]}$: "somewhat correct but not perfect".

$L = -[1 \cdot \log(0.7) + 0 \cdot \log(0.3)] =$ **0.357** (notebook). Lower loss = closer to the target.

Step 7: backpropagation

> **Backpropagation = sending the gradient of the loss backwards through the network.** We don't send the raw error back; we send **gradients**.

Gradient: how much the loss changes when a parameter changes slightly, $\mathbf{\partial L/\partial w}$. - $\partial L/\partial w > 0$: increasing w would **increase** the loss. - $\partial L/\partial w < 0$: increasing w would **decrease** the loss.

The chain rule is the heart of it. If L depends on a, a on z, and z on w, then

$$\mathbf{\partial L/\partial w = \partial L/\partial a \cdot \partial a/\partial z \cdot \partial z/\partial w}$$

and since $z = w \cdot x + b$, $\partial z/\partial w = x$. Each layer multiplies in its own local derivative.

For a 2-layer MLP with softmax + cross-entropy (the notebook implements exactly these):

Forward	Backward
$Z^{(1)} = X\,W^{(1)} + b^{(1)}$	$\delta^{(2)} = \hat{\mathbf{Y}} - \mathbf{Y}$ (softmax + CE simplifies to this)
$A^{(1)} = \mathrm{ReLU}(Z^{(1)})$	$\partial L/\partial W^{(2)} = A^{(1)\mathsf{T}}\,\delta^{(2)}$, $\partial L/\partial b^{(2)} = \delta^{(2)}$
$Z^{(2)} = A^{(1)}\,W^{(2)} + b^{(2)}$	$\delta^{(1)} = (\delta^{(2)}\,W^{(2)\mathsf{T}}) \odot \mathrm{ReLU}'(Z^{(1)})$
$\hat{Y} = \mathrm{softmax}(Z^{(2)})$	$\partial L/\partial W^{(1)} = X^{\mathsf{T}}\,\delta^{(1)}$, $\partial L/\partial b^{(1)} = \delta^{(1)}$

($\odot$ = element-by-element multiplication. For a batch of m examples, average: the gradients get a factor 1/m and the bias gradients are summed over the batch.)

Are the formulas right? The notebook checked them twice: 1. **Numerical derivative:** nudge a weight by $\pm\varepsilon$ and measure $(L(w+\varepsilon) - L(w-\varepsilon)) / 2\varepsilon$. It agreed with backprop to **1.4×10^{-11}**. 2. **PyTorch autograd:** the same network's automatic gradients agreed to about **3×10^{-17}** (floating-point rounding).

So the lecture's equations are exactly what `loss.backward()` computes.

Step 8: update the weights

$w \leftarrow w - \eta \cdot \partial L/\partial w$ $b \leftarrow b - \eta \cdot \partial L/\partial b$ (η = learning rate)

Lecture example: $w = 2$, $\partial L/\partial w = 0.5$, $\eta = 0.1 \rightarrow w = 2 - 0.1 \times 0.5 =$ **1.95**.

Why subtract? The gradient points in the direction where the loss **increases**; we want to go the opposite way, downhill.

Learning rate: too small → tiny updates and very slow training; too large → overshooting, oscillating, unstable, may never converge. **0.001** is a common starting point for Adam, but it depends on the problem.

---

4. Steps, batches and epochs

Term	Meaning
batch	a group of examples processed together (e.g. 128)
training step / iteration	one batch: forward → loss → backprop → **one weight update**
epoch	one full pass over the whole training set

Lecture arithmetic, checked in the notebook: - 50,000 examples, batch 128 → 50,000 / 128 = 390.6 → **391 batches per epoch** (the last one is smaller). 15 epochs ≈ 15 passes over the data = **5,865 updates**. - 50,000 examples, batch 100 → 500 steps per epoch; 10 epochs = **5,000 updates**.

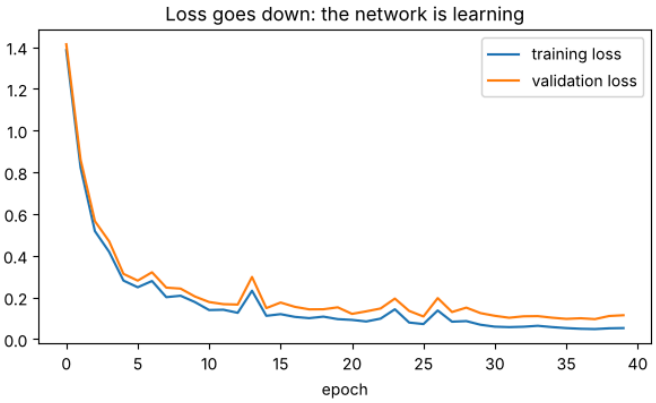
> **The weights are updated once per batch, not once per epoch.**

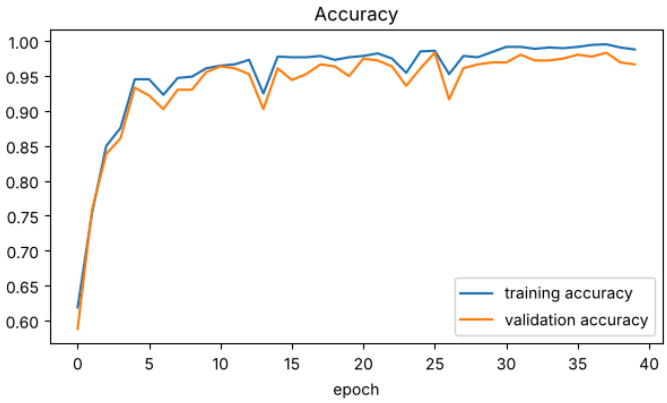
---

5. Training, validation and test sets

Set	Used for
training	learning the weights (backprop + updates)
validation	choosing hyperparameters, detecting overfitting, model selection, early stopping
test	the final, honest estimate on unseen data, **used once, never to update weights**

Notebook run (scikit-learn digits: 64 inputs → 64 ReLU → 10 softmax, written in NumPy): 1,078 training / 359 validation / 360 test images, batch 128 → **9 batches per epoch**, 40 epochs = **360 updates**. Validation accuracy reached **96.7%**; **test accuracy 96.4%**.





---

6. Underfitting and overfitting

	Training	Validation	Diagnosis
tiny network (2 hidden neurons)	66.3%	65.5%	**underfitting**: poor everywhere
64 hidden neurons, full data	99.8%	98.1%	**good fit**
big network, only 60 training images	100%	86.4%	**overfitting**: big gap

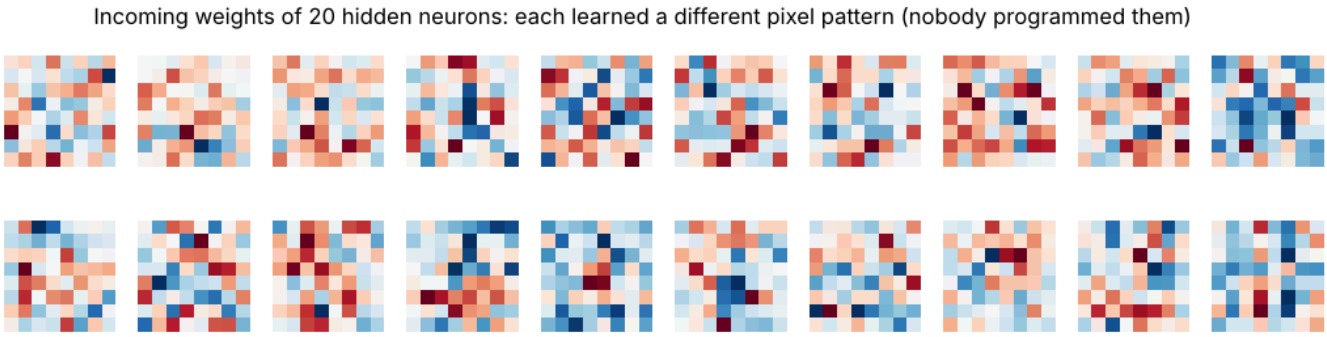
(notebook measurements)

- **Overfitting** = learned the training data too specifically. Fixes: dropout, L1/L2 regularisation, early stopping, data augmentation, more data, a smaller model (chapter 05).
- **Underfitting** = hasn't learned the relationship. Causes: model too small, too little training, poor features, too much regularisation, poor optimisation.

---

7. What does the network actually learn?

It does **not** learn explicit rules like "if Age > 40 and Income > X, predict 1". It learns **numbers**, the weights and biases, that turn hidden neurons into useful **feature detectors**, because those values lower the loss.



Notebook: each hidden neuron's 64 input weights drawn as an 8×8 image (red pushes the neuron up, blue down). Every neuron learned a different pattern. At this tiny resolution they look noisy rather than like neat strokes, but together they give 96% test accuracy.

Training = optimisation: find the weights and biases that minimise the loss.

Why several hidden layers? One hidden layer can in theory approximate many functions given enough neurons, but deep networks learn **hierarchical** representations: in images, edges → shapes → parts → objects. In tabular data the representations are less visual, but the idea is the same.

Two practical points the lecture recalls: - **Initialisation:** don't make all the weights equal (symmetry problem). Use Xavier for sigmoid/tanh and He for ReLU (chapter 08). - **Normalise the inputs:** features on wildly different scales (age ≈ 40 vs income ≈ 5,00,000) slow training. For images, divide pixels by 255 so 0–255 becomes 0–1.

---

8. Backpropagation vs the optimizer (don't mix them up)

- **Backpropagation computes** the gradients.
- The **optimizer** decides how to **use** them: plain gradient descent ($w \leftarrow w - \eta \cdot \text{grad}$) or **Adam**, which keeps running averages of the gradients (first moment) and of their squares (second moment) to adapt each weight's step size.

Forward propagation	Backward propagation
input → output	loss → earlier layers
computes predictions	computes gradients
uses weights, biases, activations	uses the loss gradient and activation **derivatives**
ends with the loss	starts from the loss
"What did we predict?"	"How should we change?"

The eight questions to remember (lecture)

1. **Input:** what information did I receive?
2. **Weighted sum:** what does each neuron compute from the inputs?
3. **Activation:** what non-linear pattern does this neuron represent?
4. **Prediction:** what does the network think?
5. **Loss:** how wrong is it?
6. **Backpropagation:** which parameters caused the error, and by how much?

7. **Optimisation:** how should the parameters change?
8. **Repeat** until the model is good enough.

Check yourself

1. $x = (1, -1, 2)$, $w = (0.3, 0.5, 0.1)$, $b = -0.2$, ReLU. Compute z and the output.
2. Does the input layer learn anything?
3. $Y = [0, 1, 0]$, $\hat{Y} = [0.2, 0.5, 0.3]$. Cross-entropy loss?
4. In softmax + cross-entropy, what is δ at the output layer?
5. Why does $\delta^{(1)}$ contain $\mathrm{ReLU}'(Z^{(1)})$? What happens to neurons whose z was negative?
6. $w = 0.8$, $\partial L/\partial w = -2$, $\eta = 0.05$. New w? Did w go up or down, and why?
7. 60,000 examples, batch size 200, 5 epochs. Steps per epoch? Total weight updates?
8. Training 99%, validation 70%. What's happening, and name three fixes.
9. Which dataset do you use to pick the learning rate? Which one only once at the end?
10. What is the difference between backpropagation and the optimizer?

Answers

1. $z = 0.3 - 0.5 + 0.2 - 0.2 = \mathbf{-0.2} \rightarrow \mathrm{ReLU} = \mathbf{0}$.
2. No: it only passes the feature values on.
3. $-\log(0.5) = \mathbf{0.693}$.
4. $\mathbf{\hat{Y} - Y}$.
5. Chain rule through the activation. $\mathrm{ReLU}' = 0$ where $z < 0$, so those neurons pass **no gradient** back (and their incoming weights don't change for that example).
6. $w = 0.8 - 0.05 \times (-2) = \mathbf{0.9}$: it went **up**, because the negative gradient says increasing w lowers the loss.
7. $60{,}000 / 200 = \mathbf{300}$ steps per epoch; $\times 5 = \mathbf{1{,}500}$ updates.
8. Overfitting. Fixes: more data or augmentation, dropout, L2, early stopping, a smaller model.
9. Validation; test.
10. Backpropagation computes the gradients; the optimizer turns them into weight updates (e.g. SGD, Adam).

08 · Weight initialization: Xavier (Glorot) and He (Kaiming) (lecture of 21-08-2026)

Notebook: `notebooks/08_weight_initialization.ipynb` (the lecture's numbers recomputed, the symmetry problem shown, a signal pushed through 10 layers with five different initializations, the same deep network trained four ways, PyTorch's defaults)

---

1. What is weight initialization?

> **Assigning the starting values of a network's weights before training begins.**

For a neuron $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$, the w's need *some* value before the first forward pass. Training then repeatedly applies **W_new = W_old − η·∂L/∂W**, so **initialization is the starting point of the whole optimisation**.

A good choice affects whether the network learns at all, how fast it converges, **vanishing** and **exploding** gradients, **symmetry** between neurons, and training stability.

---

2. Why not start with all zeros? The symmetry problem

If several neurons start with **identical weights**, they get the same inputs, so they compute the same outputs, receive the same gradients, and make the same updates. They stay **identical forever**, learning one feature between them.

Measured in the notebook (64 → 8 tanh hidden → 10 on the digits, 300 training steps):

Start	Training accuracy	Distinct hidden neurons
all weights 0	10.2%	**1 of 8**
all weights 0.01 (equal, not zero)	26.7%	**1 of 8**
random (PyTorch default)	**97.8%**	8 of 8

With all zeros it's even worse: tanh(0) = 0, so the gradient reaching the first layer is exactly zero and its weights never move.

- **Weights:** should *not* start equal or at zero (in hidden layers).
- **Biases:** can start at **zero**. Once the weights are random the neurons already differ.

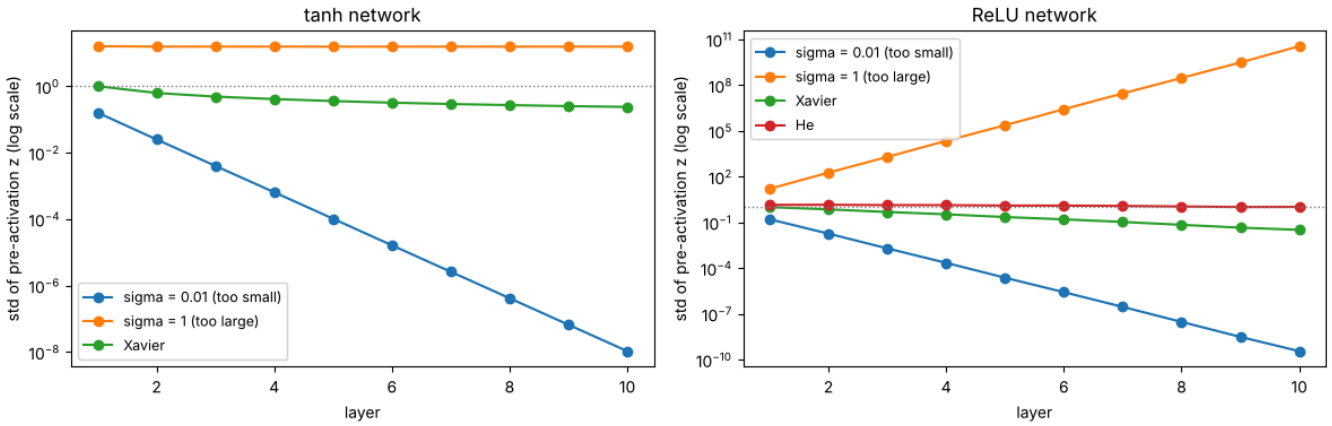
---

3. Why not just any random values?

Random values **break the symmetry**, but their **scale** matters: - **Too small** → each layer shrinks the signal, and repeated multiplication drives activations and gradients towards 0: **vanishing gradients**. - **Too large** → each layer amplifies it: **exploding gradients** (or saturated sigmoid/tanh, which also kills the gradient).

The goal of good initialization: keep the **variance** (spread) of the activations roughly constant going **forward**, and of the gradients going **backward**.

Seen in the notebook: a signal through 10 layers of width 256 (no training)



Initialization	Spread at layer 1	Layer 5	Layer 10	Verdict
tanh, σ = 0.01	0.16	0.0001	1×10^{-8}	**vanishes** (~6× smaller per layer)
tanh, σ = 1	16	15.6	15.6	**87% of neurons saturated** at ±1, so slope ≈ 0
tanh, Xavier	1.0	0.36	0.24	slow decline, usable
ReLU, σ = 0.01	0.16	2×10^{-5}	4×10^{-10}	**vanishes** (~9× per layer)
ReLU, σ = 1	16	235,000	3.6×10^{10}	**explodes** (~11× per layer)
ReLU, Xavier	1.0	0.22	0.033	shrinks ~0.7× per layer
ReLU, **He**	1.42	1.27	1.04	**stable**

(spread = standard deviation of the pre-activations z)

Why does the scale depend on the layer size? Each z adds up n_in terms, so its spread is about σ·√n_in times the input's. With n = 256, σ = 0.01 gives 0.16 (shrinks) and σ = 1 gives 16 (grows). **Xavier and He choose σ from the layer size to cancel that √n.**

---

4. Xavier / Glorot initialization (Glorot & Bengio, 2010)

Best for sigmoid and tanh. Uses both **n_in** (inputs to the layer) and **n_out** (its neurons).

- **Xavier normal:** W ~ N(0, σ²) with **$\sigma^2 = 2 / (n_{in} + n_{out})$**, so σ = √(2/(n_in + n_out)).
- **Xavier uniform:** W ~ U(−a, a) with **$a = \sqrt{6 / (n_{in} + n_{out})}$**.

Lecture example (n_in = 100, n_out = 50), checked in the notebook: Var = 2 / 150 = **0.0133**, σ ≈ **0.115**. Uniform limit a = √(6/150) = **0.200**.

Why the 6? A uniform distribution on $(-a, a)$ has variance $a^2/3$. Setting $a^2/3 = 2/(n_{in} + n_{out})$ gives $a^2 = 6/(n_{in} + n_{out})$. **Both versions have the same variance** (the notebook measured 0.0131 and 0.0133).

Why it suits sigmoid/tanh: they **saturate** for large $|z|$ (sigmoid's slope is at most 0.25 and ≈ 0 at the ends). Xavier keeps the initial z in the useful middle range so the signal and gradients don't die quickly.

---

5. He / Kaiming initialization (He et al., 2015)

Best for ReLU, Leaky ReLU, PReLU. Uses mainly **n_in**.

- **He normal:** $W \sim N(0, \sigma^2)$ with $\boldsymbol{\sigma^2 = 2 / n_{in}}$, so $\sigma = \sqrt{2/n_{in}}$.
- **He uniform:** $W \sim U(-a, a)$ with $\boldsymbol{a = \sqrt{6 / n_{in}}}$.

Lecture example ($n_{in} = 100$): Var = 2/100 = **0.02**, $\sigma = \sqrt{0.02} \approx$ **0.141**. Uniform limit $a = \sqrt{0.06} =$ **0.245**.

Why does ReLU need its own rule? ReLU sets every negative input to zero, so about **half** the signal's power is lost at each layer. He's factor **2** (instead of roughly 1) compensates for exactly that. The notebook shows this: Xavier with ReLU loses ~30% per layer, while He stays level.

---

6. Xavier vs He at a glance

	Xavier / Glorot	He / Kaiming
main purpose	keep activation/gradient variance stable	the same, **compensating for ReLU**
best for	sigmoid, tanh	ReLU, Leaky ReLU, PReLU
normal variance	$2 / (n_{in} + n_{out})$	$2 / n_{in}$
uniform range	$\pm\sqrt{6 / (n_{in} + n_{out})}$	$\pm\sqrt{6 / n_{in}}$
depends on	inputs **and** outputs	mainly inputs
typical use	networks with tanh/sigmoid	modern CNNs/MLPs with ReLU

Easy rule: sigmoid / tanh → Xavier. ReLU / Leaky ReLU / PReLU → He.

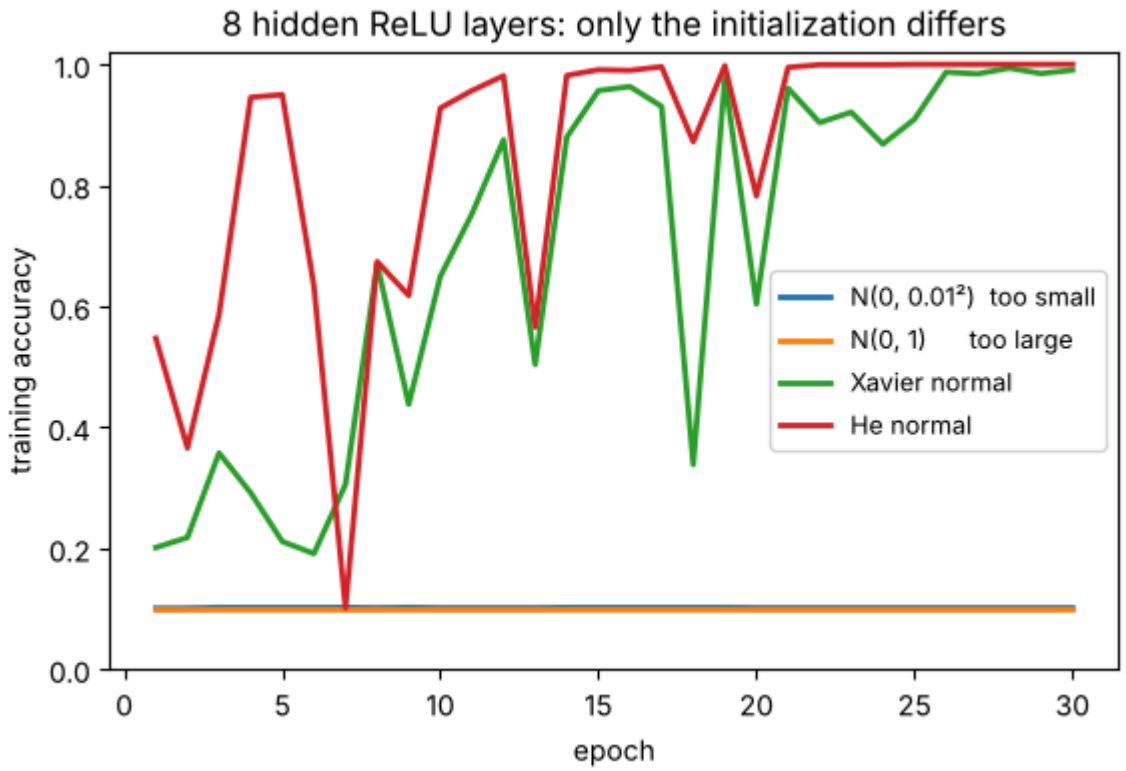
---

7. Does it really matter? (notebook experiment)

The same 8-hidden-layer ReLU network (width 128), the same data (digits), the same optimiser (SGD, lr 0.05, 30 epochs). Only the initialization changes:

Initialization	After epoch 1	After 30 epochs
$N(0, 0.01^2)$, too small	10.1%	**10.1%**, loss 2.3025 = ln 10: **never learns**
$N(0, 1)$, too large	loss `nan`	`nan` : numbers overflow, training crashes
Xavier normal	20.1%	99.1% (loss 0.051)

Initialization	After epoch 1	After 30 epochs
He normal	**54.7%**	**100%** (loss 0.0024)



Same network, same data: initialization alone decides between "never learns", "crashes" and "works". (Batch normalization and residual connections reduce this sensitivity in modern networks, but don't remove it.)

---

8. In PyTorch

```
nn.init.xavier_normal_(layer.weight)                          # or xavier_uniform_
nn.init.kaiming_normal_(layer.weight, nonlinearity="relu")    # He; or
kaiming_uniform_
nn.init.zeros_(layer.bias)

def init_he(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity="relu");
nn.init.zeros_(m.bias)
model.apply(init_he)            # applies to every layer
```

PyTorch's default for `nn.Linear` is **U(−1/√n_in, +1/√n_in)**, variance **1/(3·n_in)**. The notebook measured 0.00337 for n_in = 100 (formula: 0.00333). That's **6× smaller** than He's 2/n_in: fine for shallow networks, but set He explicitly for deep plain ReLU stacks. (This explains notebook 04's finding that ReLU did better with He than with the default.)

---

9. The complete chain (lecture)

initialization sets the scale of the weights → which sets the scale of the **activations** (forward) and **derivatives** (backward) → poor scaling gives **vanishing or exploding gradients** → which decides whether the **weight updates** are useful.

Summary: - Initialization gives the starting values; it happens **once, before** training. It is **not an optimizer**. - Zero (or equal) hidden weights → **symmetry problem**. - Arbitrary random values break symmetry but can still have a bad scale. - **Xavier** (variance 2/(n_in + n_out)) → sigmoid/tanh. **He** (variance 2/n_in) → ReLU family. - Good initialization keeps activations and gradients at useful scales.

Check yourself

1. A layer has n_in = 200 and n_out = 100. Compute the Xavier normal σ and the Xavier uniform limit.
2. Same layer with ReLU: He normal σ and He uniform limit?
3. Why can't all hidden weights start at 0? What exactly goes wrong?
4. Is it OK for biases to start at 0? Why?
5. Why does He use 2/n_in and not 1/n_in?
6. Your 10-layer ReLU network's loss stays at 2.30 on a 10-class problem from epoch 1. Most likely cause?
7. Your loss becomes `nan` in the first epoch. Name an initialization-related cause.
8. Which initializer for: a tanh RNN layer? a ReLU CNN? a GELU Transformer block (hint: ReLU-like)?
9. Show that U(−a, a) with a = √(6/n_in) has the same variance as N(0, 2/n_in).

Answers

1. σ = √(2/300) = **0.0816**; a = √(6/300) = **0.141**.
2. σ = √(2/200) = **0.1**; a = √(6/200) = **0.173**.
3. Every hidden neuron computes the same output and gets the same gradient, so they stay identical (the notebook measured 1 distinct neuron out of 8). With exact zeros and tanh, the first layer's gradient is even exactly 0.
4. Yes: once the weights are random the neurons differ, so zero biases create no symmetry.
5. ReLU zeroes about half its inputs, halving the signal's power; the factor 2 restores it.
6. Weights too small: the signal vanishes, so the output is uniform guessing (ln 10 = 2.30).
7. Weights too large: activations explode and overflow.
8. Xavier; He; He-style scaling is the usual choice for ReLU-like activations (in practice Transformers mostly use their own fixed small-std initialisation plus normalisation layers).
9. Var of U(−a, a) = $a^2/3$ = (6/n_in)/3 = 2/n_in. ✓

09 · Text → Tokens → Vectors: BPE, WordPiece, embeddings, positions (handout)

Notebook: `notebooks/09_text_tokens_vectors.ipynb` (a BPE trainer that reproduces the handout's merges, WordPiece scoring and greedy matching, **the real BERT tokenizer** on the handout's practical sentences, embedding lookup = one-hot × E, parameter and memory arithmetic, gradients reaching only the used rows, and why attention needs positions)

---

1. The big picture

A language model never sees words or letters. It sees **integer IDs**, turns them into **vectors**, and adds **position** information:

Stage	What happens	Example
1. Tokenization	text → pieces → IDs	"playing!" → ["play", "##ing", "!"] → [27, 91, 8]
2. Embedding	each ID picks a row of a trainable table	[27, 91, 8] → rows 27, 91, 8 → X, shape 3 × d
3. Position	add a position vector to each token vector	token vector + position vector → ordered input

Why it matters: tokenization affects cost, speed, how much fits in the context window, how typos and rare names are handled, code quality and **fairness across languages**. The embedding table can hold millions of parameters. The position design decides how the model sees order and distance.

Two separate learning processes (don't mix them up)

1. **Tokenizer training** learns a vocabulary and splitting rules from a text corpus, **before** the neural network is trained. Applying a finished tokenizer is deterministic: no gradient descent.
2. **Model training** uses backpropagation to learn the **embedding vectors** and every other weight.

   The tokenizer does **not** learn embeddings.

---

2. Why subwords?

Basic unit	Vocabulary	Sequence length	Advantage	Problem
whole words	very large	short	natural units	unknown words, typos, rare forms

Basic unit	Vocabulary	Sequence length	Advantage	Problem
characters / bytes	very small	long	any string representable	more positions, more computation
subwords	medium	medium	reusable pieces, few unknowns	quality varies by domain/language

Common words stay whole (playing, engineer); rare words reuse pieces (unplayable → un + play + able; playbot → play + b + ot).

The trade-off in two formulas: - **Embedding parameters = V × d** (V = vocabulary size, d = vector width). - **Attention work per layer ∝ n²·d** (n = number of tokens).

A bigger V shortens sequences but enlarges the table. A smaller V saves parameters but lengthens sequences. **If the token count doubles, the n² part of attention grows ~4×** (notebook: 100 → 200 → 400 tokens = 1× → 4× → 16×).

Quick check 1 (handout answers): spell-checker → characters/bytes or robust subwords (typos); source code → byte-level or code-trained subwords (new identifiers); a fixed list of 20 commands → whole words/commands are enough (closed domain).

---

3. Byte Pair Encoding (BPE)

Idea: start from single characters (or bytes) and **repeatedly merge the most frequent adjacent pair** into a new symbol. It's a greedy compression method: frequent patterns get their own token.

Training: 1. Split each word into characters, plus an end-of-word marker `</w>`. 2. Count every adjacent pair, **weighted by word frequency**. 3. Merge the most frequent pair everywhere. 4. Repeat until the merge budget or vocabulary size is reached. 5. Store the base symbols and the **ordered** merge rules.

> BPE pieces are **not** guaranteed to be meaningful prefixes or suffixes. They're whatever is frequent.

Worked example (reproduced exactly by the notebook)

Corpus: low ×5, lower ×2, newest ×6, widest ×3.

Initial counts: (e,s) = (s,t) = (t,) = **9** (newest ×6 + widest ×3); (w,e) = 8; (l,o) = 7. There's a **tie**, so a real implementation needs a deterministic tie-break. The handout picks (e,s), and so does Python's `max` (the first one found).

Merge	Pair	Count	Result
1	e + s	9	n e w **es** t
2	es + t	9	n e w **est**
3	est +	9	n e w **est**
4	l + o	7	**lo** w

Merge	Pair	Count	Result
5	lo + w	7	**low**
6–8	n+e, ne+w, new+est	6	**newest** is one token
9	low +	5	**low** is one token
10	w + i	3	**wi** d est

Exercise 2 (answer): after merges 1–2 the words are `l o w </w>` ×5, `l o w e r </w>` ×2, `n e w est </w>` ×6, `w i d est </w>` ×3. (l,o) still occurs **7** times; merging it gives `lo`, and then (lo,w), also 7, gives `low`.

Unseen words reuse pieces: lowest → `low` + `est</w>`; slow → `s` + `low</w>`. The worst case falls back to single characters, so **nothing is ever unknown**.

---

4. WordPiece (used by BERT)

Training intuition: association, not just frequency

Classroom score: **score(a, b) = count(ab) / (count(a) × count(b))**. The denominator penalises pairs made from pieces that are common on their own.

Pair	count(pair)	count(a)	count(b)	score
xy	10	100	20	10 / 2000 = **0.005**
ab	6	10	10	6 / 100 = **0.06**

WordPiece prefers **ab**, although xy is more frequent. *(The original method is defined by language-model likelihood gain; this score is a teaching approximation.)*

Encoding: greedy longest match, `##` = "continues the current word"

From the start of the word, take the **longest** vocabulary piece that matches, then continue with `##` pieces.

Vocabulary {play, ##ing, ##er, un, ##able, able}: playing → play + ##ing; player → play + ##er; unable → un + ##able.

Exercise 3 (vocabulary {read, ##ing, ##er, un, ##read, ##able}), run in the notebook: - reader → read + ##er ✓ - reading → read + ##ing ✓ - unreadable → un + ##read + ##able ✓ (**all three succeed**) - **Without** `##read` → `[UNK]`: after `un`, the rest must be matched by **continuation** pieces, and standalone `read` doesn't count.

> **Handout note:** the question asks which item "may fail", and its answer key says unreadable "may fail" but then shows that it **succeeds** with this exact vocabulary. The answer key contradicts itself. The notebook confirms that all three succeed, and that unreadable fails only if ##read is removed.

	BPE	WordPiece
training choice	most frequent adjacent pair	most strongly associated pair / likelihood gain
classroom score	count(a,b)	count(a,b) / (count(a)·count(b))
encoding new text	apply the learned merges in order	greedy longest vocabulary match
notation	varies (word-boundary markers)	BERT's `##` for continuations
purpose	the same: open-ended text with a manageable vocabulary	

---

5. The real BERT tokenizer (handout practical, run in the notebook)

Using the actual `bert-base-uncased` vocabulary (**30,522** tokens). `[CLS]` and `[SEP]` are special start/end tokens that BERT adds; counts include them.

Input	Tokens	Count	Observation
Tokenization is unbelievably useful.	token ##ization is **un ##bel ##ie ##va ##bly** useful .	12	even a normal word can split into 5 pieces
Tokenizashun is useful. *(typo)*	token ##iza ##shu ##n is useful .	9	**no `[UNK]`**: the typo splits into pieces
`vector<vector<int>> values;`	vector < vector < int > > values ;	11	each symbol is its own token
Thiruvananthapuram costs ₹1,23,456.	th ##ir ##u ##vana ##nt ##ha ##puram costs ₹ ##1 , 23 , 45 ##6 .	18	the place name → **7** pieces; the number → **7** tokens
playing player unplayable	playing player un ##play ##able	7	common words whole, rare ones in pieces

Fairness (handout "food for thought"), measured: "Tokenization is incredibly useful." = **6** tokens (without the specials), while the same sentence in Spanish, "La tokenización es increíblemente útil.", = **13** tokens, more than **twice** as many. The vocabulary was built mostly from English, so Spanish speakers pay more tokens (money, context window, speed) for the same meaning. **Token fertility** = tokens per word; it's high for out-of-domain text.

---

6. Embedding layers: a trainable lookup table

Token IDs are just names: ID 27 isn't "bigger" or "more meaningful" than ID 8. An **embedding matrix E** of shape **V × d** holds one trainable row per token:

token ID t → vector x = E[t] (length d); IDs [t_1 … t_n] → X of shape n × d

- **Lookup view:** IDs [27, 91, 8] → X = [E[27], E[91], E[8]], shape (3, d).

- **One-hot view:** E[t] = one_hot(t) · E. **Mathematically identical** (the notebook checks `torch.allclose` → True), but lookup is far cheaper: the one-hot route builds V numbers per token, almost all zeros, and multiplies by the whole table.

In PyTorch: `nn.Embedding(num_embeddings=V, embedding_dim=d)`; IDs of shape (batch, n) → output (batch, n, d).

Parameters and memory (checked in the notebook)

	V × d	Parameters	fp32 (4 bytes)	fp16/bf16 (2 bytes)
handout example	30,000 × 768	**23,040,000**	**92.16 MB** (≈ 87.9 MiB)	46.08 MB
Exercise 4	50,000 × 512	**25,600,000**	**102.4 MB**	51.2 MB
real bert-base-uncased	30,522 × 768	23,440,896	93.76 MB	46.88 MB

Exercise 4: E has shape **(50,000, 512)**. A batch of IDs of shape (B, n) = (8, 128) becomes **(8, 128, 512)**. Training needs extra memory for gradients, optimizer states and activations, and an untied output projection adds another V × d matrix.

How embeddings learn

The loss sends gradients back **only into the rows that were used**. For IDs **[4, 9, 4]**: rows 4 and 9 get gradients, and **row 4 accumulates two contributions**, $\partial L/\partial E[4] = \partial L/\partial x_1 + \partial L/\partial x_3$. The notebook measured a row-4 gradient of 2.0 and a row-9 gradient of 1.0 per element. All other rows: zero. (So rare tokens learn slowly.)

Static vs contextual: E has **one** row for "bank". The later Transformer layers produce **different** vectors for "river bank" and "bank loan", because they mix in context.

Weight tying: a language model's output layer can reuse E: **logits = $h \cdot E^T$**. This saves V × d parameters and links the input and output token spaces (the widths must match).

---

7. Positions: why attention needs them

"dog bites man" and "man bites dog" contain the same tokens, and **the same token always gets the same lookup vector**. Self-attention (Q = XW_Q, K = XW_K, V = XW_V, softmax($QK^T/\sqrt{d_k}$)·V) is **permutation equivariant**: shuffle the input rows and the outputs are shuffled the same way, **Attention(ΠX) = Π·Attention(X)**.

Notebook proof: without positions, every word's output vector is **identical** in both sentences (True), and any order-free summary is identical too. The model can't tell who bit whom. With position vectors added, the outputs differ (False).

Input construction: input_i = token_embedding[token_i] + position_embedding[i] (+ segment embedding). All have width d, so adding them keeps the shape n × d.

Learned absolute positions (BERT)

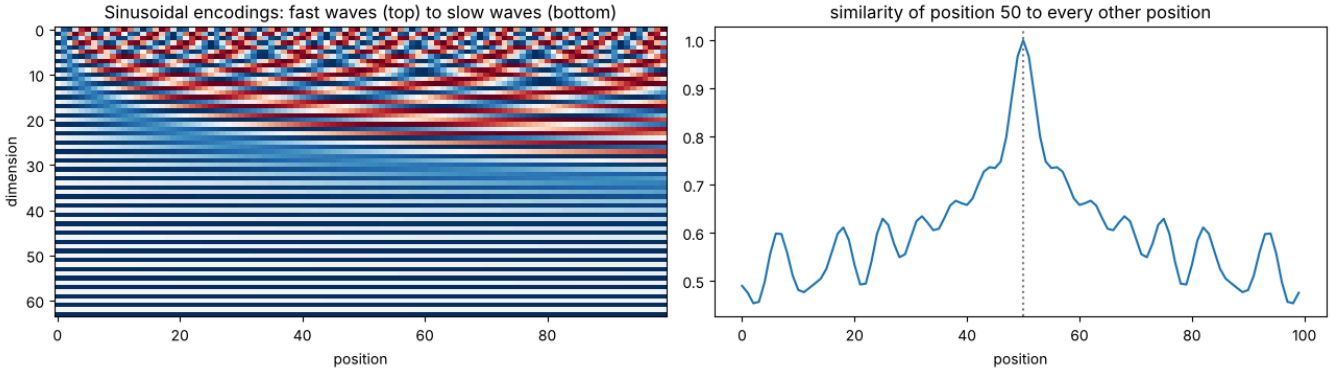
A trainable table **P** of shape **L_max × d**; position i uses row P[i]. Simple and flexible, but costs L_max × d parameters, and positions beyond L_max are undefined.

Sinusoidal encodings (original Transformer)

PE(pos, 2i) = sin(pos / 10000^(2i/d)) PE(pos, 2i+1) = cos(pos / 10000^(2i/d))

Each pair of dimensions is a wave of a different frequency (some fast, some slow), giving every position a unique multi-scale "barcode". They're fixed (no parameters), and nearby positions get similar codes.

Handout example, d = 4, position 1: [sin 1, cos 1, sin 0.01, cos 0.01] = **[0.8415, 0.5403, 0.0100, 0.99995]** (notebook ✓).



Modern alternatives: relative position biases and **RoPE** (rotary embeddings) inject position into the attention scores or rotate queries and keys. The goal is the same: make order and distance visible.

Exercise 5: (1) the same token at positions 2 and 10 has the **same token embedding**; only the position part differs. (2) The widths must match because the vectors are **added element by element**. (3) Content-only attention just **permutes its outputs** when the inputs are permuted; it can't infer an order.

---

8. Revision: the story in six statements

1. Whole-word vocabularies are brittle; character/byte sequences are long.
2. Subwords balance vocabulary size, sequence length and unknown words.
3. **BPE** greedily merges the most frequent adjacent pair.
4. **WordPiece** prefers strongly associated pairs and encodes by greedy longest match (`##`).
5. An **embedding layer** is a trainable V × d lookup table.
6. **Positions** are needed because content-only attention is permutation equivariant.

Common misconceptions (all false): a token is always a whole word · BPE and WordPiece are identical · the tokenizer learns the embeddings · an embedding is a readable definition · a token's identity already includes its position · a bigger vocabulary is always better.

Glossary: *token* (unit given to the model) · *vocabulary* (the finite set of token types with IDs) · *merge rule* (a BPE instruction combining two symbols) · *embedding* (a learned vector for an ID) · *d* (values per vector) · *contextual representation* (a later hidden state that depends on the neighbours) · *positional embedding* (learned) vs *encoding* (may be fixed) · *permutation equivariance* · *token fertility* (tokens per word).

Check yourself

1. Why not use one token per possible word?
2. V doubles and d stays fixed. What happens to the embedding parameters?
3. IDs [7, 2, 7]: which rows of E get gradients? Which gets more?
4. What information is missing from content-only self-attention?
5. Corpus: "aa" ×4, "ab" ×1 (characters, ignore `</w>`). What is BPE's first merge?
6. With vocabulary {un, ##happy, happy, ##ness}, segment "unhappiness" and "unhappy". Which fails?
7. V = 32,000, d = 4,096. Embedding parameters and fp16 size?
8. Why are token and position vectors added rather than placed side by side?
9. An 8,000-token context: is that a fixed number of words? Why not?

Answers

1. The set of possible words is open-ended: a huge table, and still unknown words, typos and names.
2. They double (V × d).
3. Rows 7 and 2; row 7 accumulates two contributions.
4. Word order (absolute and relative position).
5. (a, a): count 4 (only "aa" contains it) vs (a, b): 1. So merge a + a → "aa".
6. "unhappy" → un + ##happy ✓. "unhappiness" **fails → [UNK]**: after un, the rest ("happiness") must start with a ## piece, but ##happy doesn't match ("happi…" has an i, not a y) and ##ness doesn't start there. The standalone "happy" doesn't count after un.
7. 32,000 × 4,096 = **131,072,000** parameters; × 2 bytes = **262.1 MB** in fp16.
8. Adding keeps the width d (shape n × d), so every later layer is unchanged; both vectors must have the same width. (Concatenation would also work, but it widens the model.)
9. No: the number of words depends on the tokenizer, the language, the domain (code, numbers, names) and the spelling. The notebook showed Spanish needing 2× the tokens of English.

01 · AI vs ML vs DL vs GenAI, in code

Lecture: *AI vs ML vs DL vs GenAI & the Biological Neuron Analogy* (05-08-2026), Part A · **Notes:** `../notes/02_ai_ml_dl_genai_and_the_neuron.md`

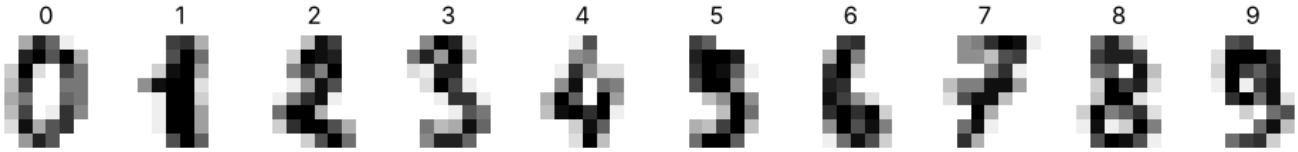
The lecture's hierarchy: **AI ⊃ ML ⊃ DL ⊃ GenAI**. The best way to feel the difference is to attack **one problem** at each level and compare:

Level	What we'll build	Who decides the features?
AI (rule-based, no learning)	hand-written if-then rules	a human writes the rules
ML (classical)	logistic regression on **hand-engineered** features	a human designs the features; the model learns weights
DL	a neural network on **raw pixels**	the network learns its own features
GenAI	a small generative network that **creates new digits**	learns the data distribution itself

The problem: scikit-learn's handwritten digits (1,797 images of 8×8 pixels, values 0–16). It ships with scikit-learn, so nothing to download. Everything runs in about a minute.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
digits = load_digits()
X, y, images = digits.data, digits.target, digits.images
X_tr, X_te, y_tr, y_te, img_tr, img_te = train_test_split(X, y, images, test_size=0.3,
random_state=0, stratify=y)
print("training images:", len(X_tr), " test images:", len(X_te))
fig, axes = plt.subplots(1, 10, figsize=(12, 1.6))
for d, ax in enumerate(axes):
    ax.imshow(images[y == d][0], cmap="gray_r"); ax.set_title(str(d)); ax.axis("off")
plt.show()
```

```
training images: 1257  test images: 540
```



Level 1 · AI without learning: hand-written rules

The 1960s–80s "expert system" way: a human looks at the data and writes rules. To keep it fair, we only try to tell **0 from 1** (the easiest pair). A plausible human rule: "a **1** is a thin vertical stroke, so its ink sits in the middle columns; a **0** is a ring, so it has ink at the left and right edges and a hole in the middle."

```
def rule_based_0_or_1(img):
    middle_ink = img[:, 3:5].sum()                # ink in the two centre columns
    side_ink = img[:, [1, 2, 5, 6]].sum()         # ink in the side columns
    centre_hole = img[3:5, 3:5].sum()             # a 0 has an empty centre
    return 1 if (middle_ink > side_ink * 0.5 and centre_hole > 10) else 0

mask = np.isin(y_te, [0, 1])
pred = np.array([rule_based_0_or_1(im) for im in img_te[mask]])
print(f"hand-written rules, 0 vs 1: {np.mean(pred == y_te[mask]):.1%} correct on {mask.sum()}
test images")
print("and for all 10 digits? We'd need rules for every digit and every handwriting style. Nobody
can write those by hand.")
```

```
hand-written rules, 0 vs 1: 97.2% correct on 109 test images
and for all 10 digits? We'd need rules for every digit and every handwriting style. Nobody can wr
ite those by hand.
```

That's **AI**: intelligent-looking behaviour from **rules a person wrote**, with no learning from data. It works for a narrow, clean task, but writing rules for all ten digits in every handwriting style is hopeless. That's the motivation for learning from data.

Level 2 · Machine learning with hand-engineered features

Classical ML: a **human chooses the features** (measurements), and the algorithm **learns** how to weigh them. We design 8 features a person might think of, then let logistic regression learn the weights.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

def engineered_features(img):
    ink = img > 4
    rows, cols = np.where(ink)
    return [img.sum(),                                    # total ink
            img[:4].sum() - img[4:].sum(),                # top-heavy or bottom-heavy?
            img[:, :4].sum() - img[:, 4:].sum(),          # left-heavy or right-heavy?
            img[3:5, 3:5].sum(),                          # ink in the centre (hole?)
            (cols.max() - cols.min()) if len(cols) else 0, # width
            (rows.max() - rows.min()) if len(rows) else 0, # height
            np.abs(np.diff(img, axis=0)).sum(),           # vertical edges
            np.abs(np.diff(img, axis=1)).sum()]           # horizontal edges

F_tr = np.array([engineered_features(im) for im in img_tr]); F_te =
np.array([engineered_features(im) for im in img_te])
ml = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(F_tr, y_tr)
print(f"ML on 8 hand-engineered features, all 10 digits: {ml.score(F_te, y_te):.1%}")
```

```
ML on 8 hand-engineered features, all 10 digits: 74.3%
```

It **learned** from data and handles all 10 digits, which is much better than rules. But it's limited by the **features a human thought of**. Ask yourself: what would be a good feature to tell 3 from 8, or 4 from 9? That's the human bottleneck the lecture calls **manual feature engineering**.

What if we give the same ML algorithm the **raw pixels** (64 numbers) instead?

```
ml_raw = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000)).fit(X_tr, y_tr)
print(f"the same logistic regression on 64 raw pixels: {ml_raw.score(X_te, y_te):.1%}")
```

```
the same logistic regression on 64 raw pixels: 97.2%
```

On tiny, centred, clean 8×8 digits, raw pixels are already a good representation, so the linear model does well. On real photos (millions of pixels, objects anywhere, varying lighting), raw pixels fed to a linear model work badly, and that is exactly where deep learning's **learned features** make the difference.

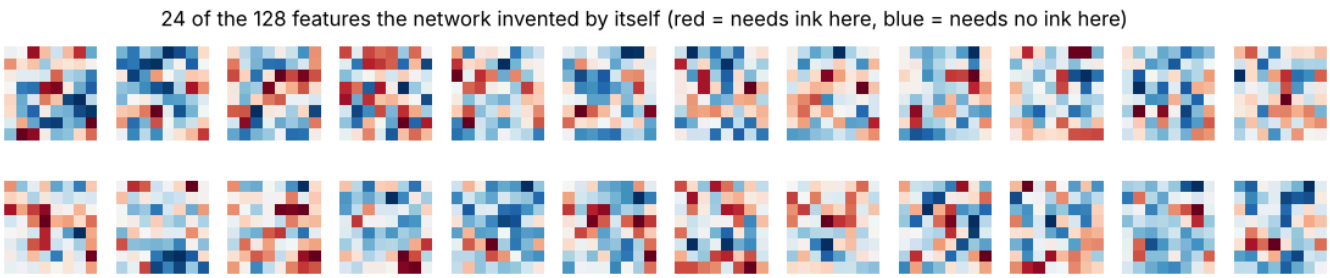
Level 3 · Deep learning: learn the features from raw pixels

A neural network with hidden layers takes the **raw pixels** and learns its own internal features in the hidden layers.

```
torch.manual_seed(0)
Xt_tr, Xt_te = torch.tensor(X_tr / 16, dtype=torch.float32), torch.tensor(X_te / 16,
dtype=torch.float32)
yt_tr, yt_te = torch.tensor(y_tr), torch.tensor(y_te)
dl = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 64), nn.ReLU(), nn.Linear(64,
10))
opt = torch.optim.Adam(dl.parameters(), lr=1e-3)
for epoch in range(60):
    perm = torch.randperm(len(Xt_tr))
    for i in range(0, len(perm), 64):
        idx = perm[i:i + 64]
        loss = nn.functional.cross_entropy(dl(Xt_tr[idx]), yt_tr[idx]); opt.zero_grad();
loss.backward(); opt.step()
with torch.no_grad():
    dl_acc = (dl(Xt_te).argmax(1) == yt_te).float().mean().item()
print(f"deep network (2 hidden layers) on raw pixels: {dl_acc:.1%}")

# What did the first hidden layer learn? Each neuron's 64 weights, drawn as an 8x8 picture
W1 = dl[0].weight.detach().numpy()
fig, axes = plt.subplots(2, 12, figsize=(14, 2.6))
for ax, w in zip(axes.ravel(), W1[:24]):
    ax.imshow(w.reshape(8, 8), cmap="RdBu"); ax.axis("off")
fig.suptitle("24 of the 128 features the network invented by itself (red = needs ink here, blue =
needs no ink here)")
plt.show()
```

```
deep network (2 hidden layers) on raw pixels: 97.6%
```



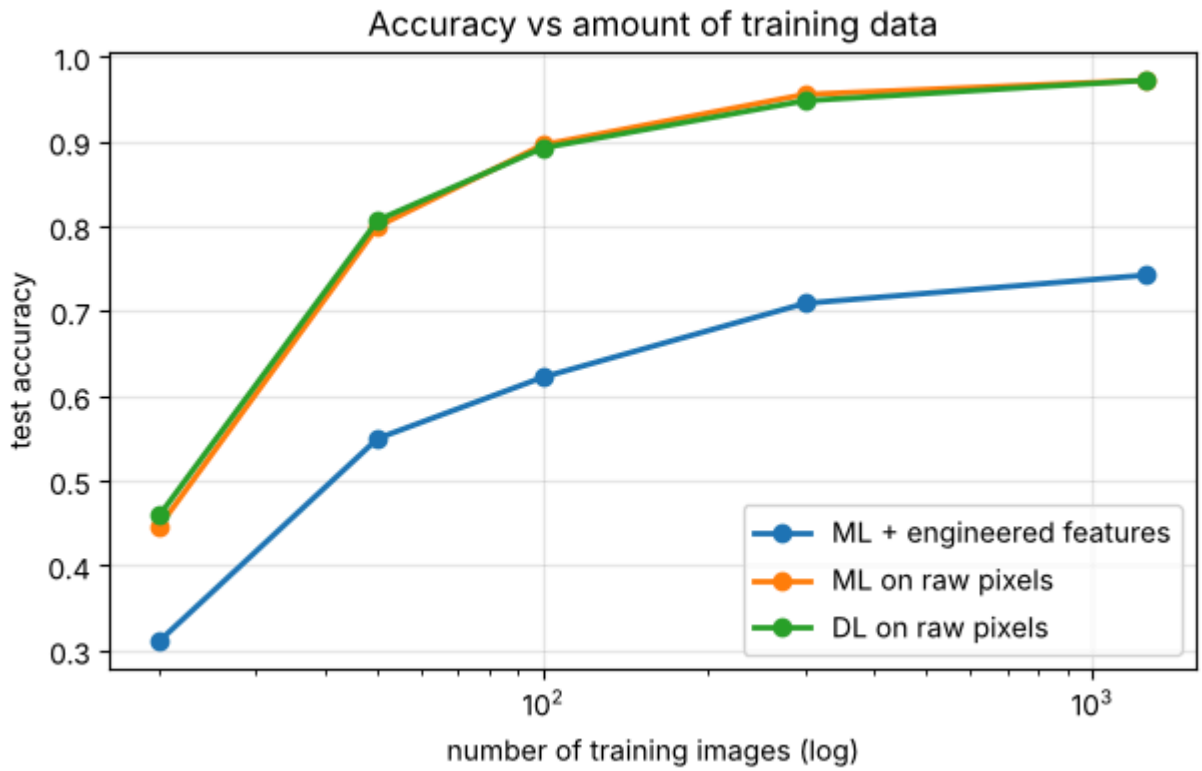
Nobody told the network to look for loops, strokes or centre holes. Its first layer **invented** its own detectors (the blobs and stroke patterns above). Deeper layers combine them. That's the lecture's **automatic, hierarchical feature learning**.

"DL needs more data": measured

The lecture says deep learning needs more data than classical ML. Let's check how accuracy grows with training-set size, for the three learners.

```python
def train_dl(Xa, ya, epochs=150, seed=0):
    torch.manual_seed(seed)
    net = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 64), nn.ReLU(),
nn.Linear(64, 10))
    o = torch.optim.Adam(net.parameters(), lr=1e-3)
    Xa, ya = torch.tensor(Xa / 16, dtype=torch.float32), torch.tensor(ya)
    for _ in range(epochs):
        perm = torch.randperm(len(Xa))
        for i in range(0, len(perm), 64):
            idx = perm[i:i + 64]
            l = nn.functional.cross_entropy(net(Xa[idx]), ya[idx]); o.zero_grad(); l.backward();
o.step()
    with torch.no_grad():
        return (net(Xt_te).argmax(1) == yt_te).float().mean().item()

sizes = [20, 50, 100, 300, len(X_tr)]
res = {"ML + engineered features": [], "ML on raw pixels": [], "DL on raw pixels": []}
for n in sizes:
    idx = np.random.default_rng(0).choice(len(X_tr), n, replace=False) if n < len(X_tr) else
np.arange(len(X_tr))
    res["ML + engineered features"].append(make_pipeline(StandardScaler(),
LogisticRegression(max_iter=2000)).fit(F_tr[idx], y_tr[idx]).score(F_te, y_te))
    res["ML on raw pixels"].append(make_pipeline(StandardScaler(),
LogisticRegression(max_iter=5000)).fit(X_tr[idx], y_tr[idx]).score(X_te, y_te))
    res["DL on raw pixels"].append(train_dl(X_tr[idx], y_tr[idx]))
plt.figure(figsize=(7, 4))
for k, v in res.items():
    plt.plot(sizes, v, "o-", lw=2, label=k)
plt.xscale("log"); plt.xlabel("number of training images (log)"); plt.ylabel("test accuracy");
plt.legend(); plt.grid(alpha=0.3)
plt.title("Accuracy vs amount of training data"); plt.show()
print("train size:", sizes)
for k, v in res.items():
    print(f"{k:>25}: " + "  ".join(f"{a:.0%}" for a in v))
```




```
train size: [20, 50, 100, 300, 1257]
 ML + engineered features: 31%  55%  62%  71%  74%
         ML on raw pixels: 45%  80%  90%  96%  97%
         DL on raw pixels: 46%  81%  89%  95%  97%
```

How to read it (honestly):

- **The engineered-feature model is worst at every data size** (31% → 74%). My 8 hand-made features throw information away, and more data can't bring it back. That is the **human bottleneck** of manual feature engineering in action: the model can only be as good as the features someone thought of.
- **Logistic regression and the deep network on raw pixels track each other almost exactly** at every size (45% → 97% vs 46% → 97%). On tiny, clean, centred 8×8 digits, raw pixels are already an excellent representation, so the deep network's learned features have nothing extra to offer.
- So this dataset does **not** show "DL needs more data and then wins". That effect appears on **large, complex** data (real photos, speech, text), where a linear model on raw input is hopeless and only learned, layered features work, and where deep networks keep improving as data grows into the millions.
- It does confirm the lecture's misconception #4: **bigger is not always better**. On small, simple data, a classical model is just as accurate and far simpler.

Level 4 · Generative AI: create digits that never existed

Everything so far was **discriminative**: image in, label out. A **generative** model learns what the data itself looks like, so it can **produce new samples**.

We train a tiny **variational autoencoder (VAE)**: it squeezes each digit into 2 numbers (a "code"), learns to rebuild the digit from the code, and learns codes that are spread like a simple bell curve. Afterwards we pick **brand-new random codes** and the decoder **draws new digits**. (You don't need the VAE maths yet. The point is the generate-new-content behaviour.)

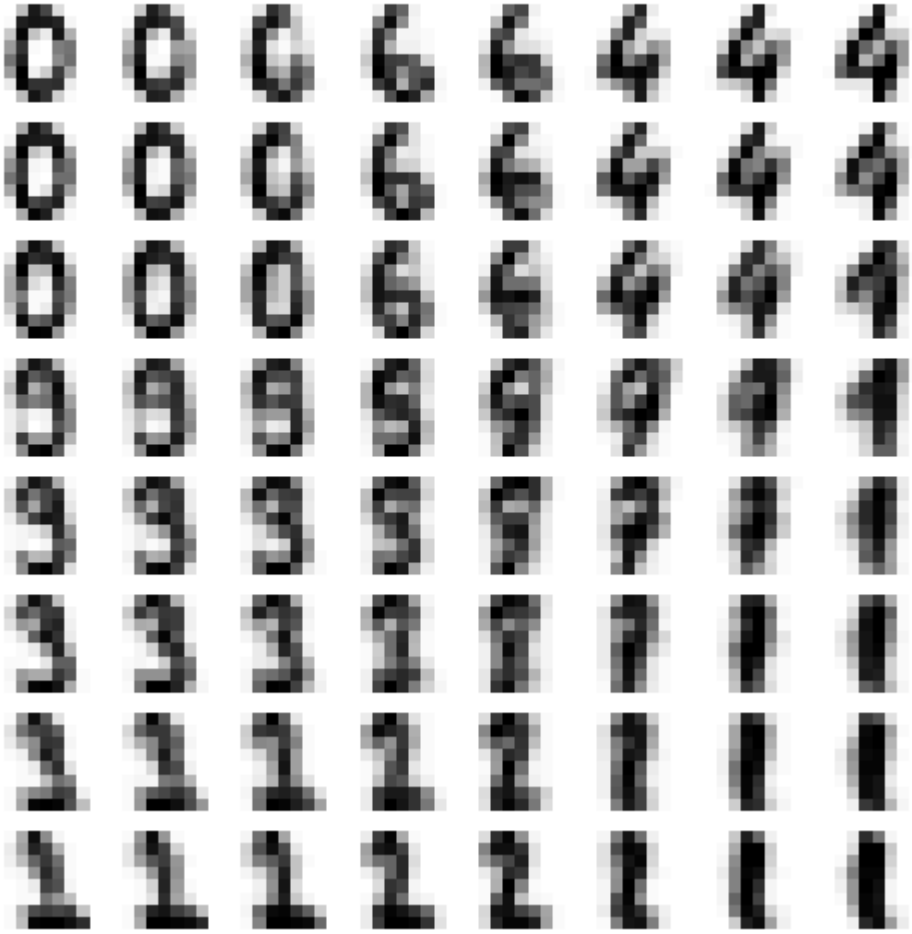
```
torch.manual_seed(0)
class TinyVAE(nn.Module):
    def __init__(self, z_dim=2):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 2 * z_dim))
        self.dec = nn.Sequential(nn.Linear(z_dim, 128), nn.ReLU(), nn.Linear(128, 64),
nn.Sigmoid())
    def forward(self, x):
        mu, logvar = self.enc(x).chunk(2, dim=1)
        z = mu + torch.randn_like(mu) * torch.exp(0.5 * logvar)        # sample a code
        return self.dec(z), mu, logvar

vae = TinyVAE(); opt = torch.optim.Adam(vae.parameters(), lr=2e-3)
Xall = torch.tensor(X / 16, dtype=torch.float32)
for epoch in range(300):
    perm = torch.randperm(len(Xall))
    for i in range(0, len(perm), 128):
        xb = Xall[perm[i:i + 128]]
        recon, mu, logvar = vae(xb)
        rec_loss = nn.functional.binary_cross_entropy(recon, xb, reduction="sum")
        kl = -0.5 * torch.sum(1 + logvar - mu ** 2 - logvar.exp())          # keep codes bell-
shaped
        loss = (rec_loss + kl) / len(xb); opt.zero_grad(); loss.backward(); opt.step()

# GENERATE: random codes in, new digits out
with torch.no_grad():
    grid = torch.tensor([[a, b] for b in np.linspace(2, -2, 8) for a in np.linspace(-2, 2, 8)],
dtype=torch.float32)
    new_digits = vae.dec(grid).numpy().reshape(-1, 8, 8)
fig, axes = plt.subplots(8, 8, figsize=(6, 6))
for ax, im in zip(axes.ravel(), new_digits):
```

```
    ax.imshow(im, cmap="gray_r"); ax.axis("off")
fig.suptitle("64 NEW digits generated from a grid of 2-number codes (none of them is a copy of a
training image)", fontsize=9)
plt.show()
```

64 NEW digits generated from a grid of 2-number codes (none of them is a copy of a training image)



```
# Prove they are new: distance from each generated digit to its NEAREST training image
gen = new_digits.reshape(64, -1); train_pix = X / 16
nearest = np.sqrt(((gen[:, None, :] - train_pix[None, :, :]) ** 2).sum(-1)).min(axis=1)
typical = np.sqrt(((train_pix[:200, None, :] - train_pix[None, :, :]) ** 2).sum(-1))
np.fill_diagonal(typical[:, :200], np.inf)
print(f"generated digit -> nearest training digit: median distance {np.median(nearest):.2f}")
print(f"real digit -> its nearest OTHER real digit: median distance
{np.median(typical.min(axis=1)):.2f}")
```

```
generated digit -> nearest training digit: median distance 0.99
real digit -> its nearest OTHER real digit: median distance 1.05
```

- The generated digits are **plausible but new**: they're about as far from their nearest training image as real digits are from each other, so the model isn't simply copying.
- Moving smoothly across the grid morphs one digit into another, because the model has learned a continuous "space of digits".
- They're blurry. A 2-number code and an 8×8 world is a toy. ChatGPT, Claude, Midjourney and other real GenAI systems apply the same "learn the data distribution, then sample from it" idea with **billions** of parameters and **web-scale** data, mostly with **Transformer** and diffusion architectures, as the lecture says.

The four levels side by side (this notebook's evidence)

```
print(f"{'level':<10}{'approach':<42}{'features chosen by':<22}{'result'}")
print(f"{'AI':<10}{'hand-written rules':<42}{'a human (rules)':<22}{np.mean(pred ==
y_te[mask]):.0%} on 0-vs-1 only")
print(f"{'ML':<10}{'logistic regression, engineered features':<42}{'a human (features)':<22}
{ml.score(F_te, y_te):.0%} on all 10 digits")
print(f"{'DL':<10}{'2-hidden-layer network, raw pixels':<42}{'the network itself':<22}
{dl_acc:.0%} on all 10 digits")
print(f"{'GenAI':<10}{'tiny VAE':<42}{'the network itself':<22}creates new digits")
```

```
level     approach                                  features chosen by    result
AI        hand-written rules                        a human (rules)       97% on 0-vs-1 only
ML        logistic regression, engineered features  a human (features)    74% on all 10 digits
DL        2-hidden-layer network, raw pixels        the network itself    98% on all 10 digits
GenAI     tiny VAE                                  the network itself    creates new digits
```

✅ Check yourself

1. Put these in the hierarchy: a chess program with hand-coded rules, a Naive Bayes spam filter, a CNN tagging beach photos, ChatGPT writing a poem. *(AI only; AI+ML; AI+ML+DL; AI+ML+DL+GenAI)*
2. What is feature engineering, and at which level does it stop being manual? *(choosing which measurements the model sees; DL learns them automatically)*
3. Why did the hand-engineered-feature model stay worst even with all the training data? *(its 8 features discard information that more data can't recover; the model is limited by the features a human chose)*
4. What makes a model *generative* rather than *discriminative*? *(it models the data itself and can produce new samples, rather than only mapping inputs to labels)*
5. "GenAI is a completely different technology from DL." True or false? *(False: it's deep learning applied at scale to content generation)*

02 · From the biological neuron to backpropagation (1943 → 1986), in code

Lectures: *History of Deep Learning* (04-08-2026) and *The Biological Neuron Analogy* (05-08-2026, Part B) · **Notes:** `../notes/01_history_of_deep_learning.md`, `../notes/02_ai_ml_dl_genai_and_the_neuron.md`

The history lecture is a timeline. Here we **run** its first four milestones:

Year	Milestone	What we code
1943	McCulloch & Pitts: the first artificial neuron	a threshold neuron that computes AND, OR and NOT, with **no learning**
1958	Rosenblatt: the perceptron	the same neuron, but its weights **learn** from examples (synaptic plasticity)
1969	Minsky & Papert: the limits	the perceptron **fails on XOR**, which led to the first "AI winter"
1986	Rumelhart, Hinton & Williams: backpropagation	a **multi-layer** network trained by backpropagation **solves XOR**

Along the way we map each part of the code to the **biological neuron** (dendrites, synapses, soma, threshold, axon). Runs in seconds.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
```

The mapping we'll keep using

Biological neuron	Artificial neuron	In the code below
dendrites receive signals	**inputs** x_1, x_2, ...	`x`
synapse strength	**weights** w_1, w_2, ...	`w`
soma integrates (sums) the signals	**weighted sum + bias**	`z = w @ x + b`
firing threshold / action potential (all-or-nothing)	**activation function**	`1 if z >= 0 else 0`
axon carries the output on	**output** to the next layer	the return value
synaptic plasticity (learning)	**weight updates**	`w += ...` (from 1958 on)

1943 · The McCulloch–Pitts neuron: logic from a threshold

Binary inputs, fixed weights, and a threshold θ: **fire (1) if the weighted sum reaches θ**. McCulloch and Pitts showed that such neurons can compute logic, which suggested that a brain-like network could, in principle, compute anything.

```
def mcculloch_pitts(x, w, theta):
    soma = np.dot(w, x)                 # integrate the incoming signals
    return int(soma >= theta)           # all-or-nothing firing

gates = {
    "AND": dict(w=[1, 1], theta=2),     # fires only if BOTH inputs fire
    "OR":  dict(w=[1, 1], theta=1),     # fires if AT LEAST ONE input fires
}
for name, p in gates.items():
    print(name, [mcculloch_pitts(x, **p) for x in X])
print("NOT", [mcculloch_pitts([x], w=[-1], theta=0) for x in [0, 1]], " (an inhibitory input:
weight -1)")
```

```
AND [0, 0, 0, 1]
OR [0, 1, 1, 1]
NOT [1, 0]  (an inhibitory input: weight -1)
```

No learning happened: **a human chose** the weights and thresholds. That's 1943's limitation, and it's what the **perceptron** changed.

1958 · Rosenblatt's perceptron: the weights learn

Same neuron (weights, sum, threshold), plus a **learning rule** that adjusts the weights after every mistake: **w ← w + η·(y − ŷ)·x**. This is the artificial version of **synaptic plasticity**: connections that should have fired more get stronger. (`03_perceptron_from_scratch.ipynb` covers this in depth.)

```
def train_perceptron(X, y, eta=0.1, epochs=50):
    w, b, history = np.zeros(2), 0.0, []
    for epoch in range(epochs):
        mistakes = 0
        for xi, yi in zip(X, y):
            y_hat = int(np.dot(w, xi) + b >= 0)
            w += eta * (yi - y_hat) * xi        # synaptic plasticity
            b += eta * (yi - y_hat)
            mistakes += int(yi != y_hat)
        history.append(mistakes)
        if mistakes == 0:
            break
    return w, b, history

for name, y in [("AND", np.array([0, 0, 0, 1])), ("OR", np.array([0, 1, 1, 1]))]:
    w, b, hist = train_perceptron(X, y)
    print(f"{name}: learned w = {w}, b = {b:.1f} in {len(hist)} epochs -> outputs {[int(np.dot(w,
x) + b >= 0) for x in X]}")
```

```
AND: learned w = [0.2 0.1], b = -0.2 in 4 epochs -> outputs [0, 0, 0, 1]
OR: learned w = [0.1 0.1], b = -0.1 in 4 epochs -> outputs [0, 1, 1, 1]
```

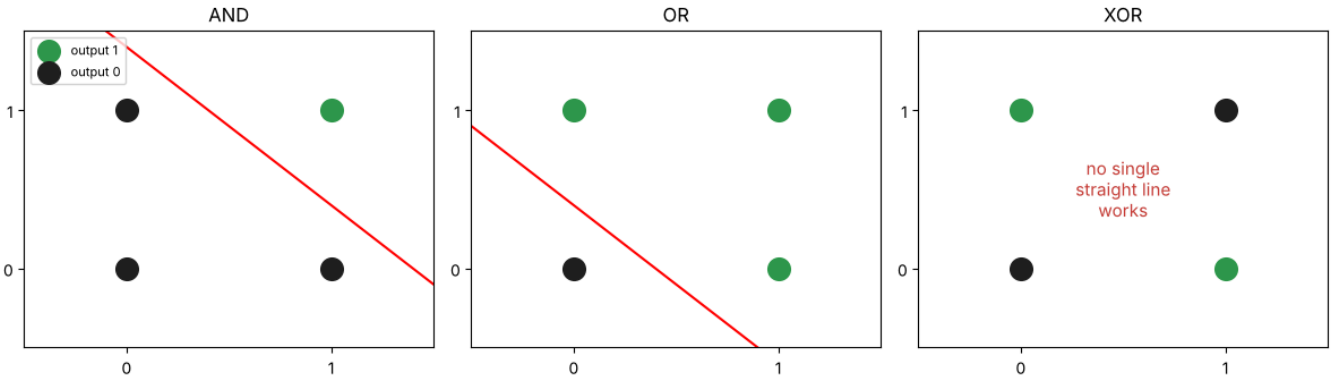
The perceptron **found weights for AND and OR by itself**. That was the excitement of 1958: a machine that learns from examples.

1969 · Minsky & Papert: the XOR problem → the AI winter

```
y_xor = np.array([0, 1, 1, 0])
w, b, hist = train_perceptron(X, y_xor, epochs=50)
print("XOR mistakes per epoch:", hist[:12], "... still", hist[-1], "mistakes after", len(hist),
"epochs")
print("outputs:", [int(np.dot(w, x) + b >= 0) for x in X], " target:", y_xor.tolist())

fig, axes = plt.subplots(1, 3, figsize=(12, 3.6))
for ax, (name, yy) in zip(axes, [("AND", [0, 0, 0, 1]), ("OR", [0, 1, 1, 1]), ("XOR", [0, 1, 1,
0])]):
    yy = np.array(yy)
    ax.scatter(*X[yy == 1].T, s=200, c="#2e9a4b", label="output 1"); ax.scatter(*X[yy == 0].T,
s=200, c="#222", label="output 0")
    ax.set_xlim(-0.5, 1.5); ax.set_ylim(-0.5, 1.5); ax.set_title(name); ax.set_xticks([0, 1]);
ax.set_yticks([0, 1])
axes[0].plot([-0.5, 1.5], [1.9, -0.1], "r-"); axes[1].plot([-0.5, 1.2], [0.9, -0.8], "r-")
axes[2].text(0.5, 0.5, "no single\nstraight line\nworks", ha="center", va="center",
color="#c8403a", fontsize=11)
axes[0].legend(loc="upper left", fontsize=8); plt.tight_layout(); plt.show()
```

```
XOR mistakes per epoch: [3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4] ... still 4 mistakes after 50 epochs
outputs: [1, 1, 0, 0]  target: [0, 1, 1, 0]
```



A single perceptron draws **one straight line**. XOR's 1s sit on one diagonal and its 0s on the other, so no line separates them, and the perceptron keeps making mistakes forever. Minsky and Papert's book *Perceptrons* (1969) made this limitation famous, funding dried up, and the field entered the **AI winter**: researchers moved to rule-based expert systems.

1986 · Backpropagation: multi-layer networks learn XOR

The fix needed two things:

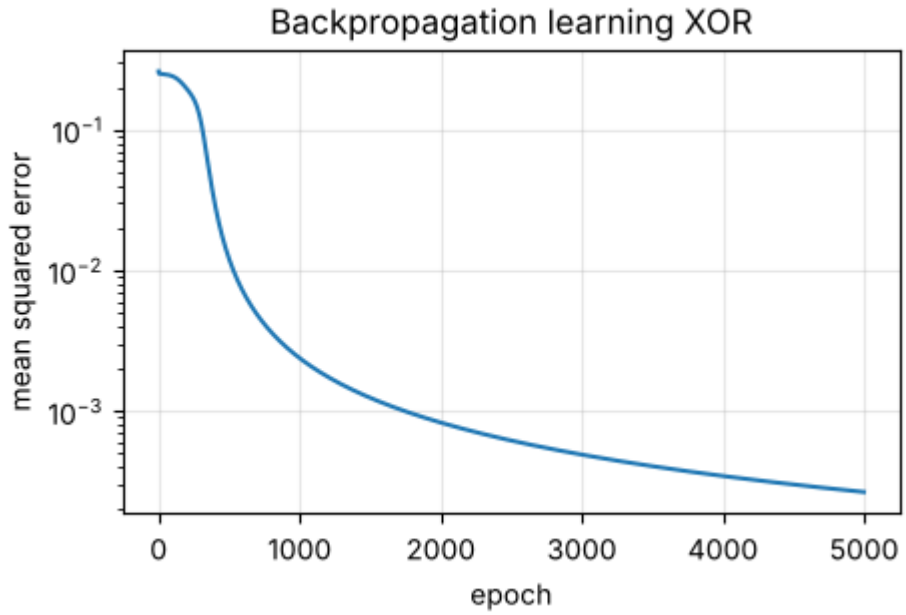
1. **Hidden layers**, so the network can bend its decision boundary.
2. A way to **train** them. The step function's slope is 0, so it gives no learning signal, and nobody knew how much each hidden neuron was to blame for an error.

Backpropagation (Rumelhart, Hinton & Williams, 1986) solved part 2: use a **smooth** activation (sigmoid), and pass the error **backwards** layer by layer with the chain rule, giving every weight its share of the blame. Here it is from scratch in NumPy (no deep-learning library), so nothing is hidden.

```
rng = np.random.default_rng(1)
sigmoid = lambda z: 1 / (1 + np.exp(-z))
W1, b1 = rng.normal(0, 1, (2, 3)), np.zeros(3)      # input (2) -> hidden (3)
W2, b2 = rng.normal(0, 1, (3, 1)), np.zeros(1)      # hidden (3) -> output (1)
Y = y_xor.reshape(-1, 1); eta = 1.0; losses = []
for epoch in range(5000):
    # forward pass: signals flow input -> hidden -> output (dendrites -> soma -> axon -> next
```

```
neuron)
    H = sigmoid(X @ W1 + b1)
    out = sigmoid(H @ W2 + b2)
    losses.append(np.mean((out - Y) ** 2))
    # backward pass: the error flows output -> hidden, each weight gets its share of the blame
    d_out = (out - Y) * out * (1 - out)              # chain rule through the output sigmoid
    d_hid = (d_out @ W2.T) * H * (1 - H)             # ... and back through the hidden sigmoids
    W2 -= eta * H.T @ d_out;  b2 -= eta * d_out.sum(0)
    W1 -= eta * X.T @ d_hid;  b1 -= eta * d_hid.sum(0)
print("outputs after training:", out.ravel().round(3), "-> rounded", (out.ravel() >
0.5).astype(int).tolist(), " target", y_xor.tolist())
plt.figure(figsize=(5, 3)); plt.plot(losses); plt.yscale("log"); plt.xlabel("epoch");
plt.ylabel("mean squared error")
plt.title("Backpropagation learning XOR"); plt.grid(alpha=0.3); plt.show()
```

```
outputs after training: [0.016 0.988 0.981 0.017] -> rounded [0, 1, 1, 0]  target [0, 1, 1, 0]
```



XOR solved. The two-layer network learned it from examples using backpropagation, which became the standard training algorithm and remains so today, inside every PyTorch `loss.backward()`.

What happened next (the rest of the timeline)

Year	Milestone	Where you meet it
1998	**LeNet-5** (LeCun): CNNs read handwritten digits and bank cheques	CV course Week 2–3
2006	**Deep Belief Networks** (Hinton): layer-wise pre-training makes deep nets trainable	the start of "modern" deep learning
2012	**AlexNet** (Krizhevsky, Sutskever, Hinton): GPUs + **ReLU** + **Dropout** win ImageNet	notebooks 04 and 05 here
2014	**GANs** (Goodfellow): generator vs discriminator	CV course Week 4
2017	**Transformers** ("Attention Is All You Need", Google)	the basis of LLMs
2020–	**Foundation models & Generative AI** (GPT, Llama, Gemini, Claude, Stable Diffusion)	notebook 01's tiny generative model is the toy version

Where the biological analogy stops

The code above captures the **computational principle** (weighted inputs → sum → threshold → output, with adjustable connections). It ignores almost everything else about real neurons:

	Biological	Our code
signal	electrochemical spikes, neurotransmitters	floating-point numbers
timing	asynchronous, continuous time	synchronous steps
learning	biochemical plasticity, not fully understood	an exact formula (backpropagation + gradient descent)
scale	~86 billion neurons, ~100 trillion synapses, ~20 watts	4 neurons (3 hidden + 1 output) and 13 learnable numbers here

✅ Check yourself

1. Match: dendrites, synapse, soma, firing threshold, axon, plasticity → ? *(inputs, weights, weighted sum + bias, activation function, output, weight updates)*
2. What could the 1943 neuron do, and what couldn't it do? *(compute logic with hand-set weights; it couldn't learn)*
3. What did the perceptron add in 1958? *(a learning rule that adjusts the weights from examples)*
4. Why did XOR cause an "AI winter"? *(a single-layer perceptron provably can't learn it, which undermined confidence and funding)*
5. What two things were needed to solve XOR, and which 1986 algorithm provided the training part? *(hidden layers + a way to train them; backpropagation with smooth activations)*

03 · The perceptron, from scratch

Lecture: *Perceptron* (06-08-2026) · **Notes:** `../notes/03_perceptron.md`

In this notebook you will:

1. Recompute the lecture's two worked examples in code (forward pass z = 1.4, and the weight update).
2. Build a `Perceptron` class in about 15 lines of NumPy and train it on the lecture's **Pass/Fail** example (hours of study x1, hours of practice x2).
3. Watch the decision boundary move as the perceptron learns.
4. Test the lecture's claims about the **learning rate**.
5. Prove the famous limitation: AND and OR are learnable, **XOR is not**. Then fix XOR with a tiny multi-layer network.

Everything runs in a few seconds. Change the numbers and re-run: that's the point.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(precision=3, suppress=True)
rng = np.random.default_rng(0)
```

1 · The two building blocks: weighted sum and step activation

Symbol	Meaning (lecture's notation)
x1, x2	inputs: hours of study, hours of practice
w1, w2	weights: how important each input is
b	bias: shifts the threshold
z	net input, **z = w1·x1 + w2·x2 + b**
f(z)	**step** activation: 1 if z ≥ 0, else 0
ŷ (y_hat)	prediction: 1 = Pass, 0 = Fail
y	the true answer
η (eta)	learning rate

```
def step(z):
    """Heaviside step: 1 if z >= 0 else 0 (works on numbers and arrays)."""
    return (np.asarray(z) >= 0).astype(int)

def forward(x, w, b):
    z = np.dot(w, x) + b          # weighted sum (net input)
    return z, step(z)             # (z, prediction)

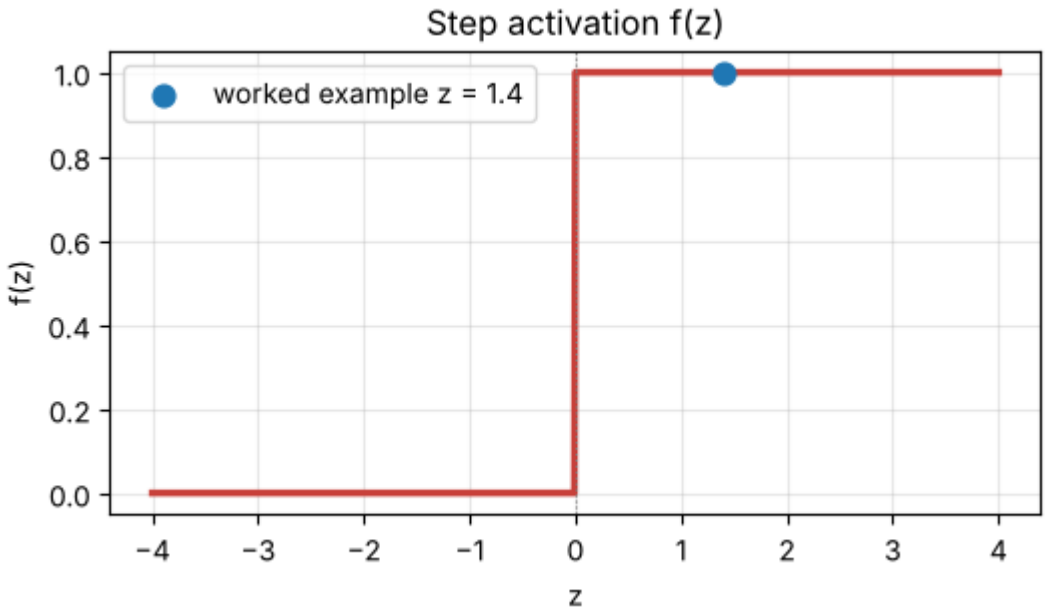
# Lecture worked example 3.2: x1=2, x2=3, w1=0.5, w2=0.8, b=-2
z, y_hat = forward(np.array([2, 3]), np.array([0.5, 0.8]), -2)
print(f"z = 2*0.5 + 3*0.8 + (-2) = {z:.1f}  ->  f(z) = {y_hat}  ({'Pass' if y_hat else 'Fail'})")
```

```
z = 2*0.5 + 3*0.8 + (-2) = 1.4  ->  f(z) = 1  (Pass)
```

That matches the lecture: **z = 1.4, ŷ = 1 (Pass)**.

The step function plotted. Note the jump at z = 0: at exactly z = 0 the lecture's rule says "fire" (1).

```
zs = np.linspace(-4, 4, 801)
plt.figure(figsize=(6, 3))
plt.plot(zs, step(zs), lw=2.5, color="#c8403a")
plt.axvline(0, color="grey", lw=0.8, ls=":")
plt.scatter([1.4], [1], s=60, zorder=3, label="worked example z = 1.4")
plt.title("Step activation f(z)"); plt.xlabel("z"); plt.ylabel("f(z)"); plt.legend();
plt.grid(alpha=0.3)
plt.show()
```



2 · Learning: the perceptron update rule

Forward propagation only **predicts**. To **learn**, compare the prediction with the truth and nudge the weights:

$w_i \leftarrow w_i + \eta \cdot (y - \hat{y}) \cdot x_i$ $b \leftarrow b + \eta \cdot (y - \hat{y})$

The error $(y - \hat{y})$ can only be −1, 0 or +1:

- **0**: the prediction was right, so nothing changes.
- **+1**: it said Fail but the answer was Pass (under-prediction), so the weights move **up** in proportion to each input.
- **−1**: it said Pass but the answer was Fail (over-prediction), so the weights move **down**.

Lecture worked example 5.2: x = (2, 3), w = (0.2, −0.4), b = −1.0, y = 1, ŷ = 0, η = 0.1.

```
def update(x, y, y_hat, w, b, eta):
    error = y - y_hat
    return w + eta * error * x, b + eta * error

w_new, b_new = update(np.array([2, 3]), 1, 0, np.array([0.2, -0.4]), -1.0, 0.1)
print("new weights:", w_new, " new bias:", round(b_new, 2))

# does the updated perceptron now get it right?
```

```
z_before, p_before = forward(np.array([2, 3]), np.array([0.2, -0.4]), -1.0)
z_after, p_after = forward(np.array([2, 3]), w_new, b_new)
print(f"before: z = {z_before:.2f} -> {p_before}   after one update: z = {z_after:.2f} ->
{p_after}")
```

```
new weights: [ 0.4 -0.1]  new bias: -0.9
before: z = -1.80 -> 0   after one update: z = -0.40 -> 0
```

Matches the lecture: **w1 = 0.4, w2 = −0.1, b = −0.9**. Notice two things the lecture doesn't spell out:

- One update moved z from −1.8 to −0.4. That is **closer** to the threshold, but the prediction is **still wrong** (0). Learning takes several passes over the data.
- Each weight moved **in proportion to its input** (x2 = 3 moved w2 more than x1 = 2 moved w1). Inputs that were "louder" get more of the blame.

3 · A `Perceptron` class, trained on Pass/Fail data

Made-up but realistic data: 60 students. A student passes if 0.6·study + 0.4·practice is above 5, a hidden rule the perceptron has to discover. Borderline students (score within 0.4 of 5) are left out, so there is a clear gap between Pass and Fail.

```
class Perceptron:
    def __init__(self, n_inputs, eta=0.1, w_init=None):
        self.w = np.zeros(n_inputs) if w_init is None else np.array(w_init, float)
        self.b = 0.0
        self.eta = eta
        self.history = []                       # (w, b, mistakes) after every epoch

    def predict(self, X):
        return step(X @ self.w + self.b)

    def fit(self, X, y, epochs=20, verbose=False):
        for epoch in range(epochs):
            mistakes = 0
            for xi, yi in zip(X, y):            # one example at a time (online learning)
                y_hat = step(xi @ self.w + self.b)
                error = yi - y_hat
                self.w += self.eta * error * xi
                self.b += self.eta * error
                mistakes += int(error != 0)
            self.history.append((self.w.copy(), self.b, mistakes))
            if verbose and (epoch < 5 or mistakes == 0 or (epoch + 1) % 10 == 0):
                print(f"epoch {epoch + 1:2d}: mistakes = {mistakes:2d}   w = {self.w}  b =
{self.b:.2f}")
            if mistakes == 0:                   # converged: every training example correct
                break
        return self

# the data
study = rng.uniform(0, 10, 200)
practice = rng.uniform(0, 10, 200)
score = 0.6 * study + 0.4 * practice
keep = np.abs(score - 5) > 0.4            # drop borderline students so a clear gap exists
study, practice, score = study[keep][:60], practice[keep][:60], score[keep][:60]
n = len(study)
y = (score > 5).astype(int)
X = np.column_stack([study, practice])
print("students:", n, "  passed:", y.sum(), "  failed:", n - y.sum())

p = Perceptron(2, eta=0.1).fit(X, y, epochs=500, verbose=True)
print("training accuracy:", (p.predict(X) == y).mean())
```

```
students: 60   passed: 29   failed: 31
epoch  1: mistakes = 31   w = [ 0.46  -0.832]  b = -1.10
epoch  2: mistakes = 21   w = [ 0.777 -0.61 ]  b = -1.60
epoch  3: mistakes = 22   w = [ 0.586 -0.275]  b = -2.20
epoch  4: mistakes = 22   w = [ 0.836 -0.388]  b = -2.80
epoch  5: mistakes = 21   w = [ 0.846 -0.088]  b = -3.30
epoch 10: mistakes = 16   w = [ 0.891 -0.048]  b = -5.50
epoch 19: mistakes =  0   w = [0.835 0.714]  b = -7.60
training accuracy: 1.0
```

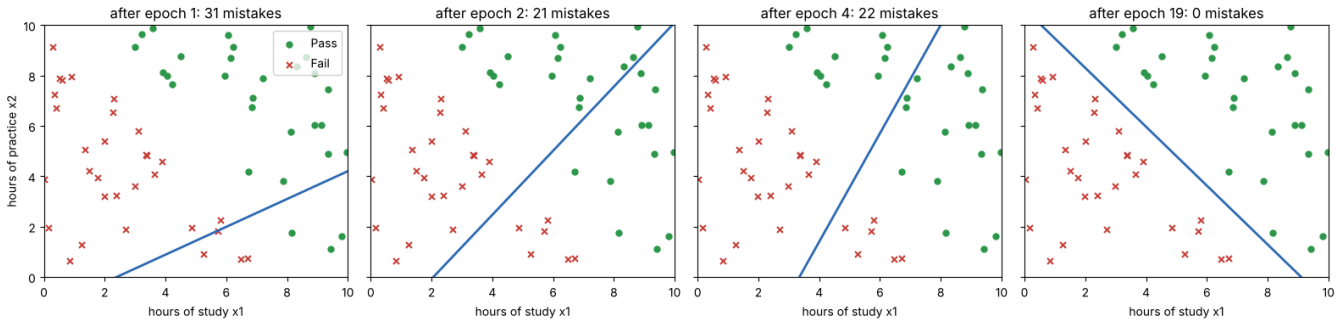
The number of mistakes per epoch shrinks to **0**: the perceptron found a line that separates every Pass from every Fail. The **perceptron convergence theorem** guarantees this happens in a finite number of steps **if** such a line exists (the data is linearly separable).

Watching the decision boundary move

The boundary is the line where z = 0: w1·x1 + w2·x2 + b = 0, i.e. x2 = −(w1·x1 + b)/w2.

```
def draw_boundary(ax, w, b, **kw):
    xs = np.array([0, 10])
    if abs(w[1]) > 1e-9:
        ax.plot(xs, -(w[0] * xs + b) / w[1], **kw)
    elif abs(w[0]) > 1e-9:
        ax.axvline(-b / w[0], **kw)

epochs_to_show = [0, 1, 3, len(p.history) - 1]
fig, axes = plt.subplots(1, 4, figsize=(16, 4), sharey=True)
for ax, e in zip(axes, epochs_to_show):
    w_e, b_e, m_e = p.history[e]
    ax.scatter(*X[y == 1].T, c="#2e9a4b", label="Pass", s=25)
    ax.scatter(*X[y == 0].T, c="#c8403a", label="Fail", s=25, marker="x")
    draw_boundary(ax, w_e, b_e, color="#2f6db5", lw=2)
    ax.set_xlim(0, 10); ax.set_ylim(0, 10)
    ax.set_title(f"after epoch {e + 1}: {m_e} mistakes"); ax.set_xlabel("hours of study x1")
axes[0].set_ylabel("hours of practice x2"); axes[0].legend(loc="upper right")
plt.tight_layout(); plt.show()
```



4 · Does the learning rate matter? (testing the lecture's claims)

The lecture says: η too small → slow; η too large → overshoot/diverge. That is true for **gradient descent** on a smooth loss (section 5 shows it). For the classic perceptron there is a surprise worth knowing:

```
for eta in [0.001, 0.01, 0.1, 1.0, 10.0]:
    pz = Perceptron(2, eta=eta).fit(X, y, epochs=200)          # weights start at zero
    pr = Perceptron(2, eta=eta, w_init=[-1.0, 1.0]).fit(X, y, epochs=200)   # weights start
elsewhere
```

```
    print(f"eta = {eta:>6}:  start at zero -> {len(pz.history):3d} epochs  |  start at (-1, 1) ->
{len(pr.history):3d} epochs")
```

```
eta =  0.001:  start at zero ->  19 epochs  |  start at (-1, 1) ->  19 epochs
eta =   0.01:  start at zero ->  19 epochs  |  start at (-1, 1) ->  13 epochs
eta =    0.1:  start at zero ->  19 epochs  |  start at (-1, 1) ->  13 epochs
eta =    1.0:  start at zero ->  19 epochs  |  start at (-1, 1) ->  18 epochs
eta =   10.0:  start at zero ->  19 epochs  |  start at (-1, 1) ->   3 epochs
```

What the numbers show: when the weights start at **zero**, every η takes **exactly the same number of epochs**. The step function only looks at the **sign** of z, and multiplying all the weights and the bias by η doesn't change any sign. So η just rescales the whole solution. When the weights start somewhere else, η does matter, because it sets how big the corrections are compared with the starting weights.

The lecture's learning-rate trade-off is really about **gradient descent on a smooth loss**, which modern networks use. Here it is:

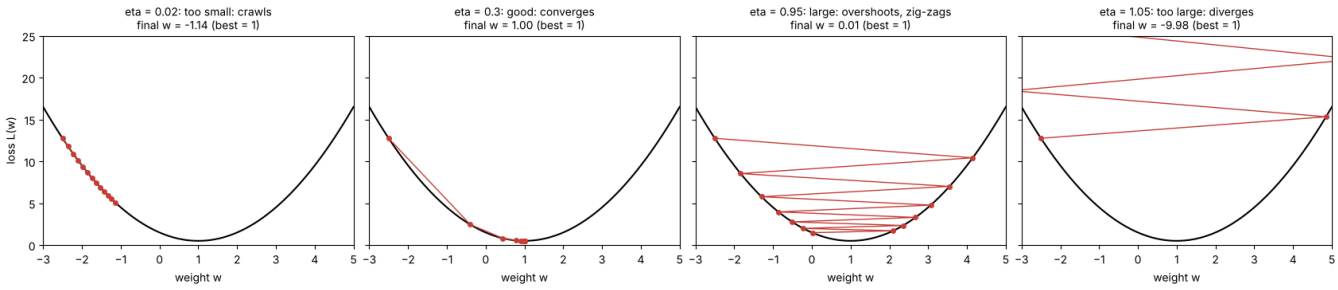
5 · Gradient descent: walking downhill on a loss curve

A loss curve $L(w) = (w - 1)^2 + 0.5$. Its slope is $2(w - 1)$. Each step: **$w \leftarrow w - \eta \cdot \text{slope}$**.

```
L = lambda w: (w - 1) ** 2 + 0.5
dL = lambda w: 2 * (w - 1)

def descend(w0, eta, steps=12):
    ws = [w0]
    for _ in range(steps):
        ws.append(ws[-1] - eta * dL(ws[-1]))
    return np.array(ws)

ws_grid = np.linspace(-3, 5, 400)
fig, axes = plt.subplots(1, 4, figsize=(17, 3.8), sharey=True)
for ax, (eta, title) in zip(axes, [(0.02, "too small: crawls"), (0.3, "good: converges"),
                                   (0.95, "large: overshoots, zig-zags"), (1.05, "too large:
diverges")]):
    path = descend(-2.5, eta)
    ax.plot(ws_grid, L(ws_grid), color="black", lw=1.5)
    ax.plot(path, L(path), "o-", color="#c8403a", ms=4, lw=1)
    ax.set_title(f"eta = {eta}: {title}\nfinal w = {path[-1]:.2f} (best = 1)", fontsize=10)
    ax.set_ylim(0, 25); ax.set_xlim(-3, 5); ax.set_xlabel("weight w")
axes[0].set_ylabel("loss L(w)")
plt.tight_layout(); plt.show()
```



This confirms the lecture's trade-off: tiny steps crawl, good steps arrive quickly, large steps zig-zag across the valley, and **too-large steps make the loss grow** (diverge).

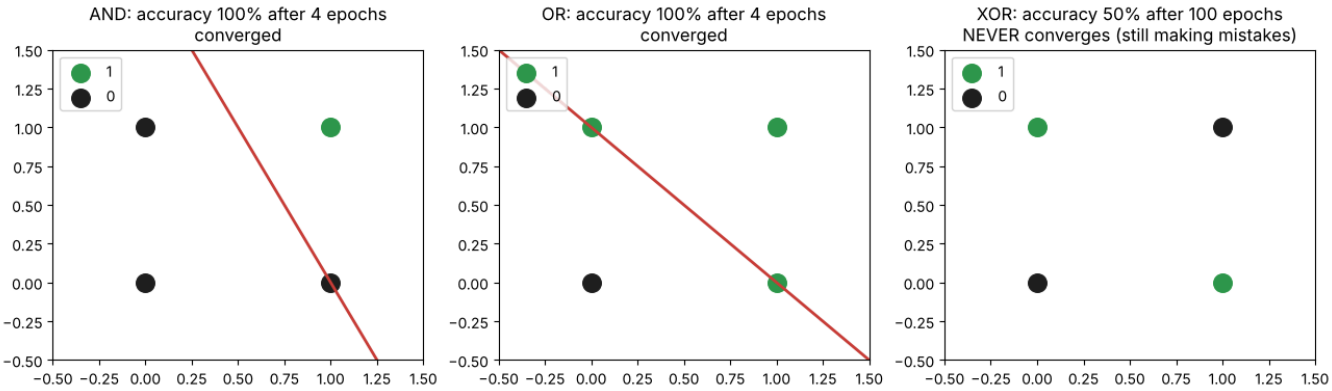
> **Local vs global minimum:** this loss has only one valley, so any converging η finds the global minimum. Real neural-network losses have many valleys and flat regions, and there the step size (and luck) affect where you end up.

6 · Linear separability: AND ✓, OR ✓, XOR ×

```
X_gate = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
gates = {"AND": np.array([0, 0, 0, 1]), "OR": np.array([0, 1, 1, 1]), "XOR": np.array([0, 1, 1,
0])}

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
for ax, (name, yg) in zip(axes, gates.items()):
    pg = Perceptron(2, eta=0.1).fit(X_gate, yg, epochs=100)
    acc = (pg.predict(X_gate) == yg).mean()
    converged = pg.history[-1][2] == 0
    ax.scatter(*X_gate[yg == 1].T, s=160, c="#2e9a4b", label="1")
    ax.scatter(*X_gate[yg == 0].T, s=160, c="#222", label="0")
    xs = np.array([-0.5, 1.5])
    w_g, b_g = pg.w, pg.b
    if abs(w_g[1]) > 1e-9:
        ax.plot(xs, -(w_g[0] * xs + b_g) / w_g[1], color="#c8403a", lw=2)
    ax.set_xlim(-0.5, 1.5); ax.set_ylim(-0.5, 1.5)
    ax.set_title(f"{name}: accuracy {acc:.0%} after {len(pg.history)} epochs\n"
                 f"{'converged' if converged else 'NEVER converges (still making mistakes)'}")
    ax.legend(loc="upper left")
    print(f"{name}: weights {pg.w}, bias {pg.b:.1f}, accuracy {acc:.0%}, converged =
{converged}")
plt.tight_layout(); plt.show()
```

```
AND: weights [0.2 0.1], bias -0.2, accuracy 100%, converged = True
OR: weights [0.1 0.1], bias -0.1, accuracy 100%, converged = True
XOR: weights [-0.1  0. ], bias 0.0, accuracy 50%, converged = False
```



A closer look: the AND line passes right through a point

Look at the AND panel: the learned line goes **exactly through (1, 0)**. Let's print z for every input:

```
p_and = Perceptron(2, eta=0.1).fit(X_gate, gates["AND"], epochs=100)
for x in X_gate:
    z = x @ p_and.w + p_and.b
    print(f"x = {x}: z = {z:+.20f} -> prediction {step(z)}")
```

```
x = [0 0]: z = -0.20000000000000003886 -> prediction 0
x = [0 1]: z = -0.10000000000000003331 -> prediction 0
x = [1 0]: z = -0.00000000000000002776 -> prediction 0
x = [1 1]: z = +0.10000000000000000555 -> prediction 1
```

For (1, 0), z is a tiny number from floating-point rounding, not a real margin. Mathematically 0.2 + (−0.2) = 0, which the step function would call **1**, a wrong answer for AND. The run is correct only because of a rounding hair.

Two lessons:

- The perceptron **stops as soon as it makes no mistakes**, however close a point sits to the line. It doesn't look for a *safe* boundary.
- Algorithms that maximise the **margin** (the gap between the line and the nearest points), like the **Support Vector Machine**, were designed to fix exactly this. A boundary with a wide margin generalises better to new, slightly different inputs.

```
# Can ANY single line do better on XOR? Try 200,000 random lines and keep the best.
W = rng.normal(size=(200_000, 2)); B = rng.normal(size=200_000)
preds = step(X_gate @ W.T + B)                          # shape (4, 200000)
best = (preds == gates["XOR"][:, None]).mean(axis=0).max()
print(f"best accuracy of any of 200,000 random lines on XOR: {best:.0%}")
```

```
best accuracy of any of 200,000 random lines on XOR: 75%
```

No straight line gets more than 3 of the 4 XOR points right (75%). This is the limitation Minsky and Papert published in 1969, and it helped start the first "AI winter".

7 · The fix: add a hidden layer (a multi-layer perceptron)

XOR = (x1 OR x2) AND NOT (x1 AND x2). Two perceptrons in a hidden layer compute OR and NAND, and one output perceptron ANDs them. **Hand-set weights:**

```
def mlp_xor(x):
    h_or   = step(x @ np.array([1, 1]) - 0.5)        # OR
    h_nand = step(x @ np.array([-1, -1]) + 1.5)      # NAND = NOT(AND)
    return step(h_or + h_nand - 1.5)                 # AND of the two hidden neurons

for x in X_gate:
    print(x, "->", mlp_xor(x))
```

```
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 0
```

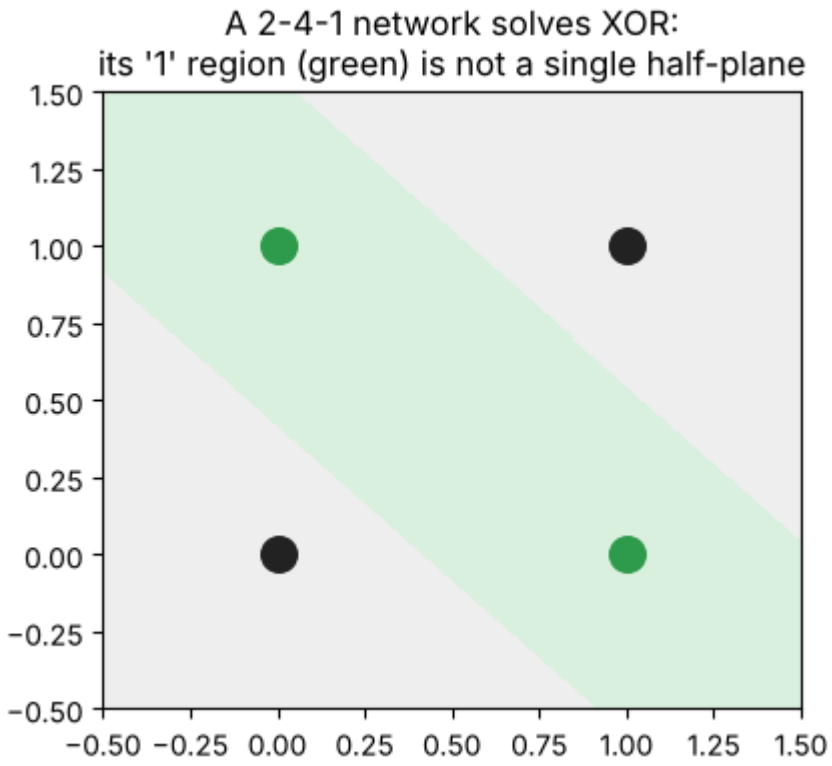
Now let a network **learn** XOR by itself. The step function's slope is zero everywhere, so gradient descent can't train it. We swap in a smooth activation (sigmoid, see notebook 04) and use **backpropagation**, the 1986 algorithm that ended the AI winter.

```
import torch, torch.nn as nn
torch.manual_seed(0)
Xt = torch.tensor(X_gate, dtype=torch.float32); yt = torch.tensor(gates["XOR"],
dtype=torch.float32).unsqueeze(1)
net = nn.Sequential(nn.Linear(2, 4), nn.Sigmoid(), nn.Linear(4, 1), nn.Sigmoid())   # 2 -> 4
hidden -> 1
opt = torch.optim.SGD(net.parameters(), lr=1.0)
for epoch in range(3000):
    loss = nn.functional.binary_cross_entropy(net(Xt), yt)
    opt.zero_grad(); loss.backward(); opt.step()
print("final loss:", round(loss.item(), 4))
print("outputs:", net(Xt).detach().squeeze().numpy().round(3), "-> rounded:", (net(Xt) >
0.5).int().squeeze().tolist())

# the curved decision region it learned
g = np.linspace(-0.5, 1.5, 200); GX, GY = np.meshgrid(g, g)
grid = torch.tensor(np.column_stack([GX.ravel(), GY.ravel()]), dtype=torch.float32)
Z = net(grid).detach().numpy().reshape(GX.shape)
plt.figure(figsize=(4.5, 4))
plt.contourf(GX, GY, Z > 0.5, levels=1, colors=["#eeeeee", "#d9f0df"])
plt.scatter(*X_gate[gates["XOR"] == 1].T, s=160, c="#2e9a4b"); plt.scatter(*X_gate[gates["XOR"]
== 0].T, s=160, c="#222")
```

```
plt.title("A 2-4-1 network solves XOR:\nits '1' region (green) is not a single half-plane");
plt.show()
```

```
final loss: 0.003
outputs: [0.003 0.997 0.997 0.003] -> rounded: [0, 1, 1, 0]
```



The hidden layer lets the network **bend** its decision region, which a single line can't do. That is the whole reason for "deep" networks.

✅ Check yourself

1. Compute by hand: x = (1, 4), w = (0.5, −0.5), b = 1. What are z and ŷ? *(z = 0.5 − 2 + 1 = −0.5 → ŷ = 0)*
2. The perceptron predicted 1 but the answer is 0, with η = 0.5 and x = (2, 1). How does each weight change? *(error = −1 → w1 −1.0, w2 −0.5, b −0.5)*
3. Why does the perceptron stop updating once it gets everything right? *(error = 0 for every example, so the update is 0)*
4. Why can't a single perceptron learn XOR, and what fixes it? *(one straight line can't separate XOR's diagonal pattern; a hidden layer, i.e. an MLP, bends the boundary)*
5. Why can't you train a step-function network with gradient descent? *(the step's slope is 0 everywhere except the jump, so there's no gradient signal; use smooth activations like sigmoid or ReLU)*

Try this: add noise so a few Pass/Fail labels are wrong (flip 3 labels) and re-run section 3. Does the perceptron still converge? *(No. The data is no longer linearly separable, so it keeps making mistakes, just like XOR.)*

04 · Activation functions: see them, break them, compare them

Lectures: *Activation Functions in Neural Networks* (07-08-2026: Step, Linear), *(Cont.)* (11-08-2026: Sigmoid, tanh, ReLU, Leaky ReLU, PReLU) and *(Cont.)* (13-08-2026: ELU, GELU, Swish/SiLU, Softmax, choosing activations) · **Notes:** `../notes/04_activation_functions.md`

In this notebook you will:

1. Plot all seven activations **and their derivatives** (the derivative is what training uses).
2. **Prove** that stacking linear layers is pointless without a non-linear activation.
3. Measure the **vanishing-gradient** problem of sigmoid and tanh.
4. Create **dying ReLU** neurons on purpose, and watch Leaky ReLU and PReLU avoid them.
5. Train the same network with each activation and compare honestly.
6. Meet the modern smooth activations (ELU, GELU, SiLU) and **Softmax**, and learn which output activation fits which task.

Runs in about 2–3 minutes on a laptop CPU.

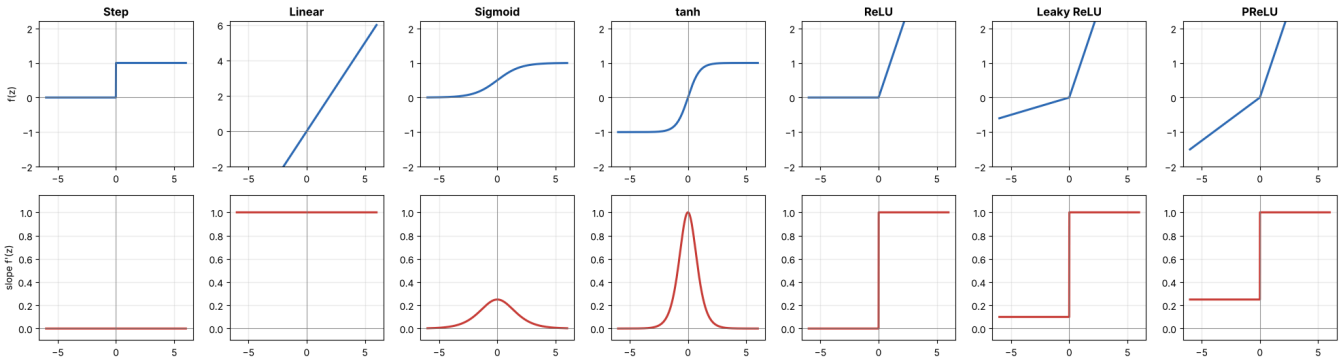
```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn
torch.manual_seed(0); np.random.seed(0)
```

1 · The seven functions and their slopes

Function	Formula	Derivative (slope)
Step	1 if z ≥ 0 else 0	0 (undefined at 0)
Linear	z	1
Sigmoid σ	$1 / (1 + e^{-z})$	$\sigma(z)\cdot(1 - \sigma(z)) \le 0.25$
tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	$1 - \tanh^2(z) \le 1$
ReLU	max(0, z)	1 if z > 0, else 0
Leaky ReLU	z if z > 0, else α·z (α fixed, e.g. 0.01)	1 or α
PReLU	z if z > 0, else a·z (**a learned**)	1 or a

```
z = np.linspace(-6, 6, 1201)
sig = 1 / (1 + np.exp(-z))
alpha, a_prelu = 0.1, 0.25      # alpha exaggerated (usual is 0.01) so the leak is visible; 0.25
= PyTorch's PReLU start value
funcs = {
    "Step":       ((z >= 0).astype(float),          np.zeros_like(z)),
    "Linear":     (z,                               np.ones_like(z)),
    "Sigmoid":    (sig,                             sig * (1 - sig)),
    "tanh":       (np.tanh(z),                      1 - np.tanh(z) ** 2),
    "ReLU":       (np.maximum(0, z),                (z > 0).astype(float)),
```

```
    "Leaky ReLU": (np.where(z > 0, z, alpha * z),    np.where(z > 0, 1, alpha)),
    "PReLU":      (np.where(z > 0, z, a_prelu * z),  np.where(z > 0, 1, a_prelu)),
}
fig, axes = plt.subplots(2, 7, figsize=(20, 5.5))
for j, (name, (f, d)) in enumerate(funcs.items()):
    axes[0, j].plot(z, f, lw=2.2, color="#2f6db5"); axes[0, j].set_title(name, fontweight="bold")
    axes[1, j].plot(z, d, lw=2.2, color="#c8403a")
    for ax in axes[:, j]:
        ax.axhline(0, color="grey", lw=0.6); ax.axvline(0, color="grey", lw=0.6);
ax.grid(alpha=0.25)
    axes[0, j].set_ylim(-2, 2.2 if name != "Linear" else 6.2)
    axes[1, j].set_ylim(-0.1, 1.15)
axes[0, 0].set_ylabel("f(z)"); axes[1, 0].set_ylabel("slope f'(z)")
plt.tight_layout(); plt.show()
print("max sigmoid slope:", round((sig * (1 - sig)).max(), 4), " max tanh slope:", round((1 -
np.tanh(z) ** 2).max(), 4))
```




```
max sigmoid slope: 0.25  max tanh slope: 1.0
```

Read the **bottom row**. During training, backpropagation multiplies these slopes together layer after layer:

- **Step:** slope 0 everywhere, so there is no learning signal (why the perceptron rule isn't gradient descent).
- **Sigmoid:** slope at most **0.25**, and near 0 once $|z| > 4$ (saturation).
- **tanh:** slope up to **1** at zero, but also near 0 once saturated.
- **ReLU:** slope exactly **1** for positive z, but **0** for negative z (the dying-ReLU risk).
- **Leaky ReLU / PReLU:** never zero, so a small signal always gets through.

Useful values: $\sigma(0) = 0.5$ and $\tanh(0) = 0$. **tanh is zero-centred, sigmoid is not.**

```
for v in [-3, -1, 0, 1, 3]:
    s = 1 / (1 + np.exp(-v))
    print(f"z = {v:>2}:  sigmoid = {s:.3f}  tanh = {np.tanh(v):+.3f}  ReLU = {max(0, v)}
LeakyReLU(0.01) = {v if v > 0 else 0.01 * v:+.2f}")
```

```
z = -3:  sigmoid = 0.047  tanh = -0.995  ReLU = 0  LeakyReLU(0.01) = -0.03
z = -1:  sigmoid = 0.269  tanh = -0.762  ReLU = 0  LeakyReLU(0.01) = -0.01
z =  0:  sigmoid = 0.500  tanh = +0.000  ReLU = 0  LeakyReLU(0.01) = +0.00
z =  1:  sigmoid = 0.731  tanh = +0.762  ReLU = 1  LeakyReLU(0.01) = +1.00
z =  3:  sigmoid = 0.953  tanh = +0.995  ReLU = 3  LeakyReLU(0.01) = +3.00
```

2 · Why linear layers alone are not enough (a proof in code)

Two linear layers: **W2·(W1·x + b1) + b2 = (W2·W1)·x + (W2·b1 + b2)**. That's just **one** linear layer. However many you stack, the network can only draw straight lines.

```
torch.manual_seed(1)
deep_linear = nn.Sequential(nn.Linear(2, 16), nn.Linear(16, 16), nn.Linear(16, 16), nn.Linear(16,
1))  # 4 layers, NO activation
# Collapse it into ONE equivalent linear layer
W, b = torch.eye(2), torch.zeros(2)
for layer in deep_linear:
    W, b = layer.weight @ W, layer.weight @ b + layer.bias
x = torch.randn(5, 2)
print("4-layer linear network:", deep_linear(x).detach().squeeze().numpy().round(4))
print("single collapsed layer:", (x @ W.T + b).detach().squeeze().numpy().round(4))
print("identical:", torch.allclose(deep_linear(x), x @ W.T + b, atol=1e-5))
```

```
4-layer linear network: [-0.1407 -0.2472 -0.2158 -0.044  -0.3272]
single collapsed layer: [-0.1407 -0.2472 -0.2158 -0.044  -0.3272]
identical: True
```

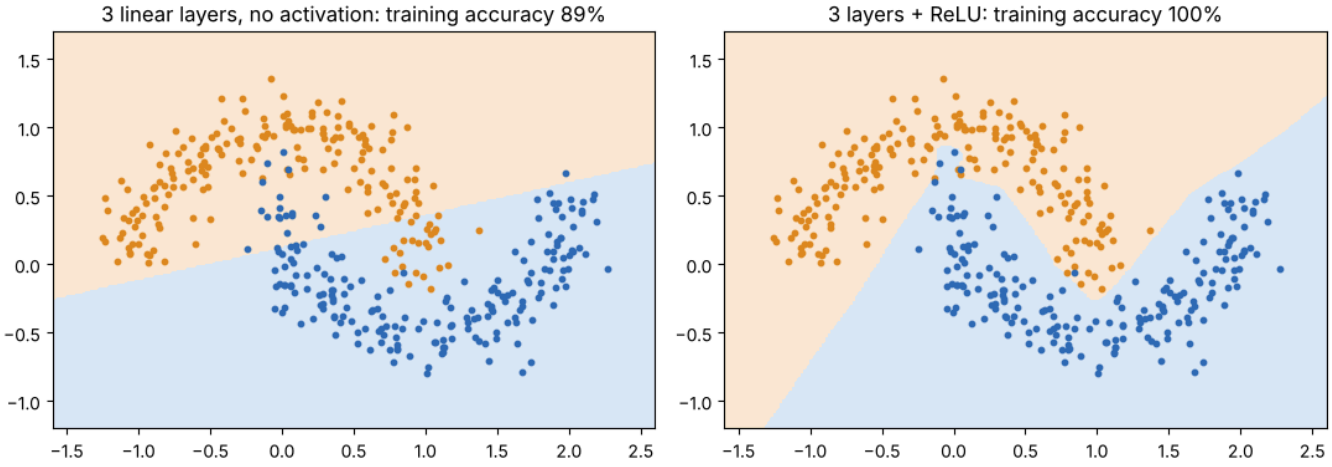
Now the practical consequence: train a deep **linear** network and a deep **ReLU** network on data that needs a curved boundary (two interleaving half-moons).

```
from sklearn.datasets import make_moons
Xm, ym = make_moons(400, noise=0.15, random_state=0)
Xm_t, ym_t = torch.tensor(Xm, dtype=torch.float32), torch.tensor(ym,
dtype=torch.float32).unsqueeze(1)

def make_net(act, depth=3, width=32):
    layers, d_in = [], 2
    for _ in range(depth):
        layers += [nn.Linear(d_in, width)] + ([act()] if act else [])
        d_in = width
    return nn.Sequential(*layers, nn.Linear(d_in, 1))

def train_bin(net, X, y, epochs=600, lr=0.01):
    opt = torch.optim.Adam(net.parameters(), lr=lr)
    for _ in range(epochs):
        loss = nn.functional.binary_cross_entropy_with_logits(net(X), y)
        opt.zero_grad(); loss.backward(); opt.step()
    return ((net(X) > 0).float() == y).float().mean().item()

g = np.linspace(-1.6, 2.6, 200); GX, GY = np.meshgrid(g, np.linspace(-1.2, 1.7, 200))
grid = torch.tensor(np.column_stack([GX.ravel(), GY.ravel()]), dtype=torch.float32)
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, (name, act) in zip(axes, [("3 linear layers, no activation", None), ("3 layers + ReLU",
nn.ReLU)]):
    torch.manual_seed(0); net = make_net(act)
    acc = train_bin(net, Xm_t, ym_t)
    Z = (net(grid) > 0).float().detach().numpy().reshape(GX.shape)
    ax.contourf(GX, GY, Z, levels=1, colors=["#fbe9d2", "#dbe7f6"])
    ax.scatter(*Xm[ym == 0].T, s=10, c="#e08a1e"); ax.scatter(*Xm[ym == 1].T, s=10, c="#2f6db5")
    ax.set_title(f"{name}: training accuracy {acc:.0%}")
plt.tight_layout(); plt.show()
```



Without an activation, the "deep" network can only draw a **straight line**. With ReLU it bends around the moons.

3 · Vanishing gradients: measured

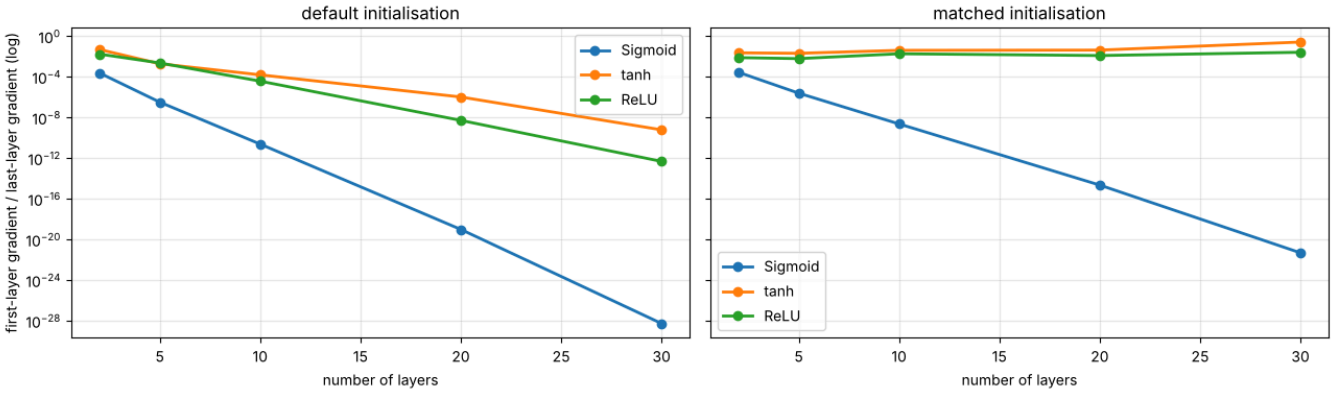
A deep stack of layers. We send one batch through it, backpropagate, and measure how big the gradient is at the **first** layer, compared with the last layer. We try two ways of choosing the starting weights:

- **PyTorch's default** initialisation, used for every activation;
- the **matching** initialisation that practitioners pair with each activation: **He (Kaiming)** for ReLU and **Xavier (Glorot)** for sigmoid and tanh. These scale the random starting weights so the signal neither shrinks nor explodes from layer to layer.

```
def first_layer_grad(act, depth, init="default", width=64, seed=0):
    torch.manual_seed(seed)
    layers = []
    for _ in range(depth):
        lin = nn.Linear(width, width)
        if init == "matched":
            if act is nn.ReLU:
                nn.init.kaiming_normal_(lin.weight, nonlinearity="relu")          # He
initialisation
            else:
                gain = nn.init.calculate_gain("tanh" if act is nn.Tanh else "sigmoid")
                nn.init.xavier_normal_(lin.weight, gain=gain)                     # Xavier
initialisation
            nn.init.zeros_(lin.bias)
        layers += [lin, act()]
    net = nn.Sequential(*layers, nn.Linear(width, 1))
    x = torch.randn(256, width)
    net(x).pow(2).mean().backward()
    return net[0].weight.grad.abs().mean().item() / net[-1].weight.grad.abs().mean().item()

depths = [2, 5, 10, 20, 30]
acts = {"Sigmoid": nn.Sigmoid, "tanh": nn.Tanh, "ReLU": nn.ReLU}
fig, axes = plt.subplots(1, 2, figsize=(13, 4), sharey=True)
for ax, init in zip(axes, ["default", "matched"]):
    for name, a in acts.items():
        r = [first_layer_grad(a, d, init) for d in depths]
        ax.semilogy(depths, r, "o-", label=name, lw=2)
        print(f"{init:>8} init, {name:>7}: " + "  ".join(f"{d} layers {v:.0e}" for d, v in
zip(depths, r)))
    ax.set_title(f"{init} initialisation"); ax.set_xlabel("number of layers"); ax.grid(alpha=0.3,
which="both"); ax.legend()
axes[0].set_ylabel("first-layer gradient / last-layer gradient (log)")
plt.tight_layout(); plt.show()
```

```
 default init, Sigmoid: 2 layers 2e-04  5 layers 3e-07  10 layers 2e-11  20 layers 1e-19  30 laye
rs 5e-29
 default init,    tanh: 2 layers 5e-02  5 layers 2e-03  10 layers 2e-04  20 layers 1e-06  30 laye
rs 6e-10
 default init,    ReLU: 2 layers 2e-02  5 layers 2e-03  10 layers 4e-05  20 layers 5e-09  30 laye
rs 5e-13
 matched init, Sigmoid: 2 layers 3e-04  5 layers 2e-06  10 layers 2e-09  20 layers 2e-15  30 laye
rs 5e-22
 matched init,    tanh: 2 layers 2e-02  5 layers 2e-02  10 layers 4e-02  20 layers 4e-02  30 laye
rs 3e-01
 matched init,    ReLU: 2 layers 8e-03  5 layers 6e-03  10 layers 2e-02  20 layers 1e-02  30 laye
rs 3e-02
```



What it shows:

- **Sigmoid vanishes either way.** With matched initialisation, the gradient reaching layer 1 of a 30-layer net is still only about 10^{-22} of the last layer's, so those early layers can't learn. Its slope is at most 0.25, and multiplying 30 such factors together is hopeless. This is the lecture's **vanishing-gradient problem**.
- **tanh and ReLU need the matching initialisation.** With PyTorch's default starting weights, both also lose most of the signal as depth grows (ReLU at least as badly as tanh here). With **He for ReLU and Xavier for tanh**, the gradient stays roughly level even at 30 layers.

> **Lesson:** "ReLU fixes vanishing gradients" is half the story. ReLU's slope of 1 *allows* the signal to survive, but only with sensible starting weights (He initialisation). Standard models pair them for you, and add normalisation layers (notebook 05).

4 · Dying ReLU

A ReLU neuron whose input is negative for **every** example outputs 0 and has slope 0. It gets **no gradient, ever again**: it is **dead**.

4a · One neuron, seen up close

We teach a single neuron the function y = x on inputs 0...1, starting from a badly negative bias (b = −2), so z = w·x + b < 0 for every input.

```
x1 = torch.linspace(0, 1, 50).unsqueeze(1); target = x1.clone()
for name, f in [("ReLU", torch.relu), ("Leaky ReLU (alpha 0.01)", lambda t:
nn.functional.leaky_relu(t, 0.01))]:
    w = torch.tensor([1.0], requires_grad=True); b = torch.tensor([-2.0], requires_grad=True)
    opt = torch.optim.SGD([w, b], lr=0.5)
    for step_i in range(2001):
        loss = ((f(x1 * w + b) - target) ** 2).mean()
        opt.zero_grad(); loss.backward()
        if step_i in (0, 2000):
            print(f"{name:>23} step {step_i:4d}: loss {loss.item():.4f}  bias {b.item():+.3f}
gradient on bias {b.grad.item():+.1e}")
        opt.step()
```

```
                  ReLU step    0: loss 0.3367  bias -2.000  gradient on bias +0.0e+00
                  ReLU step 2000: loss 0.3367  bias -2.000  gradient on bias +0.0e+00
Leaky ReLU (alpha 0.01) step    0: loss 0.3502  bias -2.000  gradient on bias -1.0e-02
Leaky ReLU (alpha 0.01) step 2000: loss 0.0000  bias -0.000  gradient on bias +6.3e-09
```

- **ReLU:** the gradient is **exactly 0**, at step 0 and still at step 2000. Nothing ever changes, and the neuron is permanently dead.
- **Leaky ReLU:** a small but non-zero gradient keeps nudging the bias up until the neuron comes back to life and fits the data (loss → 0).

4b · A whole network with most neurons knocked out

Imagine a bad update (or a bad start) has pushed every hidden bias to −0.35. We count the neurons that are never positive for any training image, then train with plain SGD.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
digits = load_digits()
Xd = digits.data / 16.0; yd = digits.target
Xtr, Xte, ytr, yte = train_test_split(Xd, yd, test_size=0.3, random_state=0, stratify=yd)
Xtr, Xte = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xte, dtype=torch.float32)
ytr, yte = torch.tensor(ytr), torch.tensor(yte)

def digit_net(act, depth=3, width=64, bias=None, init="default"):
    layers, d_in = [], 64
    for _ in range(depth):
        lin = nn.Linear(d_in, width)
        if init == "he":
            nn.init.kaiming_normal_(lin.weight, nonlinearity="relu"); nn.init.zeros_(lin.bias)
        if bias is not None:
            nn.init.constant_(lin.bias, bias)
        layers += [lin, act()]; d_in = width
    return nn.Sequential(*layers, nn.Linear(d_in, 10))

def train_digits(net, lr, epochs=40, opt_name="sgd"):
    opt = torch.optim.SGD(net.parameters(), lr=lr) if opt_name == "sgd" else
torch.optim.Adam(net.parameters(), lr=lr)
    for _ in range(epochs):
        perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), 64):
            idx = perm[i:i + 64]
            loss = nn.functional.cross_entropy(net(Xtr[idx]), ytr[idx])
            opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        return (net(Xte).argmax(1) == yte).float().mean().item()

def never_positive(net, X):
    """share of hidden neurons whose output is <= 0 for EVERY input"""
    count, total, h = 0, 0, X
    with torch.no_grad():
        for layer in net[:-1]:
            h = layer(h)
            if not isinstance(layer, nn.Linear):
                count += (h <= 0).all(dim=0).sum().item(); total += h.shape[1]
    return count / total

print(f"{'activation':>23} {'never positive: before':>23} {'after':>6} {'test acc':>9}")
for name, act in [("ReLU", nn.ReLU), ("Leaky ReLU (alpha 0.01)", lambda: nn.LeakyReLU(0.01)),
                  ("Leaky ReLU (alpha 0.1)", lambda: nn.LeakyReLU(0.1)), ("PReLU (learned
slope)", nn.PReLU)]:
    torch.manual_seed(0); net = digit_net(act, bias=-0.35)
    before = never_positive(net, Xtr)
    acc = train_digits(net, lr=0.1, epochs=60)
    print(f"{name:>23} {before:>23.0%} {never_positive(net, Xtr):>6.0%} {acc:>9.1%}")
```

```
             activation  never positive: before  after  test acc
                   ReLU                     77%    77%     10.2%
Leaky ReLU (alpha 0.01)                     77%    77%     10.2%
```

```
 Leaky ReLU (alpha 0.1)                    77%     77%      10.2%
  PReLU (learned slope)                    77%      0%      97.8%
```

Honest reading:

- **ReLU** stays stuck at chance (about 10% for 10 digits). Its dead neurons never recover.
- **Leaky ReLU with a fixed slope** does have a non-zero gradient, as the lecture says, and 4a showed that saves a single neuron. But here the signal must pass through **three** leaky layers (0.01 × 0.01 × 0.01), so with plain SGD and 60 epochs this network **also** stayed stuck, even with a 10× bigger slope (α = 0.1). "Reduces the risk" is not "removes the risk".
- **PReLU** learned to **change its slope** and escaped completely, reaching high accuracy. This is the flexibility the lecture credits it with.

PReLU's learned slopes (normal start, normal training)

```
torch.manual_seed(0); net = digit_net(nn.PReLU); acc = train_digits(net, 0.1)
slopes = [m.weight.item() for m in net if isinstance(m, nn.PReLU)]
print("PReLU negative slopes: start 0.25 each -> learned:", [round(s, 3) for s in slopes], f"
(test acc {acc:.1%})")
```

```
PReLU negative slopes: start 0.25 each -> learned: [1.794, 1.341, 2.062]   (test acc 94.8%)
```

Each PReLU layer has one extra trainable number (its negative-side slope `a`), starting at 0.25, and training moved them. A learned slope can even end up **above 1**: nothing forces it to stay small, so the network picks whatever helps. With `nn.PReLU(num_parameters=width)` you get one slope **per neuron**, which is more flexible but adds more parameters (the lecture's overfitting caveat).

5 · A fair comparison: the same network, each activation

A 6-hidden-layer network on digits, trained identically (Adam, lr 0.001, 30 epochs), averaged over 3 seeds. First all with **PyTorch's default initialisation**, then ReLU again with **He initialisation**.

```
for name, act in [("Sigmoid", nn.Sigmoid), ("tanh", nn.Tanh), ("ReLU", nn.ReLU), ("LeakyReLU",
lambda: nn.LeakyReLU(0.01)), ("PReLU", nn.PReLU)]:
    accs = []
    for seed in range(3):
        torch.manual_seed(seed); net = digit_net(act, depth=6)
        accs.append(train_digits(net, 1e-3, epochs=30, opt_name="adam"))
    print(f"{name:>15}: test accuracy {np.mean(accs):.1%} ± {np.std(accs):.1%}")
accs = []
for seed in range(3):
    torch.manual_seed(seed); net = digit_net(nn.ReLU, depth=6, init="he")
    accs.append(train_digits(net, 1e-3, epochs=30, opt_name="adam"))
print(f"{'ReLU + He init':>15}: test accuracy {np.mean(accs):.1%} ± {np.std(accs):.1%}")
```

```
        Sigmoid: test accuracy 27.3% ± 6.0%
           tanh: test accuracy 95.2% ± 0.7%
           ReLU: test accuracy 91.0% ± 2.4%
      LeakyReLU: test accuracy 93.1% ± 1.9%
          PReLU: test accuracy 94.6% ± 0.2%
 ReLU + He init: test accuracy 96.9% ± 0.5%
```

Honest reading of these numbers:

- **Sigmoid fails badly** at 6 layers (around 27%, near chance for 10 classes): that's the vanishing gradient in practice.
- With PyTorch's **default** initialisation, **tanh** (95.2%) actually beat plain **ReLU** (91.0%), with Leaky ReLU (93.1%) and PReLU (94.6%) in between.
- Give ReLU its matching **He initialisation** and it becomes the **best of all (96.9%)**, as section 3 predicted.
- **Takeaway:** "use ReLU" really means "use ReLU **with He initialisation**". Any ranking of activations depends on the setup (initialisation, optimiser, depth, data), so compare on your own validation data, as the lecture's summary says.

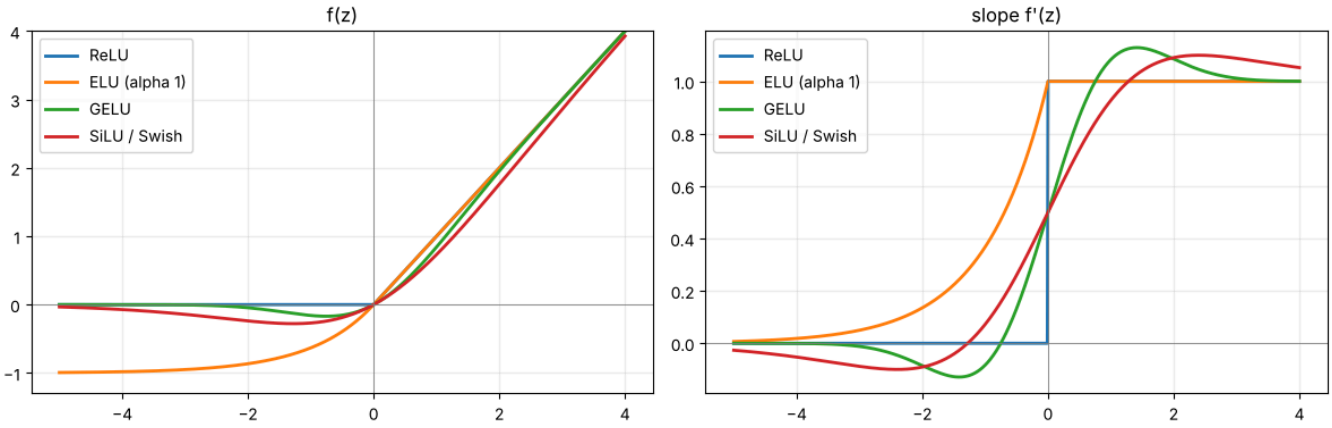
6 · Part 3 (13-08 lecture): ELU, GELU, Swish/SiLU and Softmax

Function	Formula	Negative side
ELU	z if z > 0, else α·(eᶻ − 1) (α = 1 by default)	smooth curve, levels off at **−α**
GELU	z·Φ(z), where Φ = the standard normal CDF	small negative dip, then → 0
Swish / SiLU	z·σ(z)	small negative dip, then → 0
Softmax	$p_i = e^{z_i} / \Sigma_j e^{z_j}$, applied to a **vector**	(not an element-wise activation)

GELU and SiLU are "soft" ReLUs: instead of a hard switch at 0, they multiply z by a number between 0 and 1 that grows smoothly with z (Φ(z) or σ(z)). That's the lecture's "**softly gates** inputs rather than hard-thresholding them".

```
zt = torch.linspace(-5, 4, 901, requires_grad=True)
new_funcs = {"ReLU": torch.relu, "ELU (alpha 1)": nn.functional.elu, "GELU": nn.functional.gelu,
"SiLU / Swish": nn.functional.silu}
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for name, f in new_funcs.items():
    out = f(zt); (grad,) = torch.autograd.grad(out.sum(), zt)
    axes[0].plot(zt.detach(), out.detach(), lw=2, label=name); axes[1].plot(zt.detach(), grad,
lw=2, label=name)
for ax, t in zip(axes, ["f(z)", "slope f'(z)"]):
    ax.axhline(0, color="grey", lw=0.6); ax.axvline(0, color="grey", lw=0.6);
ax.grid(alpha=0.25); ax.set_title(t); ax.legend()
axes[0].set_ylim(-1.3, 4); plt.tight_layout(); plt.show()

zz = torch.tensor([-3.0, -1.0, -0.5, 0.0, 0.5, 1.0, 3.0])
print("z             :", zz.tolist())
for name, f in new_funcs.items():
    print(f"{name:<14}:", [round(v, 3) for v in f(zz).tolist()])
zfine = torch.linspace(-5, 0, 50001)
for name in ["GELU", "SiLU / Swish"]:
    v = new_funcs[name](zfine); i = v.argmin()
    print(f"lowest {name} value: {v[i].item():.3f} at z = {zfine[i].item():.2f}")
print("ELU at z = -10:", round(nn.functional.elu(torch.tensor(-10.0)).item(), 5), "(bounded below
by -alpha = -1)")
```




```
z             : [-3.0, -1.0, -0.5, 0.0, 0.5, 1.0, 3.0]
ReLU          : [0.0, 0.0, 0.0, 0.0, 0.5, 1.0, 3.0]
ELU (alpha 1) : [-0.95, -0.632, -0.393, 0.0, 0.5, 1.0, 3.0]
GELU          : [-0.004, -0.159, -0.154, 0.0, 0.346, 0.841, 2.996]
SiLU / Swish  : [-0.142, -0.269, -0.189, 0.0, 0.311, 0.731, 2.858]
lowest GELU value: -0.170 at z = -0.75
lowest SiLU / Swish value: -0.278 at z = -1.28
ELU at z = -10: -0.99995 (bounded below by -alpha = -1)
```

What to notice:

- **ELU** equals ReLU for z > 0, but for z < 0 it curves smoothly down towards **−1** (= −α), so it can output negative values and its slope never drops to exactly 0.
- **GELU** and **SiLU** dip slightly below zero (lowest values about −0.17 and −0.28) and then return towards 0. Going down and then up means they are **non-monotonic**, which the lecture lists for Swish (GELU has the same property).
- All three are **smooth at 0** (no corner), unlike ReLU.
- **GELU** is the activation inside BERT and GPT-style Transformer feed-forward blocks. **SiLU** appears in, for example, EfficientNet and LLaMA's feed-forward blocks (as part of SwiGLU).

Softmax: the lecture's example

Logits: cat 2.5, dog 1.2, horse 0.3.

```
logits = torch.tensor([2.5, 1.2, 0.3])
p = torch.softmax(logits, dim=0)
for name, zv, pv in zip(["cat", "dog", "horse"], logits.tolist(), p.tolist()):
    print(f"{name:<6} logit {zv:<4.1f} -> e^z = {np.exp(zv):6.3f} -> probability {pv:.3f}")
print("sum of probabilities:", round(p.sum().item(), 6))
```

```
cat    logit 2.5  -> e^z = 12.182 -> probability 0.723
dog    logit 1.2  -> e^z =  3.320 -> probability 0.197
horse  logit 0.3  -> e^z =  1.350 -> probability 0.080
sum of probabilities: 1.0
```

The probabilities are **0.723 / 0.197 / 0.080**. (The lecture's table rounds them to 0.73 / 0.20 / **0.07**; the correct rounding for horse is **0.08**, and only then do they sum to 1.)

Numerically stable softmax

e^z overflows for large z (e^1000 is infinity in floating point). The fix: subtract the largest logit first. This gives exactly the same answer, because the common factor e^{−max} cancels top and bottom.

```
big = np.array([1000.0, 999.0, 998.0])
with np.errstate(over="ignore", invalid="ignore"):
    naive = np.exp(big) / np.exp(big).sum()
stable = np.exp(big - big.max()) / np.exp(big - big.max()).sum()
print("naive  softmax:", naive)
print("stable softmax:", stable.round(4))
print("torch.softmax :", torch.softmax(torch.tensor(big), 0).numpy().round(4), "(built-ins are
already stable)")
```

```
naive  softmax: [nan nan nan]
stable softmax: [0.6652 0.2447 0.09  ]
torch.softmax : [0.6652 0.2447 0.09  ] (built-ins are already stable)
```

This is why the lecture says "use numerically stable implementations": use `torch.softmax` / `F.cross_entropy` rather than writing `exp(z) / exp(z).sum()` yourself.

Multi-class vs multi-label: softmax or independent sigmoids?

- **Single-label** (exactly one answer: cat *or* dog *or* horse) → **softmax**: the classes compete and the probabilities sum to 1.
- **Multi-label** (several can be true: a photo containing a cat *and* a dog) → **one sigmoid per label**: each is an independent yes/no.

```
scores = torch.tensor([3.0, 2.8, -2.0])          # a photo that clearly has a cat AND a dog, no
horse
print("softmax (classes compete):", torch.softmax(scores, 0).numpy().round(3), " sum =",
round(torch.softmax(scores, 0).sum().item(), 3))
print("sigmoids (independent)   :", torch.sigmoid(scores).numpy().round(3))
```

```
softmax (classes compete): [0.548 0.449 0.004]  sum = 1.0
sigmoids (independent)   : [0.953 0.943 0.119]
```

Softmax **forces** cat and dog to share the probability (0.548 and 0.449), so neither looks certain, even though both are present. Independent sigmoids say "cat: yes (0.953), dog: yes (0.943), horse: no (0.119)". Use sigmoids (with BCE, notebook 06) for multi-label.

ReLU sparsity: the lecture's example

```
inp = torch.tensor([-3.0, -1.0, 0.5, 2.0, -4.0, 1.0])
out = torch.relu(inp)
print("input :", inp.tolist()); print("ReLU  :", out.tolist())
print(f"exact zeros: {(out == 0).sum().item()} of {len(out)}")
torch.manual_seed(0)
h = torch.relu(digit_net(nn.ReLU)[0](Xtr))      # first hidden layer of an untrained digits
network
print(f"in a real (untrained) ReLU layer on the digits: {(h == 0).float().mean().item():.0%} of
the activations are exactly 0")
```

```
input : [-3.0, -1.0, 0.5, 2.0, -4.0, 1.0]
ReLU  : [0.0, 0.0, 0.5, 2.0, 0.0, 1.0]
exact zeros: 3 of 6
in a real (untrained) ReLU layer on the digits: 51% of the activations are exactly 0
```

"Sparse" means **many outputs are exactly zero**: 3 of 6 here, and 51% in a real untrained layer. Only some neurons "fire" for each input, which is cheap to compute and often makes the features easier to separate.

How do ELU, GELU and SiLU do on our digits benchmark?

Same setup as section 5 (6 hidden layers, Adam, lr 0.001, 30 epochs, 3 seeds, PyTorch default initialisation).

```
for name, act in [("ELU", nn.ELU), ("GELU", nn.GELU), ("SiLU", nn.SiLU)]:
    accs = []
    for seed in range(3):
        torch.manual_seed(seed); net = digit_net(act, depth=6)
        accs.append(train_digits(net, 1e-3, epochs=30, opt_name="adam"))
    print(f"{name:>15}: test accuracy {np.mean(accs):.1%} ± {np.std(accs):.1%}")
```

```
            ELU: test accuracy 95.1% ± 0.5%
           GELU: test accuracy 93.4% ± 1.6%
           SiLU: test accuracy 92.5% ± 1.5%
```

Honest reading: ELU 95.1%, GELU 93.4%, SiLU 92.5%. With the same default initialisation, all three beat plain ReLU (91.0%), and ELU matched tanh (95.2%). But none beat **ReLU + He initialisation (96.9%)**. On a small task like this, the newer activations are not automatically better: exactly the lecture's closing advice, "validate it on the actual task". (Differences of 1–2 points are within the seed-to-seed spread.)

Choosing the output activation (lecture table)

Task	Output activation	Why
binary classification	**sigmoid**	one probability between 0 and 1
single-label multi-class	**softmax**	the probabilities form one distribution
multi-label	**one sigmoid per label**	independent yes/no decisions
unrestricted regression	**linear / identity**	the output can be any real number
positive-only regression (prices, counts)	often **softplus** ($\log(1 + e^z)$) or another positive transform	guarantees a positive output
bounded regression	**scaled sigmoid / tanh**	restricts the output to a known interval

7 · Which activation where (lecture summary, now with evidence)

Situation	Use	Evidence in this notebook
hidden layers (default)	**ReLU** (+ He initialisation)	gradient stays level with He initialisation (section 3); best accuracy with He initialisation (section 5)
dying-ReLU concern	**Leaky ReLU**	revives a dead neuron (4a), but a small fixed slope may not rescue a badly broken deep net (4b)
want the leak learned	**PReLU**	learned its way out of 77% knocked-out neurons (4b)
bounded, zero-centred hidden state (RNN/LSTM)	**tanh**	tanh(0) = 0, range (−1, 1); strong with default initialisation (section 5)

Situation	Use	Evidence in this notebook
binary classification output	**Sigmoid**	output in (0, 1), a probability; a poor hidden-layer choice (sections 3 and 5)
regression output	**Linear**	unbounded continuous output
classic perceptron only	**Step**	slope 0, so it can't be trained by gradient descent
smooth ReLU alternative with negative outputs	**ELU**	levels off at −α; smooth (section 6)
Transformer-style models	**GELU**	the standard choice in BERT/GPT feed-forward blocks
modern CNNs / LLM blocks that specify it	**Swish / SiLU**	smooth, non-monotonic (section 6)
single-label multi-class output	**Softmax**	probabilities summing to 1 (section 6)
multi-label output	**independent sigmoids**	softmax would force labels to compete (section 6)

The lecture's rule of thumb: start with ReLU in ordinary hidden layers; if neurons die, try Leaky ReLU/PReLU and inspect the activations; use GELU or SiLU when the architecture calls for them; and **never choose an activation just because it's newer: validate it on your task**.

✅ Check yourself

1. What is the maximum slope of sigmoid, and why does that cause vanishing gradients in deep networks? *(0.25; multiplied over many layers it shrinks towards 0)*
2. Why can't a 10-layer network with linear activations learn the moons? *(it collapses to one linear layer, so a straight boundary)*
3. Vanishing gradient vs dying ReLU: what's the difference? *(vanishing = slopes small but non-zero, from saturating sigmoid/tanh; dying ReLU = slope exactly 0 because the input is always negative)*
4. Leaky ReLU vs PReLU? *(Leaky: fixed slope α chosen by you; PReLU: slope learned during training)*
5. Which activation would you put on the output of a house-price model? A spam detector? *(Linear; Sigmoid)*
6. Why is "ReLU solves vanishing gradients" only half true? *(ReLU's slope of 1 lets the signal through, but only with suitable initialisation, i.e. He; with poor starting weights the signal still shrinks)*
7. Softmax of logits (2, 0)? *($e^2 / (e^2 + 1) = 0.881$ and 0.119)*
8. A photo can be tagged "beach", "sunset" and "people" at the same time. Softmax or sigmoids? *(independent sigmoids: it's multi-label)*
9. What is the lowest value ELU (α = 1) can output? *(it approaches −1 but never reaches it)*
10. Why subtract max(z) before computing softmax? *(prevents e^z overflowing; the answer is unchanged)*

05 · Normalization and regularization, by experiment

Lecture: *Normalization and Regularization in Neural Networks* (04-09-2026) · **Notes:** `../notes/05_normalization_and_regularization.md`

Two different problems, two different toolboxes:

	Normalization	Regularization
Problem it solves	badly **scaled** numbers make optimisation slow or unstable	the model **memorises** the training set (overfitting)
Goal	stable, fast training	better results on **unseen** data
Tools	standardisation, min-max, BatchNorm, LayerNorm	L1, L2, dropout, early stopping, augmentation, weight decay

In this notebook you will:

1. **See** gradient descent fail on unscaled features and fly on standardised ones.
2. Compare min-max and standardisation with an outlier, and see why test statistics must never leak in.
3. Compute **BatchNorm** and **LayerNorm** by hand and check them against PyTorch.
4. Make a network **overfit**, then try every regulariser from the lecture on it, measured honestly.

Runs in about 2–3 minutes on a laptop CPU.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn
rng = np.random.default_rng(0); torch.manual_seed(0)
```

```
<torch._C.Generator at 0x7f109ebd8dd0>
```

1 · Why scale matters: gradient descent on raw vs standardised features

Predict a price from two features with very different scales: **x1 = a rating from 0 to 1**, and **x2 = a floor area from 500 to 3000**. The true rule is y = 3·x1 + 0.002·x2 (+ noise). We fit it with plain gradient descent.

```
n = 200
x1 = rng.uniform(0, 1, n); x2 = rng.uniform(500, 3000, n)
y = 3 * x1 + 0.002 * x2 + rng.normal(0, 0.1, n)
X_raw = np.column_stack([x1, x2])

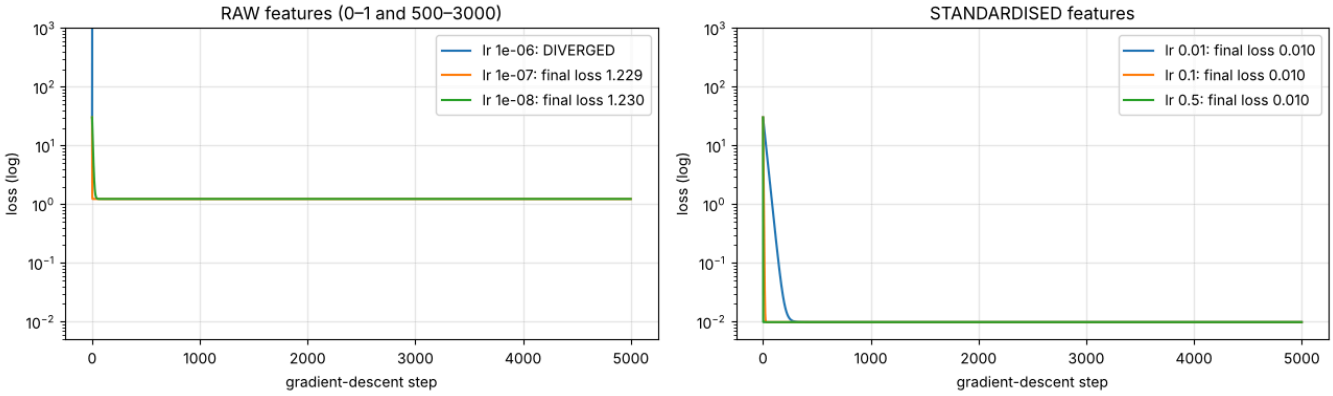
def gradient_descent(X, y, lr, steps=5000):
    w, b, losses = np.zeros(X.shape[1]), 0.0, []
    for _ in range(steps):
        err = X @ w + b - y
        loss = (err ** 2).mean()
```

```
        if not np.isfinite(loss) or loss > 1e6:
            losses.append(np.inf); break
        losses.append(loss)
        w -= lr * 2 * X.T @ err / len(y); b -= lr * 2 * err.mean()
    return np.array(losses)

mu, sigma = X_raw.mean(axis=0), X_raw.std(axis=0)
X_std = (X_raw - mu) / sigma                        # standardisation: (x - mean) / std

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for lr in [1e-6, 1e-7, 1e-8]:
    L = gradient_descent(X_raw, y, lr)
    label = f"lr {lr:g}: " + ("DIVERGED" if np.isinf(L[-1]) else f"final loss {L[-1]:.3f}")
    axes[0].semilogy(L[np.isfinite(L)], label=label); print("raw         ", label)
for lr in [0.01, 0.1, 0.5]:
    L = gradient_descent(X_std, y, lr)
    first = int(np.argmax(L < 0.02)) if (L < 0.02).any() else None
    axes[1].semilogy(L, label=f"lr {lr:g}: final loss {L[-1]:.3f}"); print("standardised", f"lr
{lr:g}: final loss {L[-1]:.4f}, reached < 0.02 after {first} steps")
axes[0].set_title("RAW features (0–1 and 500–3000)"); axes[1].set_title("STANDARDISED features")
for ax in axes:
    ax.set_xlabel("gradient-descent step"); ax.set_ylabel("loss (log)"); ax.set_ylim(5e-3, 1e3);
ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```

```
raw          lr 1e-06: DIVERGED
raw          lr 1e-07: final loss 1.229
raw          lr 1e-08: final loss 1.230
standardised lr 0.01: final loss 0.0098, reached < 0.02 after 199 steps
standardised lr 0.1: final loss 0.0098, reached < 0.02 after 19 steps
standardised lr 0.5: final loss 0.0098, reached < 0.02 after 1 steps
```



What happened:

- **Raw features:** the big feature x2 (hundreds to thousands) produces huge gradients, so the learning rate must be tiny to avoid **diverging**. But at that tiny rate the small feature x1 barely learns, and the loss **stalls**: the model is stuck far from the answer after 5,000 steps.
- **Standardised features:** both features are on the same scale, learning rates up to 100,000× larger are safe, and the loss reaches the noise floor in 199 steps (lr 0.01), 19 steps (lr 0.1) or even a single step (lr 0.5).

This is the lecture's point: normalization doesn't change **what** the model can learn, it makes the optimisation problem **easy to navigate**.

2 · Standardisation vs min-max, and data leakage

- **Standardisation (z-score):** $x' = (x - \mu) / \sigma$ → mean 0, standard deviation 1.
- **Min-max scaling:** $x' = (x - \min) / (\max - \min)$ → range [0, 1].

- **Images:** pixel / 255 → [0, 1].

Outliers: min-max's weakness

```
salaries = np.r_[rng.normal(50, 10, 99), 1000]           # 99 normal salaries (in thousands) +
one billionaire-style outlier
minmax = (salaries - salaries.min()) / (salaries.max() - salaries.min())
zscore = (salaries - salaries.mean()) / salaries.std()
print(f"min-max: the 99 normal values all lie in [{minmax[:99].min():.3f},
{minmax[:99].max():.3f}]  (squashed into {100 * (minmax[:99].max() - minmax[:99].min()):.0f}% of
the range)")
print(f"z-score: the 99 normal values lie in [{zscore[:99].min():.2f}, {zscore[:99].max():.2f}],
the outlier at {zscore[-1]:.1f}")
```

```
min-max: the 99 normal values all lie in [0.000, 0.049]  (squashed into 5% of the range)
z-score: the 99 normal values lie in [-0.35, 0.15], the outlier at 9.9
```

With min-max, one extreme value sets the range, so all the normal values get squashed into a tiny sliver near 0. Standardisation is affected too (the outlier shifts the mean and inflates σ), but less dramatically. For heavy outliers, people use robust scaling (median and interquartile range) or clip the outliers first.

Data leakage: fit the scaler on the training set ONLY

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
X_tr, X_te = train_test_split(X_raw, test_size=0.3, random_state=0)
right = StandardScaler().fit(X_tr)                  # learns mu and sigma from TRAIN data only
wrong = StandardScaler().fit(np.vstack([X_tr, X_te]))  # peeks at the test set
print("mean learned from train only:", right.mean_.round(3), "  from train+test:",
wrong.mean_.round(3))
X_tr_s, X_te_s = right.transform(X_tr), right.transform(X_te)   # SAME train statistics applied
to both
print("test set after correct scaling: mean", X_te_s.mean(axis=0).round(3), "(not exactly 0, and
that's fine)")
```

```
mean learned from train only: [5.360000e-01 1.763709e+03]   from train+test: [5.400000e-01 1.8043
61e+03]
test set after correct scaling: mean [0.045 0.192] (not exactly 0, and that's fine)
```

The rule from the lecture: **compute μ and σ (or min/max) from the training data, then apply those same numbers to validation and test data.** If test data helps choose the scaling, information about the test set has leaked into training, and the reported test score is no longer a fair "unseen data" score. Here the difference in μ is small, but in real projects (time series, small data, data from different hospitals) leakage can make results look much better than they really are.

3 · Normalization inside the network: BatchNorm and LayerNorm

BatchNorm by hand

For each **feature**, using the statistics of the **mini-batch**:

μ_B = mean over the batch · σ²_B = variance over the batch · x̂ = (x − μ_B) / √(σ²_B + ε) · y = γ·x̂ + β (γ, β learned)

```
torch.manual_seed(0)
batch = torch.randn(8, 4) * torch.tensor([1., 10., 0.1, 5.]) + torch.tensor([0., 50., -3., 7.])
# 8 samples, 4 features on wild scales
bn = nn.BatchNorm1d(4)                    # gamma starts at 1, beta at 0
bn.train()
out_torch = bn(batch)

mu_B = batch.mean(dim=0)                  # per FEATURE, across the batch
var_B = batch.var(dim=0, unbiased=False)
out_hand = (batch - mu_B) / torch.sqrt(var_B + bn.eps) * bn.weight + bn.bias
print("matches PyTorch:", torch.allclose(out_hand, out_torch, atol=1e-5))
print("per-feature mean after BN:", out_torch.mean(0).detach().numpy().round(3), " std:",
out_torch.std(0, unbiased=False).detach().numpy().round(3))
print("running mean kept for inference:", bn.running_mean.numpy().round(2), "(moved 10% of the
way towards this batch's mean, momentum 0.1)")
```

```
matches PyTorch: True
per-feature mean after BN: [-0.  0. -0.  0.]  std: [1.    1.    0.998 1.   ]
running mean kept for inference: [ 0.02  4.66 -0.3   0.67] (moved 10% of the way towards this bat
ch's mean, momentum 0.1)
```

Training mode vs inference mode

During training BatchNorm uses **the current batch's** statistics. At inference there may be just **one** example (no meaningful batch), so it uses the **running averages** it collected during training. That's why `model.eval()` matters:

```
single = batch[:1]
bn.train(); print("train mode, one sample:", bn(torch.cat([single, single])).detach()
[0].numpy().round(3), " <- batch of identical rows: variance 0, so everything becomes 0")
bn.eval();  print("eval mode,  one sample:", bn(single).detach()[0].numpy().round(3), " <- uses
running statistics")
```

```
train mode, one sample: [-0.  0.  0.  0.]  <- batch of identical rows: variance 0, so everything
becomes 0
eval mode,  one sample: [-1.09   7.508 -2.727  1.898]  <- uses running statistics
```

LayerNorm by hand

For each **sample**, using the statistics of **its own features**. It doesn't depend on the batch at all, which is why transformers and sequence models use it.

```
ln = nn.LayerNorm(4)
out_ln = ln(batch)
mu_L = batch.mean(dim=1, keepdim=True)              # per SAMPLE, across its features
var_L = batch.var(dim=1, unbiased=False, keepdim=True)
out_ln_hand = (batch - mu_L) / torch.sqrt(var_L + ln.eps) * ln.weight + ln.bias
print("matches PyTorch:", torch.allclose(out_ln_hand, out_ln, atol=1e-5))
print("each SAMPLE now has mean ~0:", out_ln.mean(1).detach().numpy().round(3))
print("same output for one sample alone as inside the batch:", torch.allclose(ln(batch[:1]),
out_ln[:1], atol=1e-6))
```

```
matches PyTorch: True
each SAMPLE now has mean ~0: [ 0. -0. -0.  0.  0. -0.  0.  0.]
same output for one sample alone as inside the batch: True
```

	BatchNorm	LayerNorm
averages over	the **batch** (per feature/channel)	the **features** (per sample)
depends on batch size	yes (very small batches give noisy statistics)	no
train vs eval behaviour	different (running statistics)	identical

	BatchNorm	LayerNorm
typical home	CNNs, feed-forward nets	transformers, RNNs

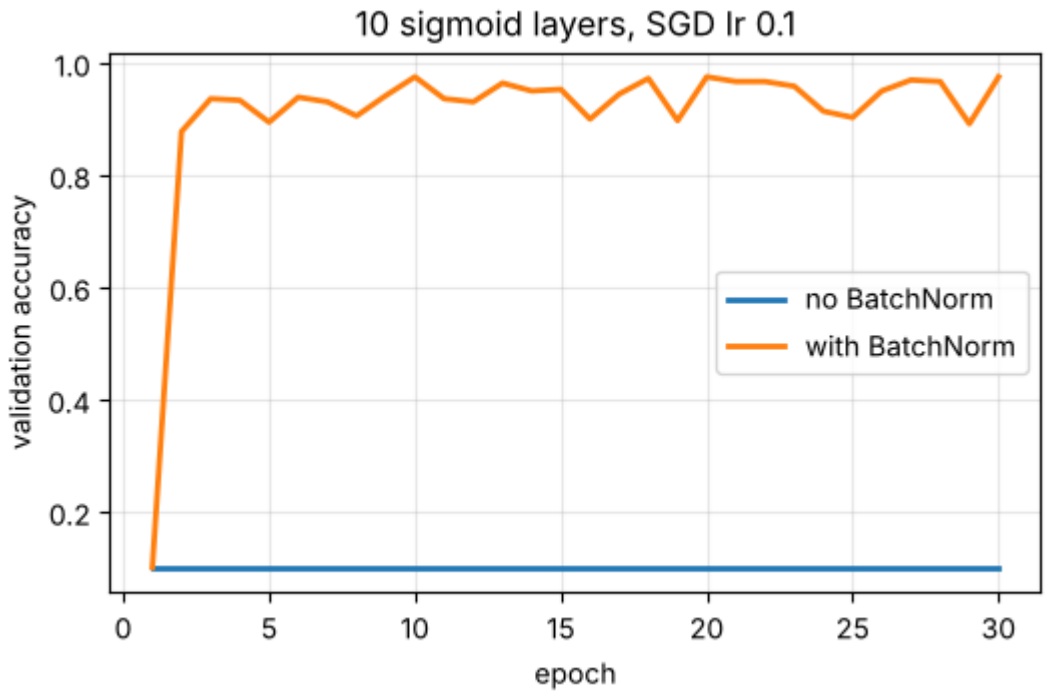
Does BatchNorm actually help training? A 10-layer network on digits

```
from sklearn.datasets import load_digits
digits = load_digits()
Xd, yd = digits.data / 16.0, digits.target       # image normalisation: pixels 0..16 -> 0..1
Xtr_all, Xtmp, ytr_all, ytmp = train_test_split(Xd, yd, test_size=0.4, random_state=0,
stratify=yd)
Xva_all, Xte_all, yva_all, yte_all = train_test_split(Xtmp, ytmp, test_size=0.5, random_state=0,
stratify=ytmp)
T = lambda a: torch.tensor(a, dtype=torch.float32)

def deep_net(use_bn, depth=10, width=64):
    layers, d_in = [], 64
    for _ in range(depth):
        layers += [nn.Linear(d_in, width)] + ([nn.BatchNorm1d(width)] if use_bn else []) +
[nn.Sigmoid()]
        d_in = width
    return nn.Sequential(*layers, nn.Linear(d_in, 10))

curves = {}
for use_bn in [False, True]:
    torch.manual_seed(0); net = deep_net(use_bn)
    opt = torch.optim.SGD(net.parameters(), lr=0.1)
    Xt, yt = T(Xtr_all), torch.tensor(ytr_all)
    accs = []
    for epoch in range(30):
        net.train()
        perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 32):
            idx = perm[i:i + 32]
            loss = nn.functional.cross_entropy(net(Xt[idx]), yt[idx]); opt.zero_grad();
loss.backward(); opt.step()
        net.eval()
        with torch.no_grad():
            accs.append((net(T(Xva_all)).argmax(1) ==
torch.tensor(yva_all)).float().mean().item())
    curves["with BatchNorm" if use_bn else "no BatchNorm"] = accs
    print(f"{'with' if use_bn else 'no  '} BatchNorm: validation accuracy after 5 epochs
{accs[4]:.1%}, after 30 epochs {accs[-1]:.1%}")
plt.figure(figsize=(6, 3.5))
for k, v in curves.items(): plt.plot(range(1, 31), v, label=k, lw=2)
plt.xlabel("epoch"); plt.ylabel("validation accuracy"); plt.title("10 sigmoid layers, SGD lr
0.1"); plt.legend(); plt.grid(alpha=0.3); plt.show()
```

```
no   BatchNorm: validation accuracy after 5 epochs 10.0%, after 30 epochs 10.0%
with BatchNorm: validation accuracy after 5 epochs 89.4%, after 30 epochs 97.5%
```



A deep sigmoid network is the textbook hard case (notebook 04 showed its gradients vanish). Without BatchNorm it **doesn't learn at all** in 30 epochs (10% = guessing among 10 digits). Inserting BatchNorm after every linear layer keeps each layer's inputs centred and scaled, so the sigmoids stay out of their flat regions, and the same network reaches about 89% after 5 epochs and 97.5% after 30.

4 · Regularization: make a network overfit, then fight it

To see overfitting clearly we make it easy to happen: only **200** training images, and a **big** network (64 → 512 → 512 → 10, about 300,000 weights). We keep a **validation** set to make decisions and a separate **test** set we look at only at the end.

```
Xtr, Xtmp, ytr, ytmp = train_test_split(Xd, yd, train_size=200, random_state=0, stratify=yd)
Xva, Xte, yva, yte = train_test_split(Xtmp, ytmp, test_size=0.5, random_state=0, stratify=ytmp)
Xtr, Xva, Xte = T(Xtr), T(Xva), T(Xte); ytr, yva, yte = torch.tensor(ytr), torch.tensor(yva),
torch.tensor(yte)
print("train", len(Xtr), " validation", len(Xva), " test", len(Xte))

def big_net(p_drop=0.0):
    return nn.Sequential(nn.Linear(64, 512), nn.ReLU(), nn.Dropout(p_drop),
                         nn.Linear(512, 512), nn.ReLU(), nn.Dropout(p_drop), nn.Linear(512, 10))

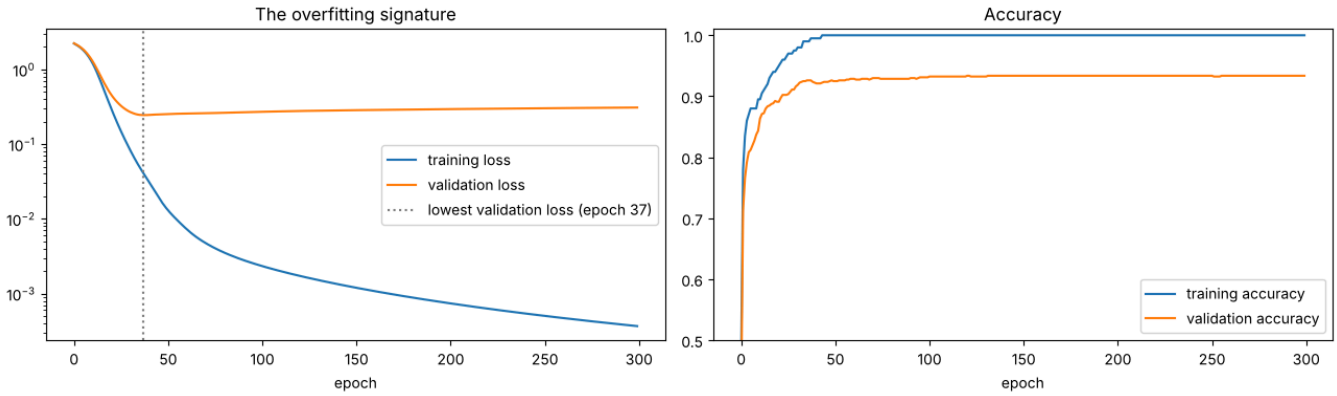
def evaluate(net, X, y):
    net.eval()
    with torch.no_grad():
        out = net(X); loss = nn.functional.cross_entropy(out, y).item(); acc = (out.argmax(1) ==
y).float().mean().item()
    net.train(); return loss, acc

def shift_1px(X):
    """data augmentation: move each 8x8 digit by up to 1 pixel (label unchanged)"""
    imgs = X.view(-1, 8, 8).clone()
    dx, dy = np.random.randint(-1, 2, size=(2, len(imgs)))
    for i in range(len(imgs)):
        imgs[i] = torch.roll(imgs[i], shifts=(int(dy[i]), int(dx[i])), dims=(0, 1))
    return imgs.view(-1, 64)
```

```python
def train_reg(p_drop=0.0, l1=0.0, l2=0.0, weight_decay=0.0, optimizer="adam", augment=False,
epochs=300, seed=0):
    torch.manual_seed(seed); np.random.seed(seed)
    net = big_net(p_drop)
    Opt = torch.optim.AdamW if optimizer == "adamw" else torch.optim.Adam
    opt = Opt(net.parameters(), lr=1e-3, weight_decay=weight_decay)
    hist = []
    for epoch in range(epochs):
        xb = shift_1px(Xtr) if augment else Xtr
        loss = nn.functional.cross_entropy(net(xb), ytr)
        weights = [m.weight for m in net if isinstance(m, nn.Linear)]
        if l1: loss = loss + l1 * sum(w.abs().sum() for w in weights)          # L1 penalty:
lambda * sum|w|
        if l2: loss = loss + l2 / 2 * sum((w ** 2).sum() for w in weights)     # L2 penalty:
lambda/2 * sum w^2
        opt.zero_grad(); loss.backward(); opt.step()
        hist.append(evaluate(net, Xtr, ytr) + evaluate(net, Xva, yva))         # (train loss,
train acc, val loss, val acc)
    return net, np.array(hist)

net0, h0 = train_reg()
best_epoch = int(h0[:, 2].argmin())
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].plot(h0[:, 0], label="training loss"); axes[0].plot(h0[:, 2], label="validation loss")
axes[0].axvline(best_epoch, color="grey", ls=":", label=f"lowest validation loss (epoch
{best_epoch})")
axes[0].set_yscale("log"); axes[0].set_xlabel("epoch"); axes[0].legend(); axes[0].set_title("The
overfitting signature")
axes[1].plot(h0[:, 1], label="training accuracy"); axes[1].plot(h0[:, 3], label="validation
accuracy")
axes[1].set_xlabel("epoch"); axes[1].legend(); axes[1].set_title("Accuracy");
axes[1].set_ylim(0.5, 1.01)
plt.tight_layout(); plt.show()
print(f"training accuracy {h0[-1, 1]:.1%}, validation accuracy {h0[-1, 3]:.1%}")
print(f"validation loss: lowest {h0[best_epoch, 2]:.3f} at epoch {best_epoch}, at the end {h0[-1,
2]:.3f}")
```

```
train 200  validation 798  test 799
```




```
training accuracy 100.0%, validation accuracy 93.4%
validation loss: lowest 0.245 at epoch 37, at the end 0.309
```

This is the **overfitting signature** from the lecture:

- **training loss keeps falling** towards 0, and training accuracy hits 100% (memorised);
- **validation loss bottoms out and then rises.** The network grows ever more confident on the training images in ways that don't carry over, so on unseen images its wrong answers become more confidently wrong.

Validation **accuracy** often plateaus rather than collapsing, which is why the lecture says to **compare losses, not just training accuracy**.

Every regulariser from the lecture, on the same problem

Each setting is trained 3 times (3 random seeds) and averaged, because single runs on 200 images are noisy.

```
settings = {
    "no regularization":          dict(),
    "L2 penalty (lambda 1e-3)":   dict(l2=1e-3),
    "L1 penalty (lambda 1e-4)":   dict(l1=1e-4),
    "dropout p = 0.5":            dict(p_drop=0.5),
    "augmentation (1-px shift)":  dict(augment=True),
    "Adam + weight_decay 1e-2":   dict(weight_decay=1e-2),
    "AdamW weight_decay 1e-2":    dict(weight_decay=1e-2, optimizer="adamw"),
}
rows = []
for name, kw in settings.items():
    final_val, final_test, es_test, zeros = [], [], [], []
    for seed in range(3):
        net, h = train_reg(seed=seed, **kw)
        final_val.append(h[-1, 3]); final_test.append(evaluate(net, Xte, yte)[1])
        weights = torch.cat([m.weight.detach().flatten() for m in net if isinstance(m,
nn.Linear)])
        zeros.append((weights.abs() < 1e-3).float().mean().item())
    rows.append((name, np.mean(final_val), np.mean(final_test), np.std(final_test),
np.mean(zeros)))
print(f"{'setting':>27} {'val acc':>8} {'test acc':>14} {'weights ~0':>11}")
for name, va, te, sd, z in rows:
    print(f"{name:>27} {va:>8.1%} {te:>8.1%} ± {sd:.1%} {z:>10.0%}")
```

```
                    setting  val acc       test acc  weights ~0
         no regularization    93.4%    93.5% ± 0.3%         2%
   L2 penalty (lambda 1e-3)    93.9%    94.1% ± 0.2%        24%
   L1 penalty (lambda 1e-4)    93.7%    93.7% ± 0.2%        88%
            dropout p = 0.5    94.1%    93.9% ± 0.5%         2%
  augmentation (1-px shift)    93.1%    91.8% ± 0.6%         2%
   Adam + weight_decay 1e-2    93.6%    93.7% ± 0.2%        24%
    AdamW weight_decay 1e-2    93.5%    93.5% ± 0.3%         2%
```

Early stopping

Keep training, but remember the weights from the epoch with the **lowest validation loss**, and stop once it hasn't improved for `patience` epochs.

```
import copy
def train_early_stopping(patience=20, max_epochs=300, seed=0):
    torch.manual_seed(seed); np.random.seed(seed)
    net = big_net(); opt = torch.optim.Adam(net.parameters(), lr=1e-3)
    best_loss, best_state, best_epoch, waited = np.inf, None, 0, 0
    for epoch in range(max_epochs):
        loss = nn.functional.cross_entropy(net(Xtr), ytr); opt.zero_grad(); loss.backward();
opt.step()
        val_loss, _ = evaluate(net, Xva, yva)
        if val_loss < best_loss:
            best_loss, best_state, best_epoch, waited = val_loss,
copy.deepcopy(net.state_dict()), epoch, 0
        else:
            waited += 1
            if waited >= patience:
                break
    net.load_state_dict(best_state)
    return net, best_epoch, epoch

es_acc = []
for seed in range(3):
    net, best_ep, stopped = train_early_stopping(seed=seed)
```

```
    es_acc.append(evaluate(net, Xte, yte)[1])
    print(f"seed {seed}: best validation loss at epoch {best_ep}, stopped at epoch {stopped}
(instead of 300), test accuracy {es_acc[-1]:.1%}")
print(f"early stopping: test accuracy {np.mean(es_acc):.1%} ± {np.std(es_acc):.1%}")
```

```
seed 0: best validation loss at epoch 37, stopped at epoch 57 (instead of 300), test accuracy 93.
5%
seed 1: best validation loss at epoch 36, stopped at epoch 56 (instead of 300), test accuracy 92.
7%
seed 2: best validation loss at epoch 50, stopped at epoch 70 (instead of 300), test accuracy 93.
4%
early stopping: test accuracy 93.2% ± 0.3%
```

Honest reading of the regularization results

Before reading the tables: with 200 training images and 3 seeds, differences of about **±1 percentage point are within the noise**. Look for patterns, not for a winner.

- **L1 does exactly what the lecture says to the weights.** It drives most weights to (almost) zero, making the network **sparse**. Compare the "weights ~0" column with the others. Its effect on accuracy is modest.
- **L2 (penalty in the loss) and Adam's `weight_decay`** behave alike (both leave about 24% of the weights near zero), because with Adam, `weight_decay` is just an L2 term added to the gradient.
- **AdamW** applies **decoupled** decay directly to the weights: each step shrinks w by lr × weight_decay × w = 0.001 × 0.01 = 0.001% here, which over 300 steps is very weak. That's why its row looks like no regularization. Same number, different mechanism: the lecture's reason AdamW exists. In practice AdamW is used with larger values (e.g. 0.05–0.1) and longer training.
- **The best test results came from L2 (94.1%) and dropout (93.9%)**, both about half a point above no regularization (93.5%). That gain is real but small, about the size of the seed-to-seed noise.
- **Early stopping** gave a similar test accuracy (93.2% vs 93.5%), but stopped after about 60 epochs instead of 300 (5× cheaper) and kept the weights from the lowest validation loss.
- **Augmentation by 1-pixel shifts did not help here.** On an 8×8 digit a 1-pixel shift is a big change (12% of the width) and `torch.roll` wraps pixels around the edge. The lecture's warning applies: *"incorrect transformations can create mislabeled or unrealistic examples"*. On real-sized images, small shifts and crops are standard and usually help.
- **None of them is magic.** The model still reaches 100% training accuracy, and the gap to validation shrinks a little. The biggest single cure for overfitting is **more data**.

> **Rule from the lecture:** establish a clear **baseline** first, then add regularization one change at a time, judging on **validation** data. Look at the test set only once, at the end.

✅ Check yourself

1. Normalization or regularization: which one is mainly about optimisation, and which about generalisation? *(normalization: optimisation; regularization: generalisation)*
2. Why did gradient descent on raw features either diverge or stall? *(x2's large scale forces a tiny learning rate, at which x1 barely learns)*

3. A feature has one huge outlier. Min-max or standardisation, and why? *(standardisation, or robust scaling; min-max squashes everything else into a tiny range)*
4. You compute μ and σ on the whole dataset before splitting. What's wrong? *(data leakage: the test set influenced training, so the score is optimistic)*
5. BatchNorm vs LayerNorm: what does each average over? Which one behaves differently in train and eval mode? *(batch per feature vs features per sample; BatchNorm)*
6. What does L1 do to weights that L2 doesn't? *(drives many exactly or nearly to zero: sparsity)*
7. How do you spot overfitting from the curves? *(training loss keeps falling while validation loss stops improving or rises)*
8. Why does AdamW exist? *(with Adam, adding L2 to the loss is not the same as true weight decay; AdamW decouples the decay from the adaptive gradient step)*

06 · Loss functions: MSE, MAE, binary and categorical cross-entropy

Lecture: *Loss Functions in Neural Networks* (18-08-2026) · **Notes:** `../notes/06_loss_functions.md`

> A **loss function** tells a network **how wrong** its prediction is. Training adjusts the weights to make it smaller. Different problems need different losses, because "error" means different things for regression, binary and multi-class classification.

In this notebook you will:

1. Recompute every worked example of the lecture (MSE = 466.67, MAE = 20, BCE and CCE values).
2. **See** why MSE is sensitive to outliers and MAE isn't, by fitting a line to data with one outlier.
3. **See** why classification uses cross-entropy rather than MSE, by measuring the gradient on a confidently wrong prediction.
4. Show that categorical and **sparse** categorical cross-entropy are the same loss with different label formats, and learn PyTorch's version of each (plus the "logits vs probabilities" trap).

Runs in seconds.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.nn.functional as F
torch.manual_seed(0)
```

```
<torch._C.Generator at 0x7ff5a5bf8e50>
```

1 · Regression losses

MSE = (1/n) Σ $(y_i - \hat{y}_i)^2$ MAE = (1/n) Σ $|y_i - \hat{y}_i|$

The lecture's example: actual (100, 200, 300), predicted (90, 220, 270).

```
y = np.array([100, 200, 300]); y_hat = np.array([90, 220, 270])
err = y - y_hat
print("errors:", err, " squared:", err ** 2, " absolute:", np.abs(err))
print(f"MSE = ({' + '.join(map(str, err ** 2))}) / 3 = {np.mean(err ** 2):.2f}")
print(f"MAE = ({' + '.join(map(str, np.abs(err)))}) / 3 = {np.mean(np.abs(err)):.2f}")
print("PyTorch agrees:", F.mse_loss(torch.tensor(y_hat, dtype=torch.float), torch.tensor(y,
dtype=torch.float)).item(),
      F.l1_loss(torch.tensor(y_hat, dtype=torch.float), torch.tensor(y,
dtype=torch.float)).item())
```

```
errors: [ 10 -20  30]  squared: [100 400 900]  absolute: [10 20 30]
MSE = (100 + 400 + 900) / 3 = 466.67
MAE = (10 + 20 + 30) / 3 = 20.00
PyTorch agrees: 466.6666564941406 20.0
```

Why squaring matters: errors of 2 and 10

```
for e in [2, 10, 100]:
    print(f"error {e:>3}: MSE contribution {e ** 2:>6}   MAE contribution {e:>3}")
print("an error 5x bigger counts 25x more under MSE, but only 5x more under MAE")
errors = np.array([2, 3, 4, 2, 3, 100])
print(f"\nerrors {errors.tolist()}: MSE = {np.mean(errors ** 2):.1f} (the 100 is {100 ** 2 /
np.sum(errors ** 2):.1%} of it), MAE = {np.mean(errors):.1f} (the 100 is {100 / errors.sum():.0%}
of it)")
```

```
error   2: MSE contribution      4   MAE contribution   2
error  10: MSE contribution    100   MAE contribution  10
error 100: MSE contribution  10000   MAE contribution 100
an error 5x bigger counts 25x more under MSE, but only 5x more under MAE

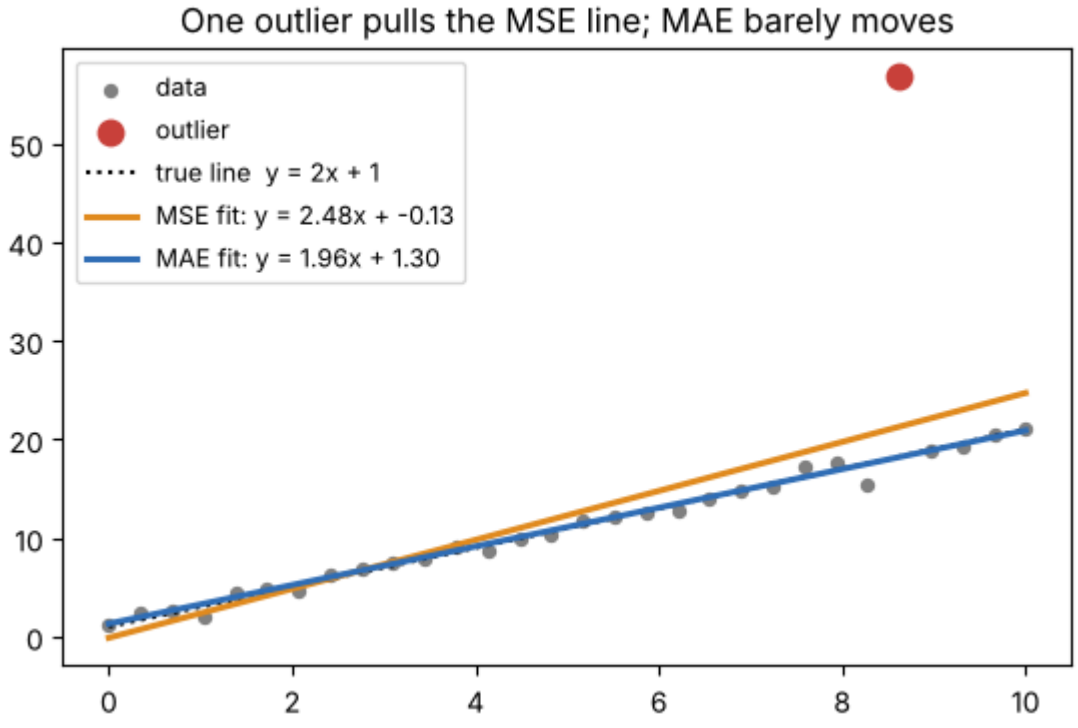
errors [2, 3, 4, 2, 3, 100]: MSE = 1673.7 (the 100 is 99.6% of it), MAE = 19.0 (the 100 is 88% of
it)
```

Seeing it: fit a line with MSE and with MAE when one point is an outlier

```
rng = np.random.default_rng(1)
x = np.linspace(0, 10, 30); y_line = 2 * x + 1 + rng.normal(0, 0.8, 30)
y_line[25] += 40                                    # one bad measurement
X_t, Y_t = torch.tensor(x, dtype=torch.float32), torch.tensor(y_line, dtype=torch.float32)

def fit(loss_fn, steps=3000, lr=0.01):
    w = torch.zeros(1, requires_grad=True); b = torch.zeros(1, requires_grad=True)
    opt = torch.optim.Adam([w, b], lr=0.1)
    for _ in range(steps):
        loss = loss_fn(w * X_t + b, Y_t); opt.zero_grad(); loss.backward(); opt.step()
    return w.item(), b.item()

fits = {"MSE": fit(F.mse_loss), "MAE": fit(F.l1_loss)}
plt.figure(figsize=(6.5, 4))
plt.scatter(x, y_line, s=18, c="grey", label="data"); plt.scatter(x[25], y_line[25], s=80,
c="#c8403a", label="outlier")
plt.plot(x, 2 * x + 1, "k:", label="true line  y = 2x + 1")
for (name, (w, b)), col in zip(fits.items(), ["#e08a1e", "#2f6db5"]):
    plt.plot(x, w * x + b, color=col, lw=2.2, label=f"{name} fit: y = {w:.2f}x + {b:.2f}")
plt.legend(fontsize=9); plt.title("One outlier pulls the MSE line; MAE barely moves"); plt.show()
```



The single outlier drags the **MSE** line towards itself (its squared error is huge, so the optimiser works hard to reduce it). The **MAE** line stays close to the true line. So:

- **MSE** when large errors are especially bad and the data has no severe outliers;
- **MAE** when outliers are present and you want robustness.

MAE also reads naturally: "on average, off by ₹X".

MAE's catch: |x| has a **corner at 0** (not differentiable there), and its gradient is ±1 everywhere, so it doesn't shrink as you approach the answer. Optimisers cope, but behave differently than on smooth MSE. (Huber loss, MSE near 0 and MAE far away, is a popular compromise; it's not in the lecture.)

2 · Binary cross-entropy (BCE)

BCE = −[y·log(p) + (1 − y)·log(1 − p)], where p is the predicted probability of class 1 (from a **sigmoid**).

```
bce = lambda y, p: -(y * np.log(p) + (1 - y) * np.log(1 - p))
print("actual y = 1:")
for p in [0.99, 0.90, 0.60, 0.10, 0.01]:
    print(f"   predicted p = {p:<4}  ->  loss = {bce(1, p):.3f}")
```

```
actual y = 1:
   predicted p = 0.99  ->  loss = 0.010
   predicted p = 0.9   ->  loss = 0.105
   predicted p = 0.6   ->  loss = 0.511
   predicted p = 0.1   ->  loss = 2.303
   predicted p = 0.01  ->  loss = 4.605
```

The lecture's table in numbers: 0.99 → **tiny** (0.010), 0.10 → **large** (2.303), 0.01 → **huge** (4.605). **BCE heavily punishes confident wrong predictions.**

Why not MSE for classification? Compare the gradients

Suppose y = 1, but the network is confidently wrong: its logit is z = −5, so p = sigmoid(−5) ≈ 0.007. How hard does each loss push the logit to fix it?

```
for z0 in [-5.0, -2.0, 0.0, 2.0]:
    z = torch.tensor(z0, requires_grad=True)
    p = torch.sigmoid(z)
    (g_mse,) = torch.autograd.grad((p - 1.0) ** 2, z)
    z2 = torch.tensor(z0, requires_grad=True)
    (g_bce,) = torch.autograd.grad(F.binary_cross_entropy(torch.sigmoid(z2), torch.tensor(1.0)),
z2)
    print(f"logit {z0:+.0f} (p = {p.item():.3f}):  gradient with MSE {g_mse.item():+.4f}   with
BCE {g_bce.item():+.4f}")
```

```
logit -5 (p = 0.007):  gradient with MSE -0.0132   with BCE -0.9933
logit -2 (p = 0.119):  gradient with MSE -0.1850   with BCE -0.8808
logit +0 (p = 0.500):  gradient with MSE -0.2500   with BCE -0.5000
logit +2 (p = 0.881):  gradient with MSE -0.0250   with BCE -0.1192
```

When the model is confidently wrong (logit −5), MSE's gradient almost vanishes (about −0.013), because the sigmoid is saturated, while BCE's stays strong (about −0.99). With BCE + sigmoid, the gradient on the logit is simply **p − y**: the more wrong, the harder the push. This is the

mathematical reason the lecture gives "classification is about probabilities and likelihood": BCE is the negative log-likelihood of the correct class.

3 · Categorical cross-entropy (CCE) with softmax

CCE = $-\Sigma_k\, y_k \cdot \log(p_k)$. With a one-hot label, only the correct class's term survives: **CCE = −log(p_correct)**.

```
p_digits = np.array([0.01, 0.02, 0.01, 0.03, 0.04, 0.02, 0.01, 0.82, 0.02, 0.02])   # lecture's softmax output
print("sums to", p_digits.sum().round(3), "-> predicted class:", p_digits.argmax())
one_hot_7 = np.eye(10)[7]
print("one-hot label for 7:", one_hot_7.astype(int))
print(f"CCE when p(7) = 0.82: {-np.sum(one_hot_7 * np.log(p_digits)):.3f}")
print(f"CCE when p(7) = 0.01: {-np.log(0.01):.3f}   <- confidently wrong, heavily punished")
```

```
sums to 1.0 -> predicted class: 7
one-hot label for 7: [0 0 0 0 0 0 0 1 0 0]
CCE when p(7) = 0.82: 0.198
CCE when p(7) = 0.01: 4.605   <- confidently wrong, heavily punished
```

Categorical vs sparse categorical: same loss, different label format

- **Categorical CE** expects **one-hot** labels: 7 → [0,0,0,0,0,0,0,1,0,0].
- **Sparse categorical CE** expects the **integer** 7.

"Sparse" refers to how the **label** is written, **not** to the predictions. In PyTorch, `F.cross_entropy` / `nn.CrossEntropyLoss` accepts either.

```
logits = torch.randn(4, 10)                       # 4 images, 10 classes (raw scores, NOT softmax)
labels_int = torch.tensor([7, 2, 0, 9])           # "sparse" integer labels
labels_onehot = F.one_hot(labels_int, 10).float() # "categorical" one-hot labels
print("integer labels :", F.cross_entropy(logits, labels_int).item())
print("one-hot labels :", F.cross_entropy(logits, labels_onehot).item())
manual = -(labels_onehot * F.log_softmax(logits, dim=1)).sum(1).mean()
print("by hand (-sum y*log softmax):", manual.item())
```

```
integer labels : 2.5375328063964844
one-hot labels : 2.5375328063964844
by hand (-sum y*log softmax): 2.5375328063964844
```

All three numbers are identical: **same objective, different target representation**.

Keras name	PyTorch	labels	output layer
`binary_crossentropy`	`nn.BCEWithLogitsLoss` (on logits) or `nn.BCELoss` (on sigmoid probabilities)	0/1	sigmoid
`categorical_crossentropy`	`nn.CrossEntropyLoss` with one-hot/probability targets	one-hot	softmax
`sparse_categorical_crossentropy`	`nn.CrossEntropyLoss` with integer targets	integers	softmax
`mse` / `mae`	`nn.MSELoss` / `nn.L1Loss`	numbers	linear

⚠️ The most common PyTorch bug: softmax twice

`nn.CrossEntropyLoss` **already applies log-softmax** to the raw logits. If your model ends in `nn.Softmax` and you also use `CrossEntropyLoss`, you apply softmax twice:

```
logits = torch.tensor([[2.0, 0.0, -2.0]]); target = torch.tensor([0])
print("correct (logits in):          ", round(F.cross_entropy(logits, target).item(), 4))
print("bug (softmax, then the loss):  ", round(F.cross_entropy(F.softmax(logits, 1),
target).item(), 4), "<- squashed, wrong, weak gradients")
```

```
correct (logits in):           0.1429
bug (softmax, then the loss):   0.6417 <- squashed, wrong, weak gradients
```

The same rule applies to binary problems: prefer `BCEWithLogitsLoss` on raw scores (numerically stable) over `Sigmoid` + `BCELoss`.

4 · Choosing a loss (the lecture's decision table, verified above)

Problem	Loss	Output layer	Target format
regression	**MSE**	linear	continuous
regression with outliers	**MAE**	linear	continuous
binary classification	**binary CE**	sigmoid	0 / 1
multi-class	**categorical CE**	softmax	one-hot
multi-class	**sparse categorical CE**	softmax	integer
multi-label (several can be true)	binary CE on each output	one sigmoid per label	0/1 vector

What each loss is "asking":

- **MSE:** "how far is my number from the real number?"
- **MAE:** "how much numerical error, without over-weighting big errors?"
- **BCE:** "how well does my probability match the yes/no answer?"
- **CCE:** "how much probability did I give the correct class?"

✅ Check yourself

1. Actual (10, 20), predicted (12, 14). MSE? MAE? *(errors −2, 6 → MSE (4 + 36)/2 = 20, MAE (2 + 6)/2 = 4)*
2. Why is MSE outlier-sensitive? *(squared errors: one big error dominates the sum)*
3. y = 1, p = 0.5: BCE? *(−ln 0.5 = 0.693)*
4. Why does sigmoid + MSE learn slowly when confidently wrong? *(the sigmoid is saturated, so MSE's gradient ≈ 0; BCE's gradient on the logit is p − y)*
5. Sparse vs categorical CE: what's different? *(only the label format: integer vs one-hot)*
6. Your model ends with `nn.Softmax` and you use `nn.CrossEntropyLoss`. What's wrong? *(softmax applied twice: remove the Softmax layer and feed logits)*

07 · How an MLP works: forward, loss, backpropagation, update, from scratch

Lecture: *Working of MLP / ANN* (20-08-2026) · **Notes:** `../notes/07_working_of_mlp.md`

The lecture's eight steps, all in code:

input → weighted sum → activation → … → prediction → loss → backpropagation (gradients) → update → repeat

In this notebook you will:

1. Recompute the lecture's small examples (neuron z = 1.3, ReLU on [−2, 3], loss 0.357, update 2 → 1.95).
2. Build a 2-layer MLP in **plain NumPy** using **exactly the lecture's backpropagation formulas** ($\delta^{(2)} = \hat{Y} - Y$, …).
3. **Prove those formulas are right**: compare them with numerical derivatives and with PyTorch's automatic gradients.
4. Train it on handwritten digits with mini-batches, counting **steps vs batches vs epochs**, with training, validation and test sets.
5. See **underfitting and overfitting**, and what the hidden neurons learned.

Runs in under a minute.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
```

1 · The lecture's small examples

One neuron: z = w·x + b, then a = f(z)

```
x = np.array([2, 3, 1]); w = np.array([0.5, 0.2, -0.4]); b = 0.1
z = w @ x + b
print(f"z = (2)(0.5) + (3)(0.2) + (1)(-0.4) + 0.1 = {z:.1f}   ->   ReLU(z) = {max(0, z):.1f}")
print("ReLU applied to Z = [-2, 3]:", np.maximum(0, np.array([-2, 3])))
```

```
z = (2)(0.5) + (3)(0.2) + (1)(-0.4) + 0.1 = 1.3   ->   ReLU(z) = 1.3
ReLU applied to Z = [-2, 3]: [0 3]
```

Loss for Y = [1, 0], Ŷ = [0.7, 0.3], and one gradient-descent update

```
Y, Y_hat = np.array([1, 0]), np.array([0.7, 0.3])
print(f"L = -[1*log(0.7) + 0*log(0.3)] = {-(Y * np.log(Y_hat)).sum():.3f}")
w_old, grad, lr = 2.0, 0.5, 0.1
print(f"w_new = w - lr * dL/dw = {w_old} - {lr} * {grad} = {w_old - lr * grad}")
print("we SUBTRACT because the gradient points towards increasing loss")
```

```
L = -[1*log(0.7) + 0*log(0.3)] = 0.357
w_new = w - lr * dL/dw = 2.0 - 0.1 * 0.5 = 1.95
we SUBTRACT because the gradient points towards increasing loss
```

2 · A 2-layer MLP in NumPy, with the lecture's equations

Forward	Backward (softmax + cross-entropy)
$Z^{(1)} = X\,W^{(1)} + b^{(1)}$	$\delta^{(2)} = \hat{Y} - Y$
$A^{(1)} = \mathrm{ReLU}(Z^{(1)})$	$\partial L/\partial W^{(2)} = A^{(1)\top}\delta^{(2)}$, $\partial L/\partial b^{(2)} = \delta^{(2)}$
$Z^{(2)} = A^{(1)}\,W^{(2)} + b^{(2)}$	$\delta^{(1)} = [\delta^{(2)}\,W^{(2)\top}] \odot \mathrm{ReLU}'(Z^{(1)})$
$\hat{Y} = \mathrm{softmax}(Z^{(2)})$	$\partial L/\partial W^{(1)} = X^{\top}\delta^{(1)}$, $\partial L/\partial b^{(1)} = \delta^{(1)}$
$L = -\Sigma\, y \log \hat{y}$	update: $W \leftarrow W - \eta\,\partial L/\partial W$

For a batch of m examples we average the loss, so the gradients get a factor 1/m and the bias gradients are **summed over the batch**.

```
def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)              # numerically stable (doesn't change the
result)
    e = np.exp(Z)
    return e / e.sum(axis=1, keepdims=True)

class MLP:
    def __init__(self, n_in, n_hidden, n_out, seed=0):
        r = np.random.default_rng(seed)
        self.W1 = r.normal(0, np.sqrt(2 / n_in), (n_in, n_hidden)); self.b1 = np.zeros(n_hidden)
# He init (notebook 08)
        self.W2 = r.normal(0, np.sqrt(2 / n_hidden), (n_hidden, n_out)); self.b2 =
np.zeros(n_out)

    def forward(self, X):
        self.X = X
        self.Z1 = X @ self.W1 + self.b1          # step 2: weighted sums (pre-activations)
        self.A1 = np.maximum(0, self.Z1)         # step 3: activation (ReLU)
        self.Z2 = self.A1 @ self.W2 + self.b2    # step 4: next layer
        self.Y_hat = softmax(self.Z2)            # step 5: output probabilities
        return self.Y_hat

    def loss(self, Y):                           # step 6: cross-entropy, averaged over the batch
        return -np.mean(np.sum(Y * np.log(self.Y_hat + 1e-12), axis=1))

    def backward(self, Y):                       # step 7: backpropagation (the lecture's
formulas)
        m = len(Y)
        d2 = (self.Y_hat - Y) / m                            # delta(2) = Y_hat - Y   (/m: average
over batch)
        self.dW2 = self.A1.T @ d2                            # dL/dW2 = A1^T delta(2)
        self.db2 = d2.sum(axis=0)                            # dL/db2 = delta(2)
        d1 = (d2 @ self.W2.T) * (self.Z1 > 0)                # delta(1) = [delta(2) W2^T] (*)
ReLU'(Z1)
        self.dW1 = self.X.T @ d1                             # dL/dW1 = X^T delta(1)
        self.db1 = d1.sum(axis=0)                            # dL/db1 = delta(1)

    def step(self, lr):                          # step 8: gradient-descent update
        for p in ["W1", "b1", "W2", "b2"]:
            setattr(self, p, getattr(self, p) - lr * getattr(self, "d" + p))

net = MLP(4, 5, 3)
Xb = rng.normal(size=(6, 4)); Yb = np.eye(3)[rng.integers(0, 3, 6)]
```

```
net.forward(Xb); print("loss on a random batch:", round(net.loss(Yb), 4)); net.backward(Yb)
print("gradient shapes:", {k: getattr(net, k).shape for k in ["dW1", "db1", "dW2", "db2"]})
```

```
loss on a random batch: 1.0162
gradient shapes: {'dW1': (4, 5), 'db1': (5,), 'dW2': (5, 3), 'db2': (3,)}
```

3 · Are the formulas right? Two independent checks

Check 1: numerical derivative. Nudge one weight by ±ε, measure how the loss changes, and divide: **(L(w + ε) − L(w − ε)) / 2ε**. This is the definition of "how much the loss changes when a parameter changes slightly".

```
def numerical_grad(net, X, Y, name, idx, eps=1e-5):
    P = getattr(net, name); old = P[idx]
    P[idx] = old + eps; net.forward(X); lp = net.loss(Y)
    P[idx] = old - eps; net.forward(X); lm = net.loss(Y)
    P[idx] = old
    return (lp - lm) / (2 * eps)

net.forward(Xb); net.backward(Yb)
worst = 0
for name, idx in [("W1", (0, 0)), ("W1", (3, 4)), ("b1", (2,)), ("W2", (1, 2)), ("b2", (0,))]:
    analytic = getattr(net, "d" + name)[idx]; numeric = numerical_grad(net, Xb, Yb, name, idx)
    worst = max(worst, abs(analytic - numeric))
    print(f"{name}{list(idx)}: backprop {analytic:+.8f}   numerical {numeric:+.8f}")
print(f"largest difference: {worst:.1e}")
```

```
W1[0, 0]: backprop +0.15445220   numerical +0.15445220
W1[3, 4]: backprop -0.02160575   numerical -0.02160575
b1[2]: backprop -0.01735445   numerical -0.01735445
W2[1, 2]: backprop -0.12472377   numerical -0.12472377
b2[0]: backprop +0.11099402   numerical +0.11099402
largest difference: 1.4e-11
```

Check 2: PyTorch's autograd computes the gradients of the same network automatically (the chain rule done by the computer).

```
import torch
T = lambda a: torch.tensor(a, dtype=torch.float64, requires_grad=True)
W1, b1, W2, b2 = T(net.W1), T(net.b1), T(net.W2), T(net.b2)
Xt, Yt = torch.tensor(Xb), torch.tensor(Yb)
loss = -(Yt * torch.log_softmax(torch.relu(Xt @ W1 + b1) @ W2 + b2, dim=1)).sum(1).mean()
loss.backward()
net.forward(Xb); net.backward(Yb)
for name, t in [("W1", W1), ("b1", b1), ("W2", W2), ("b2", b2)]:
    print(f"{name}: max |backprop - autograd| = {np.abs(getattr(net, 'd' + name) -
t.grad.numpy()).max():.1e}")
```

```
W1: max |backprop - autograd| = 2.8e-17
b1: max |backprop - autograd| = 3.1e-17
W2: max |backprop - autograd| = 2.8e-17
b2: max |backprop - autograd| = 2.8e-17
```

Both checks agree with the hand-derived formulas to many decimal places (differences at the level of floating-point rounding). The lecture's backpropagation equations are exactly what PyTorch's `loss.backward()` computes.

4 · Train it on handwritten digits

The scikit-learn digits: 8×8 images, so **64 inputs → 64 hidden ReLU neurons → 10 softmax outputs**. We split the data into training (learn weights), validation (watch for overfitting, choose

settings) and test (final score, used once).

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
d = load_digits()
X_all = d.data / 16.0                                  # normalise pixels to 0..1 (lecture: why
normalisation matters)
Y_all = np.eye(10)[d.target]                           # one-hot targets for categorical cross-
entropy
X_tr, X_tmp, Y_tr, Y_tmp = train_test_split(X_all, Y_all, test_size=0.4, random_state=0,
stratify=d.target)
X_va, X_te, Y_va, Y_te = train_test_split(X_tmp, Y_tmp, test_size=0.5, random_state=0,
stratify=Y_tmp.argmax(1))
print("train", X_tr.shape, " validation", X_va.shape, " test", X_te.shape)

def accuracy(net, X, Y):
    return (net.forward(X).argmax(1) == Y.argmax(1)).mean()

def train(net, epochs=40, batch_size=128, lr=0.5, seed=0, verbose=True):
    r = np.random.default_rng(seed); steps = 0; hist = []
    for epoch in range(epochs):
        order = r.permutation(len(X_tr))
        for start in range(0, len(order), batch_size):           # one batch = one training step
            idx = order[start:start + batch_size]
            net.forward(X_tr[idx]); net.backward(Y_tr[idx]); net.step(lr)
            steps += 1
        net.forward(X_tr); tr_loss = net.loss(Y_tr)
        net.forward(X_va); va_loss = net.loss(Y_va)
        hist.append((tr_loss, va_loss, accuracy(net, X_tr, Y_tr), accuracy(net, X_va, Y_va)))
        if verbose and (epoch < 3 or (epoch + 1) % 10 == 0):
            print(f"epoch {epoch + 1:>2}: steps so far {steps:>3} | train loss {tr_loss:.3f} acc
{hist[-1][2]:.1%} | val loss {va_loss:.3f} acc {hist[-1][3]:.1%}")
    return np.array(hist), steps

net = MLP(64, 64, 10)
hist, steps = train(net)
batches_per_epoch = int(np.ceil(len(X_tr) / 128))
print(f"\n{len(X_tr)} training images / batch size 128 = {batches_per_epoch} batches per epoch ->
40 epochs = {steps} weight updates")
print(f"TEST accuracy (looked at once, at the end): {accuracy(net, X_te, Y_te):.1%}")
```

```
train (1078, 64)  validation (359, 64)  test (360, 64)
epoch  1: steps so far   9 | train loss 1.385 acc 61.9% | val loss 1.413 acc 58.8%
epoch  2: steps so far  18 | train loss 0.823 acc 75.3% | val loss 0.860 acc 75.8%
epoch  3: steps so far  27 | train loss 0.518 acc 85.0% | val loss 0.566 acc 83.8%
epoch 10: steps so far  90 | train loss 0.178 acc 96.1% | val loss 0.205 acc 95.5%
epoch 20: steps so far 180 | train loss 0.096 acc 97.7% | val loss 0.153 acc 95.0%
epoch 30: steps so far 270 | train loss 0.069 acc 98.4% | val loss 0.125 acc 96.9%
epoch 40: steps so far 360 | train loss 0.053 acc 98.8% | val loss 0.115 acc 96.7%

1078 training images / batch size 128 = 9 batches per epoch -> 40 epochs = 360 weight updates
TEST accuracy (looked at once, at the end): 96.4%
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))
axes[0].plot(hist[:, 0], label="training loss"); axes[0].plot(hist[:, 1], label="validation
loss"); axes[0].set_xlabel("epoch"); axes[0].legend()
axes[1].plot(hist[:, 2], label="training accuracy"); axes[1].plot(hist[:, 3], label="validation
accuracy"); axes[1].set_xlabel("epoch"); axes[1].legend()
axes[0].set_title("Loss goes down: the network is learning"); axes[1].set_title("Accuracy")
plt.tight_layout(); plt.show()
```



Steps vs batches vs epochs (the lecture's arithmetic)

```
for n, bs, ep in [(50_000, 128, 15), (50_000, 100, 10)]:
    per_epoch = int(np.ceil(n / bs))
    print(f"{n:,} examples, batch {bs}: {n / bs:.1f} -> {per_epoch} batches per epoch; {ep}
epochs = {per_epoch * ep:,} weight updates")
```

```
50,000 examples, batch 128: 390.6 -> 391 batches per epoch; 15 epochs = 5,865 weight updates
50,000 examples, batch 100: 500.0 -> 500 batches per epoch; 10 epochs = 5,000 weight updates
```

Weights are updated **once per batch**, not once per epoch. With 50,000 examples, batch size 100 and 10 epochs, that's **5,000** updates, exactly the lecture's number. (50,000/128 = 390.6, so the last batch of each epoch is smaller: 391 batches.)

5 · Underfitting vs overfitting

```
results = []
for name, hidden, n_train, epochs in [("tiny network (2 hidden neurons)", 2, None, 40),
                                      ("good fit (64 hidden)", 64, None, 40),
                                      ("big network, only 60 training images", 256, 60, 400)]:
    global X_tr, Y_tr
    X_tr_full, Y_tr_full = X_tr, Y_tr
    if n_train:
        X_tr, Y_tr = X_tr[:n_train], Y_tr[:n_train]
    h, _ = train(MLP(64, hidden, 10), epochs=epochs, batch_size=32, lr=0.2, verbose=False)
    results.append((name, h[-1, 2], h[-1, 3]))
    X_tr, Y_tr = X_tr_full, Y_tr_full
for name, tr, va in results:
    gap = tr - va
    verdict = "UNDERFITTING (poor everywhere)" if tr < 0.85 else ("OVERFITTING (big train-
validation gap)" if gap > 0.1 else "good generalisation")
    print(f"{name:<38} train {tr:.1%}  validation {va:.1%}  -> {verdict}")
```

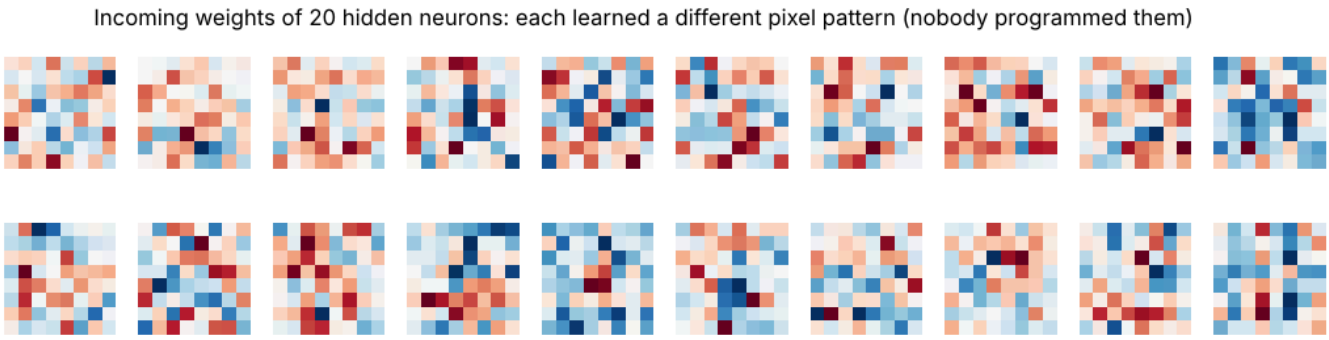
```
tiny network (2 hidden neurons)        train 66.3%  validation 65.5%  -> UNDERFITTING (poor every
where)
good fit (64 hidden)                   train 99.8%  validation 98.1%  -> good generalisation
big network, only 60 training images   train 100.0%  validation 86.4%  -> OVERFITTING (big train-
validation gap)
```

- **Underfitting:** the model is too small, so it's poor on training **and** validation data.
- **Overfitting:** it memorised its few training images (100% on them) but does much worse on new ones.
- **Fixes:** more data, a smaller model, dropout, L1/L2, early stopping (notebook 05).

6 · What did the hidden neurons learn?

Each hidden neuron has 64 incoming weights, one per pixel, which we can draw as an 8×8 picture. Red pixels push the neuron up, blue push it down.

```
fig, axes = plt.subplots(2, 10, figsize=(13, 3))
for ax, i in zip(axes.ravel(), range(20)):
    ax.imshow(net.W1[:, i].reshape(8, 8), cmap="RdBu_r"); ax.axis("off")
fig.suptitle("Incoming weights of 20 hidden neurons: each learned a different pixel pattern
(nobody programmed them)")
plt.show()
```



Each neuron ended up with a **different** pattern of positive and negative pixel weights. At 8×8 pixels and only 40 epochs they look noisy, not like neat pen strokes, yet together they are enough for about 96% test accuracy. The network doesn't learn rules like "if the top is closed, it's a 9". It learns **numbers** (weights) that turn neurons into useful detectors, because that lowers the loss. **Training is an optimisation problem: find the weights and biases that minimise the loss.**

Backpropagation vs the optimizer (don't mix them up)

- **Backpropagation** computes the **gradients** (section 3).
- The **optimizer** decides how to **use** them: plain gradient descent (our `step`), or **Adam**, which keeps running averages of the gradients and their squares to adapt each weight's step size.

Forward propagation	Backward propagation
input → output	loss → earlier layers
computes predictions	computes gradients
uses weights, biases, activations	uses the loss gradient and activation **derivatives**
ends with the loss	starts from the loss
"what did we predict?"	"how should we change?"

✅ Check yourself

1. Compute a neuron: x = (1, −1, 2), w = (0.3, 0.5, 0.1), b = −0.2, ReLU. *($z = 0.3 - 0.5 + 0.2 - 0.2 = -0.2 \rightarrow$ output 0)*
2. In softmax + cross-entropy, what is δ at the output? *($\hat{Y} - Y$)*
3. Why does the hidden-layer $\delta^{(1)}$ multiply by ReLU′($Z^{(1)}$)? *(chain rule through the activation; neurons with negative input pass no gradient)*
4. 60,000 examples, batch 200, 5 epochs: how many weight updates? *(300 per epoch × 5 = 1,500)*

5. Which data set is used to choose hyper-parameters, and which only once at the end? *(validation; test)*
6. Backpropagation vs optimizer? *(backprop computes gradients; the optimizer turns them into weight updates)*

08 · Weight initialization: zeros, random, Xavier and He

Lecture: *Weight Initialization in Neural Networks* (21-08-2026) · **Notes:** `../notes/08_weight_initialization.md`

> **Weight initialization** means choosing the weights' values **before training starts**. It is the optimiser's starting point. A bad start can make a network learn nothing at all.

In this notebook you will:

1. Recompute the lecture's Xavier and He numbers (Var 0.0133, σ ≈ 0.115; Var 0.02, σ ≈ 0.141) and check that the "uniform" versions have the same variance.
2. **See the symmetry problem:** start every weight equal, and the hidden neurons stay identical clones forever.
3. Push a signal through a 10-layer network and **watch it vanish or explode** with weights that are too small or too large, and stay stable with Xavier (tanh) and He (ReLU).
4. Train the same 8-hidden-layer network with four initializations and compare.
5. Learn what PyTorch does by default, and how to choose an initializer yourself.

Runs in under a minute on a CPU.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.nn.functional as F
from sklearn.datasets import load_digits
torch.manual_seed(0)
```

```
<torch._C.Generator at 0x7fc9ea118e30>
```

1 · The lecture's formulas, in numbers

	Normal: W ~ N(0, σ²)	**Uniform: W ~ U(−a, a)**
Xavier / Glorot (sigmoid, tanh)	σ² = 2 / (n_in + n_out)	a = √(6 / (n_in + n_out))
He / Kaiming (ReLU family)	σ² = 2 / n_in	a = √(6 / n_in)

Why the 6 in the uniform version? A uniform distribution on (−a, a) has variance **a²/3**, so a² = 3σ² gives the same variance: 3 × 2 = 6.

```
n_in, n_out = 100, 50
var_x = 2 / (n_in + n_out); var_h = 2 / n_in
print(f"Xavier normal: Var = 2/({n_in}+{n_out}) = {var_x:.4f}, sigma = {np.sqrt(var_x):.3f}")
print(f"He normal    : Var = 2/{n_in} = {var_h:.4f}, sigma = {np.sqrt(var_h):.3f}")
a_x = np.sqrt(6 / (n_in + n_out)); a_h = np.sqrt(6 / n_in)
print(f"Xavier uniform limit a = {a_x:.3f}  -> variance a^2/3 = {a_x ** 2 / 3:.4f} (same as
normal)")
print(f"He uniform limit     a = {a_h:.3f}  -> variance a^2/3 = {a_h ** 2 / 3:.4f} (same as
normal)")
```

```
# and PyTorch's built-in initializers produce exactly these scales
W = torch.empty(n_out, n_in)          # PyTorch stores Linear weights as (out, in)
for name, fn in [("xavier_normal_", nn.init.xavier_normal_), ("xavier_uniform_",
nn.init.xavier_uniform_),
                 ("kaiming_normal_", lambda w: nn.init.kaiming_normal_(w, nonlinearity="relu")),
                 ("kaiming_uniform_", lambda w: nn.init.kaiming_uniform_(w,
nonlinearity="relu"))]:
    fn(W); print(f"  nn.init.{name:<17} measured variance {W.var().item():.4f}   max |w|
{W.abs().max().item():.3f}")
```

```
Xavier normal: Var = 2/(100+50) = 0.0133, sigma = 0.115
He normal    : Var = 2/100 = 0.0200, sigma = 0.141
Xavier uniform limit a = 0.200  -> variance a^2/3 = 0.0133 (same as normal)
He uniform limit     a = 0.245  -> variance a^2/3 = 0.0200 (same as normal)
  nn.init.xavier_normal_    measured variance 0.0131   max |w| 0.474
  nn.init.xavier_uniform_   measured variance 0.0133   max |w| 0.200
  nn.init.kaiming_normal_   measured variance 0.0206   max |w| 0.540
  nn.init.kaiming_uniform_  measured variance 0.0197   max |w| 0.245
```

The measured variances match the formulas (to sampling noise), and the uniform versions never exceed their limits a.

2 · Why not all zeros? The symmetry problem

Take a 2-layer network (64 → 8 hidden → 10) on the digits data. Start **every weight with the same value** and train it. Every hidden neuron receives the same inputs with the same weights, so it computes the same output and gets the same gradient, every single step.

```
d = load_digits()
X = torch.tensor(d.data / 16, dtype=torch.float32); y = torch.tensor(d.target)

def train_small(init_value=None, steps=300):
    torch.manual_seed(0)
    net = nn.Sequential(nn.Linear(64, 8), nn.Tanh(), nn.Linear(8, 10))
    if init_value is not None:
        for p in net.parameters(): nn.init.constant_(p, init_value)
    opt = torch.optim.SGD(net.parameters(), lr=0.5)
    for _ in range(steps):
        loss = F.cross_entropy(net(X), y); opt.zero_grad(); loss.backward(); opt.step()
    acc = (net(X).argmax(1) == y).float().mean().item()
    return net, loss.item(), acc

for label, v in [("all zeros", 0.0), ("all 0.01 (equal, not zero)", 0.01), ("random (PyTorch
default)", None)]:
    net, L, acc = train_small(v)
    W1 = net[0].weight.detach()
    distinct = torch.unique(W1.round(decimals=6), dim=0).shape[0]
    print(f"{label:<28} loss {L:.3f}  train acc {acc:.1%}  distinct hidden neurons: {distinct} of
8")
```

```
all zeros                    loss 2.302  train acc 10.2%  distinct hidden neurons: 1 of 8
all 0.01 (equal, not zero)   loss 1.587  train acc 26.7%  distinct hidden neurons: 1 of 8
random (PyTorch default)     loss 0.129  train acc 97.8%  distinct hidden neurons: 8 of 8
```

With **equal** starting weights (zeros or 0.01), after 300 training steps the 8 hidden neurons are still **1 distinct neuron**: eight copies of the same feature, so the layer is effectively one neuron wide. Training accuracy: 10.2% (zeros) and 26.7% (all 0.01), against **97.8%** for random weights. (With all zeros it's even worse: tanh(0) = 0, so the output layer sees nothing, and the gradient reaching the first layer is exactly zero, so W1 never moves at all.)

Random initialization **breaks the symmetry**: the 8 neurons start different, receive different gradients, and learn 8 different features.

> **Biases can start at zero.** Once the weights are random, the neurons already differ, so zero biases cause no symmetry problem (the lecture says the same; PyTorch users often do `nn.init.zeros_(layer.bias)`).

3 · Too small, too large, just right: a signal through 10 layers

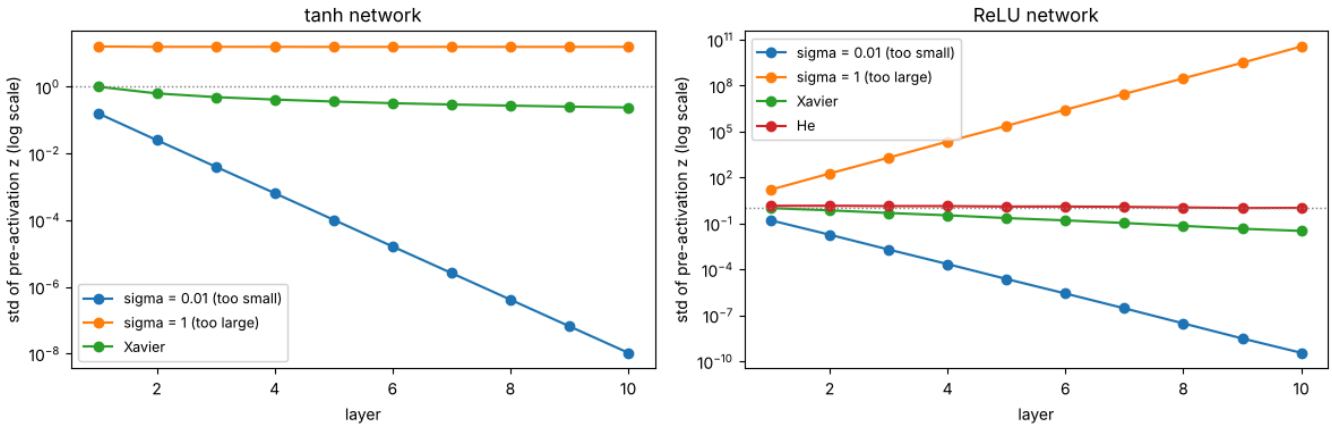
Feed 1,000 random inputs through 10 layers of width 256 **without any training** and measure the spread (standard deviation) of each layer's **pre-activation** z = x·W. If the spread shrinks towards 0, the signal **vanishes**; if it grows without bound, it **explodes**. Backpropagation multiplies by the same weights on the way back, so gradients behave the same way.

```
def forward_stds(sigma_fn, act, depth=10, width=256, n=1000, seed=0):
    g = torch.Generator().manual_seed(seed)
    x = torch.randn(n, width, generator=g)
    stds, sat = [], []
    for _ in range(depth):
        W = torch.randn(width, width, generator=g) * sigma_fn(width, width)
        z = x @ W
        stds.append(z.std().item()); x = act(z)
        sat.append((x.abs() > 0.99).float().mean().item())
    return stds, sat

configs = {
    "tanh, sigma = 0.01 (too small)": (lambda i, o: 0.01, torch.tanh),
    "tanh, sigma = 1 (too large)":    (lambda i, o: 1.0, torch.tanh),
    "tanh, Xavier":                    (lambda i, o: np.sqrt(2 / (i + o)), torch.tanh),
    "ReLU, sigma = 0.01 (too small)": (lambda i, o: 0.01, torch.relu),
    "ReLU, sigma = 1 (too large)":    (lambda i, o: 1.0, torch.relu),
    "ReLU, Xavier":                    (lambda i, o: np.sqrt(2 / (i + o)), torch.relu),
    "ReLU, He":                        (lambda i, o: np.sqrt(2 / i), torch.relu),
}
results = {k: forward_stds(*v) for k, v in configs.items()}
for k, (s, sat) in results.items():
    print(f"{k:<32} layer 1: {s[0]:>9.3g}   layer 5: {s[4]:>9.3g}   layer 10: {s[9]:>9.3g}")
```

```
tanh, sigma = 0.01 (too small)   layer 1:      0.16   layer 5:  0.000103   layer 10:  1.06e-08
tanh, sigma = 1 (too large)      layer 1:        16   layer 5:      15.6   layer 10:      15.6
tanh, Xavier                     layer 1:         1   layer 5:     0.358   layer 10:     0.238
ReLU, sigma = 0.01 (too small)   layer 1:      0.16   layer 5:  2.35e-05   layer 10:  3.58e-10
ReLU, sigma = 1 (too large)      layer 1:        16   layer 5:  2.35e+05   layer 10:  3.58e+10
ReLU, Xavier                     layer 1:         1   layer 5:     0.224   layer 10:    0.0326
ReLU, He                         layer 1:      1.42   layer 5:      1.27   layer 10:      1.04
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for k, (s, _) in results.items():
    ax = axes[0] if k.startswith("tanh") else axes[1]
    ax.semilogy(range(1, 11), s, "o-", label=k.split(", ")[1])
for ax, t in zip(axes, ["tanh network", "ReLU network"]):
    ax.set_title(t); ax.set_xlabel("layer"); ax.set_ylabel("std of pre-activation z (log scale)")
    ax.axhline(1, color="grey", ls=":", lw=1); ax.legend(fontsize=9)
plt.tight_layout(); plt.show()
print("tanh with sigma = 1: fraction of activations with |tanh| > 0.99 (saturated), layers 1-3:",
      [f"{v:.0%}" for v in results["tanh, sigma = 1 (too large)"][1][:3]])
```




```
tanh with sigma = 1: fraction of activations with |tanh| > 0.99 (saturated), layers 1-3: ['87%',
'87%', '87%']
```

What the measurements show:

- **Too small (σ = 0.01):** the spread shrinks by roughly 6× (tanh) and 9× (ReLU) per layer and is essentially zero by layer 10: **vanishing**.
- **Too large, ReLU (σ = 1):** the spread grows by about 11× per layer, reaching about 36 billion by layer 10: **exploding**.
- **Too large, tanh (σ = 1):** the pre-activations are huge, but tanh squashes them, and most activations sit at ±1, **saturated**, where tanh's slope is ≈ 0. The forward signal doesn't explode, but the gradients through those neurons **vanish**.
- **He with ReLU:** the spread stays between 1.0 and 1.4 through all 10 layers (He's factor 2 exactly compensates ReLU zeroing half the values).
- **Xavier with ReLU:** shrinks steadily, about 0.7× per layer, because Xavier doesn't include ReLU's factor 2. That's why the lecture's rule is **ReLU → He**.
- **Xavier with tanh:** shrinks slowly. For tanh, Xavier is the standard choice and far better than the bad options; the decay comes from tanh flattening values that aren't tiny.

Why σ = 0.01 is "small" depends on width: each z sums 256 terms, so its spread is σ·√256 ≈ 16σ times the input's. Xavier and He pick σ **from the layer size** precisely to cancel that √n.

4 · Does it matter for training? Same network, four starts

An 8-hidden-layer ReLU network (width 128) on the digits, plain SGD (lr 0.05, batch 64, 30 epochs). Only the initialization changes.

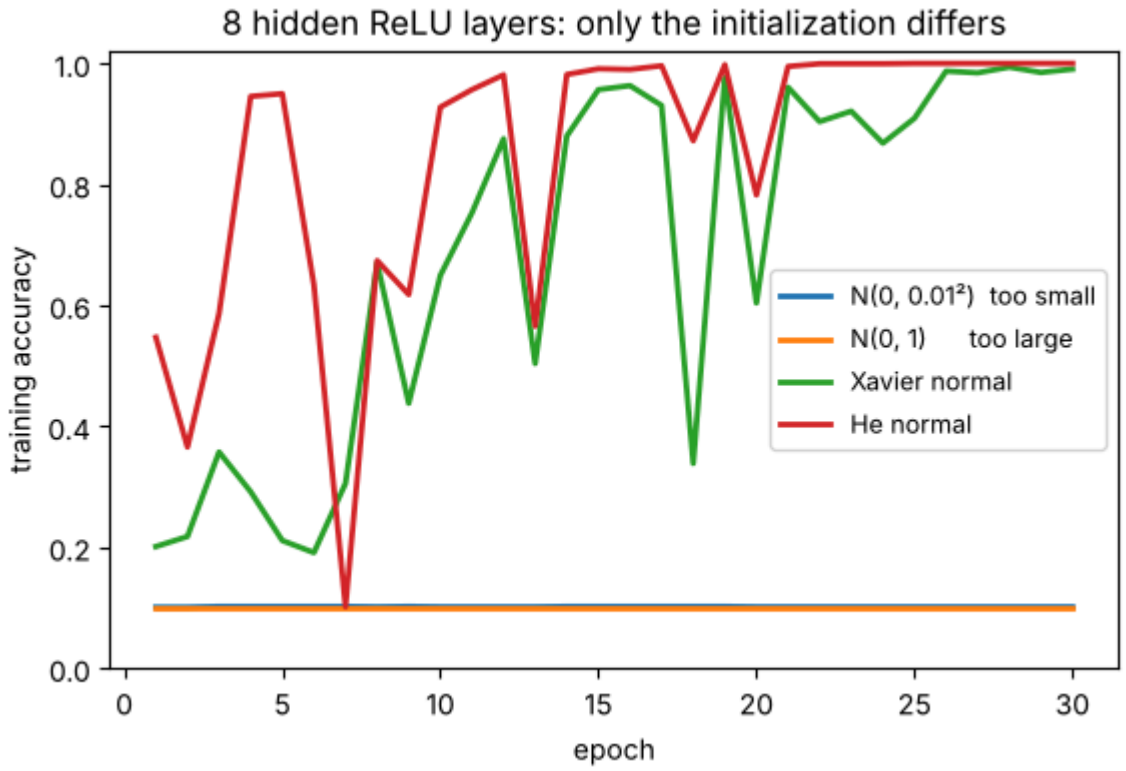
```
def make_deep(init_fn, depth=8, width=128):
    torch.manual_seed(0); layers, i = [], 64
    for _ in range(depth):
        L = nn.Linear(i, width); init_fn(L.weight); nn.init.zeros_(L.bias); layers += [L,
nn.ReLU()]; i = width
    L = nn.Linear(i, 10); init_fn(L.weight); nn.init.zeros_(L.bias); layers.append(L)
    return nn.Sequential(*layers)

inits = {
    "N(0, 0.01²)  too small": lambda w: nn.init.normal_(w, 0, 0.01),
    "N(0, 1)      too large": lambda w: nn.init.normal_(w, 0, 1.0),
    "Xavier normal":          nn.init.xavier_normal_,
    "He normal":              lambda w: nn.init.kaiming_normal_(w, nonlinearity="relu"),
}
```

```
histories = {}
for name, f in inits.items():
    net = make_deep(f); opt = torch.optim.SGD(net.parameters(), lr=0.05)
    hist = []
    for ep in range(30):
        perm = torch.randperm(len(X))
        for b in range(0, len(X), 64):
            idx = perm[b:b + 64]
            loss = F.cross_entropy(net(X[idx]), y[idx]); opt.zero_grad(); loss.backward();
opt.step()
        with torch.no_grad():
            out = net(X); hist.append((F.cross_entropy(out, y).item(), (out.argmax(1) ==
y).float().mean().item()))
    histories[name] = hist
    print(f"{name:<24} after epoch 1: loss {hist[0][0]:.3f} acc {hist[0][1]:.1%}   after 30: loss
{hist[-1][0]:.4f} acc {hist[-1][1]:.1%}")
```

```
N(0, 0.01²)  too small   after epoch 1: loss 2.303 acc 10.1%   after 30: loss 2.3025 acc 10.1%
N(0, 1)      too large   after epoch 1: loss nan acc 9.9%   after 30: loss nan acc 9.9%
Xavier normal            after epoch 1: loss 2.279 acc 20.1%   after 30: loss 0.0512 acc 99.1%
He normal                after epoch 1: loss 1.618 acc 54.7%   after 30: loss 0.0024 acc 100.0%
```

```
plt.figure(figsize=(6.5, 4))
for name, hist in histories.items():
    acc = [a for _, a in hist]
    plt.plot(range(1, 31), acc, lw=2, label=name)
plt.xlabel("epoch"); plt.ylabel("training accuracy"); plt.ylim(0, 1.02)
plt.title("8 hidden ReLU layers: only the initialization differs"); plt.legend(fontsize=9);
plt.show()
```



- **Too small:** stuck at chance (≈ 10%, loss 2.30 = ln 10) for all 30 epochs. The signal and the gradients vanish before reaching the early layers, so nothing learns.
- **Too large:** the loss becomes `nan` (numbers overflow) and the network never recovers.
- **Xavier:** learns, but after a slow start (20.1% after epoch 1, 99.1% after 30).
- **He:** learns fastest (54.7% after epoch 1) and reaches the lowest loss (0.0024, 100% training accuracy), as expected for ReLU.

Same architecture, same data, same optimiser: initialization alone decides between "doesn't learn", "crashes" and "works".

> In practice, **Batch Normalization** (chapter 05) and residual connections make deep networks much less sensitive to initialization. But the starting scale still matters, and every framework picks one for you.

5 · What does PyTorch do by default?

`nn.Linear` and `nn.Conv2d` initialize weights with `kaiming_uniform_(a=√5)`, which works out to **U(−1/√n_in, 1/√n_in)**, variance **1/(3·n_in)**. That's smaller than He's 2/n_in (by a factor of 6), a historical choice. For shallow networks it's fine; for deep plain ReLU stacks you may want to set He explicitly.

```
layer = nn.Linear(100, 50)
w = layer.weight.detach()
print(f"default nn.Linear(100, 50): max |w| = {w.abs().max():.4f} (1/sqrt(100) = {1 /
np.sqrt(100):.4f}),"
      f" variance = {w.var():.5f} (1/(3*100) = {1 / 300:.5f})")
print(f"He would use variance 2/100 = 0.02, i.e. {0.02 / (1 / 300):.0f}x larger")

# how to set it yourself
def init_he(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity="relu")
        nn.init.zeros_(m.bias)
model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))
model.apply(init_he)       # .apply visits every sub-module
print(f"after model.apply(init_he): first-layer weight std = {model[0].weight.std():.4f}
(sqrt(2/784) = {np.sqrt(2 / 784):.4f})")
```

```
default nn.Linear(100, 50): max |w| = 0.0999 (1/sqrt(100) = 0.1000), variance = 0.00337 (1/(3*10
0) = 0.00333)
He would use variance 2/100 = 0.02, i.e. 6x larger
after model.apply(init_he): first-layer weight std = 0.0505 (sqrt(2/784) = 0.0505)
```

6 · Summary: choosing an initializer

Activation in the hidden layers	Use	PyTorch
sigmoid, tanh	**Xavier / Glorot**	`nn.init.xavier_normal_` or `xavier_uniform_`
ReLU, Leaky ReLU, PReLU	**He / Kaiming**	`nn.init.kaiming_normal_(w, nonlinearity="relu")`
biases	zeros	`nn.init.zeros_`

The chain the lecture draws: initialization sets the **scale of the weights** → which sets the **scale of activations** (forward) and **derivatives** (backward) → too small vanishes, too large explodes → which decides whether the weight updates do anything useful.

Initialization happens **once, before** training. It is **not** an optimiser.

✅ Check yourself

1. A layer has n_in = 200, n_out = 100. Xavier normal σ? He normal σ? *(√(2/300) ≈ 0.082; √(2/200) = 0.1)*
2. He uniform limit for n_in = 150? *(√(6/150) = 0.2)*
3. What exactly goes wrong if all hidden weights start equal? *(every hidden neuron computes the same thing and gets the same gradient, so they stay identical: section 2 measured 1 distinct neuron out of 8)*
4. Why does He use 2/n_in instead of 1/n_in? *(ReLU zeroes about half its inputs, halving the signal's power; the factor 2 restores it)*
5. Your 10-layer ReLU network's loss stays at 2.30 on a 10-class problem from the first epoch. Likely cause? *(weights too small: vanishing signal; the loss ln 10 means uniform guessing)*
6. Is it OK to initialize biases to zero? *(yes, as long as the weights are random)*

09 · Text → Tokens → Vectors: BPE, WordPiece, embeddings, positions

Handout: *Text → Tokens → Vectors* · **Notes:** `../notes/09_text_tokens_vectors.md`

How a Transformer turns raw text into an ordered sequence of vectors, in three stages:

"playing!" → tokens ["play", "##ing", "!"] → IDs [27, 91, 8] → rows of an embedding table → + position vectors → ordered input

In this notebook you will:

1. **Build a BPE trainer from scratch** (the handout's take-home challenge) and reproduce its worked example merge by merge.
2. Compute the **WordPiece** association score and write the **greedy longest-match** segmenter, then solve the handout's exercises in code.
3. Run the **real BERT WordPiece tokenizer** (its actual 30,522-token vocabulary) on the handout's practical samples.
4. Verify that an **embedding layer** is a lookup table (= one-hot × matrix), compute its parameters and memory, and see that **only the used rows get gradients**.
5. Build **sinusoidal positional encodings** and prove that attention **without positions can't tell "dog bites man" from "man bites dog"**.

Runs in under a minute. The only download is BERT's vocabulary file (about 230 KB).

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn
from collections import Counter
torch.manual_seed(0)
```

```
<torch._C.Generator at 0x7fe70e9f0e10>
```

1 · Why subwords? The trade-off in one table

Unit	Vocabulary size	Sequence length	Advantage	Problem
whole words	very large	short	natural units	unknown words, typos, rare forms
characters / bytes	very small	long	any string representable	more positions → more computation
subwords	medium	medium	reusable pieces, few unknowns	segmentation quality varies by domain/language

Two costs pull in opposite directions:

- **embedding parameters = V × d**, so a bigger vocabulary means a bigger table;

- **attention work per layer ∝ $n^2 \cdot d$**, so more tokens make it quadratically slower.

```
for n in [100, 200, 400]:
    print(f"sequence length {n:>3} tokens -> relative attention work {(n / 100) ** 2:>4.0f}x")
```

```
sequence length 100 tokens -> relative attention work    1x
sequence length 200 tokens -> relative attention work    4x
sequence length 400 tokens -> relative attention work   16x
```

Doubling the number of tokens makes the n^2 part of attention about **4×** more work. That's why a tokenizer that splits text into fewer, well-chosen pieces matters for cost and speed.

2 · Byte Pair Encoding (BPE), from scratch

Training rule: start from characters, then repeatedly **merge the most frequent adjacent pair** (counts weighted by word frequency), recording each merge. It is a greedy **compression** procedure, not a dictionary of meaningful morphemes.

The handout's corpus: `low` ×5, `lower` ×2, `newest` ×6, `widest` ×3, with `</w>` marking the end of a word.

```
corpus = [(["l", "o", "w", "</w>"], 5),
          (["l", "o", "w", "e", "r", "</w>"], 2),
          (["n", "e", "w", "e", "s", "t", "</w>"], 6),
          (["w", "i", "d", "e", "s", "t", "</w>"], 3)]

def pair_counts(corpus):                    # the handout's starter code
    counts = {}
    for symbols, frequency in corpus:
        for pair in zip(symbols, symbols[1:]):
            counts[pair] = counts.get(pair, 0) + frequency
    return counts

def merge_word(symbols, pair):
    out, i = [], 0
    while i < len(symbols):
        if i < len(symbols) - 1 and (symbols[i], symbols[i + 1]) == pair:
            out.append(symbols[i] + symbols[i + 1]); i += 2      # glue the pair into one symbol
        else:
            out.append(symbols[i]); i += 1
    return out

counts = pair_counts(corpus)
for pair, c in sorted(counts.items(), key=lambda kv: -kv[1])[:8]:
    print(pair, c)
```

```
('e', 's') 9
('s', 't') 9
('t', '</w>') 9
('w', 'e') 8
('l', 'o') 7
('o', 'w') 7
('n', 'e') 6
('e', 'w') 6
```

This matches the handout: **(e,s), (s,t) and (t,) tie at 9**, (w,e) has 8, (l,o) has 7. A real implementation needs a **deterministic tie-break**. Python's `max` keeps the **first** pair it finds with the highest count, which here is (e,s), the same choice the handout makes.

```
def train_bpe(corpus, K, verbose=True):
    corpus = [(list(s), f) for s, f in corpus]
```

```
    merges = []
    for k in range(K):
        counts = pair_counts(corpus)
        if not counts:
            break
        best = max(counts, key=counts.get)                      # first pair with the maximum
count
        corpus = [(merge_word(s, best), f) for s, f in corpus]
        merges.append(best)
        if verbose:
            words = "   ".join(" ".join(s) + f"(x{f})" for s, f in corpus)
            print(f"merge {k + 1:>2}: {best[0]!r:>6} + {best[1]!r:<6} (count {counts[best]:>2}) -
> {words}")
    return merges, corpus

merges, final = train_bpe(corpus, K=10)
```

```
merge  1:    'e' + 's'    (count  9) -> l o w </w>(x5)   l o w e r </w>(x2)   n e w es t </w>(x6)
w i d es t </w>(x3)
merge  2:   'es' + 't'    (count  9) -> l o w </w>(x5)   l o w e r </w>(x2)   n e w est </w>(x6)
w i d est </w>(x3)
merge  3:  'est' + '</w>' (count  9) -> l o w </w>(x5)   l o w e r </w>(x2)   n e w est</w>(x6)
w i d est</w>(x3)
merge  4:    'l' + 'o'    (count  7) -> lo w </w>(x5)   lo w e r </w>(x2)   n e w est</w>(x6)   w
i d est</w>(x3)
merge  5:   'lo' + 'w'    (count  7) -> low </w>(x5)   low e r </w>(x2)   n e w est</w>(x6)   w i
d est</w>(x3)
merge  6:    'n' + 'e'    (count  6) -> low </w>(x5)   low e r </w>(x2)   ne w est</w>(x6)   w i
d est</w>(x3)
merge  7:   'ne' + 'w'    (count  6) -> low </w>(x5)   low e r </w>(x2)   new est</w>(x6)   w i d
est</w>(x3)
merge  8:  'new' + 'est</w>' (count  6) -> low </w>(x5)   low e r </w>(x2)   newest</w>(x6)   w i
d est</w>(x3)
merge  9:  'low' + '</w>' (count  5) -> low</w>(x5)   low e r </w>(x2)   newest</w>(x6)   w i d e
st</w>(x3)
merge 10:    'w' + 'i'    (count  3) -> low</w>(x5)   low e r </w>(x2)   newest</w>(x6)   wi d es
t</w>(x3)
```

Exactly the handout's story: **merge 1 (e,s) → es**, **merge 2 (es,t) → est**, then the ending `est</w>`. Then (l,o) → lo and (lo,w) → low, so the frequent word **low** becomes a single token, and so does **newest** (merge 8). **widest** is still in pieces after 10 merges, because it is less frequent. That answers **Exercise 2**: after the first two merges, (l,o) still occurs 7 times (low ×5 + lower ×2), and merging it creates `lo`, so `(lo, w)` (also 7) can form `low`.

Applying learned merges to new words

Encoding a new word = split it into characters, then **apply the stored merges in the order they were learned**.

```
def bpe_encode(word, merges):
    symbols = list(word) + ["</w>"]
    for pair in merges:
        symbols = merge_word(symbols, pair)
    return symbols

for w in ["lowest", "newer", "wider", "slow", "low"]:
    print(f"{w:>7} -> {bpe_encode(w, merges)}")
```

```
 lowest -> ['low', 'est</w>']
  newer -> ['new', 'e', 'r', '</w>']
  wider -> ['wi', 'd', 'e', 'r', '</w>']
   slow -> ['s', 'low</w>']
    low -> ['low</w>']
```

The unseen word **lowest** reuses learned pieces (`low` + `est</w>`). Nothing is ever "unknown": the worst case falls back to single characters.

3 · WordPiece

3a · The training score: association, not just frequency

Classroom score: **score(a, b) = count(ab) / (count(a) × count(b))**. Common pieces are penalised by the denominator.

```
score = lambda c_ab, c_a, c_b: c_ab / (c_a * c_b)
print(f"pair xy: 10 / (100 x 20) = {score(10, 100, 20):.3f}")
print(f"pair ab:  6 / (10 x 10)  = {score(6, 10, 10):.3f}  <- WordPiece prefers ab, although xy
is more frequent")
```

```
pair xy: 10 / (100 x 20) = 0.005
pair ab:  6 / (10 x 10)  = 0.060  <- WordPiece prefers ab, although xy is more frequent
```

3b · Encoding: greedy longest match, with `##` for continuation pieces

```
def wordpiece(word, vocab, unk="[UNK]"):
    pieces, start = [], 0
    while start < len(word):
        end, found = len(word), None
        while start < end:                                  # try the LONGEST piece first
            piece = word[start:end] if start == 0 else "##" + word[start:end]
            if piece in vocab:
                found = piece; break
            end -= 1
        if found is None:
            return [unk]                                    # classic WordPiece gives up on the
whole word
        pieces.append(found); start = end
    return pieces

vocab_a = {"play", "##ing", "##er", "un", "##able", "able"}
for w in ["playing", "player", "unable"]:
    print(f"{w:>9} -> {wordpiece(w, vocab_a)}")

print("\nExercise 3 vocabulary: read, ##ing, ##er, un, ##read, ##able")
vocab_b = {"read", "##ing", "##er", "un", "##read", "##able"}
for w in ["reader", "reading", "unreadable"]:
    print(f"{w:>11} -> {wordpiece(w, vocab_b)}")
print("without ##read:", wordpiece("unreadable", vocab_b - {"##read"}))
```

```
   playing -> ['play', '##ing']
    player -> ['play', '##er']
    unable -> ['un', '##able']

Exercise 3 vocabulary: read, ##ing, ##er, un, ##read, ##able
      reader -> ['read', '##er']
     reading -> ['read', '##ing']
  unreadable -> ['un', '##read', '##able']
without ##read: ['[UNK]']
```

Exercise 3 answered by running it: all three words segment successfully with the listed vocabulary (`unreadable → un + ##read + ##able`). The handout's question says "which item *may* fail", and its answer key reaches the same conclusion: it succeeds here. It **would** fail

(`[UNK]`) without `##read`, because after `un` the rest of the word must be matched with **continuation** pieces, and the standalone `read` doesn't count. The last line proves it.

4 · The real thing: BERT's WordPiece tokenizer

The handout's practical uses `bert-base-uncased`. We load its actual vocabulary file (30,522 tokens): first from Hugging Face (the normal route), and if that fails, from a GitHub mirror of the same file. Then we run the fast `tokenizers` library, the engine inside Hugging Face's `AutoTokenizer`.

```
import os, urllib.request
os.makedirs("data", exist_ok=True)
VOCAB = "data/bert-base-uncased-vocab.txt"
if not os.path.exists(VOCAB):
    for url in ["https://huggingface.co/bert-base-uncased/resolve/main/vocab.txt",
                "https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/bert-
base-uncased-vocab.txt"]:
        try:
            urllib.request.urlretrieve(url, VOCAB); print("vocabulary from", url.split("/")[2]);
break
        except Exception as e:
            print("failed:", url.split("/")[2], type(e).__name__)
vocab_lines = open(VOCAB, encoding="utf-8").read().splitlines()
print("vocabulary size:", len(vocab_lines), "| first special tokens:", vocab_lines[0],
vocab_lines[100:104])

from tokenizers import BertWordPieceTokenizer     # pip install tokenizers  (Hugging Face's fast
tokenizer library)
bert = BertWordPieceTokenizer(VOCAB, lowercase=True)
samples = ["Tokenization is unbelievably useful.",
           "Tokenizashun is useful.",                        # deliberate typo
           "vector<vector<int>> values;",                    # source code
           "Thiruvananthapuram costs ₹1,23,456.",            # rare place name + Indian number
format
           "playing player unplayable",
           "Tokenization is incredibly useful.",             # English ...
           "La tokenización es increíblemente útil."]        # ... and the same sentence in
Spanish
for text in samples:
    enc = bert.encode(text)
    print(f"\n{text}\n  {len(enc.ids):>2} tokens: {enc.tokens}")
```

```
vocabulary size: 30522 | first special tokens: [PAD] ['[UNK]', '[CLS]', '[SEP]', '[MASK]']

Tokenization is unbelievably useful.
  12 tokens: ['[CLS]', 'token', '##ization', 'is', 'un', '##bel', '##ie', '##va', '##bly', 'usefu
l', '.', '[SEP]']

Tokenizashun is useful.
   9 tokens: ['[CLS]', 'token', '##iza', '##shu', '##n', 'is', 'useful', '.', '[SEP]']

vector<vector<int>> values;
  11 tokens: ['[CLS]', 'vector', '<', 'vector', '<', 'int', '>', '>', 'values', ';', '[SEP]']

Thiruvananthapuram costs ₹1,23,456.
  18 tokens: ['[CLS]', 'th', '##ir', '##u', '##vana', '##nt', '##ha', '##puram', 'costs', '₹', '#
#1', ',', '23', ',', '45', '##6', '.', '[SEP]']

playing player unplayable
   7 tokens: ['[CLS]', 'playing', 'player', 'un', '##play', '##able', '[SEP]']

Tokenization is incredibly useful.
   8 tokens: ['[CLS]', 'token', '##ization', 'is', 'incredibly', 'useful', '.', '[SEP]']

La tokenización es increíblemente útil.
  15 tokens: ['[CLS]', 'la', 'token', '##iza', '##cion', 'es', 'inc', '##re', '##ible', '##ment',
'##e', 'ut', '##il', '.', '[SEP]']
```

Observation table (the handout's practical, filled in from the run above):

- **Common sentence:** "tokenization" → `token ##ization`, and even "unbelievably" splits into **5** pieces. Frequent-looking words aren't always single tokens.
- **Typo:** "Tokenizashun" doesn't become `[UNK]`. It breaks into smaller pieces (`token ##iza ##shu ##n`), so subwords handle typos gracefully, at the cost of more tokens.
- **Source code:** every symbol becomes its own token, and identifiers are kept when they are English words.
- **Place name + number:** the rare Indian place name shatters into **7** pieces, and "₹1,23,456" (Indian digit grouping) becomes 7 tokens. A 3-word sentence cost 16 tokens plus the 2 special tokens: high **token fertility** for text outside the tokenizer's main training domain.
- **English vs Spanish, same meaning:** "Tokenization is incredibly useful." needs 6 tokens without the specials, while the Spanish sentence needs 13, more than **twice** as many, because the vocabulary was built mostly from English text. This is the handout's **cost and fairness** question: users of some languages pay more tokens (money, context window, speed) for the same meaning.
- `[CLS]` and `[SEP]` are special tokens BERT adds at the start and end.

5 · Embedding layers: a trainable lookup table

Token IDs are just names (ID 27 is not "bigger" than ID 8). The embedding matrix **E** has shape **V × d**: one learnable row per token. Embedding a sequence of n IDs gives an **n × d** matrix.

```
embedding = nn.Embedding(num_embeddings=100, embedding_dim=4)        # the handout's code
ids = torch.tensor([[27, 91, 8]])                                      # (batch=1, sequence=3)
x = embedding(ids)
print("E shape:", tuple(embedding.weight.shape), "| output shape:", tuple(x.shape))
print("x[0, 0] is exactly row 27 of E:", torch.equal(x[0, 0], embedding.weight[27]))
one_hot = nn.functional.one_hot(ids, num_classes=100).float()
print("lookup == one-hot @ E:", torch.allclose(x, one_hot @ embedding.weight))
print(f"one-hot route: builds {one_hot.numel()} numbers ({int(one_hot.sum())} non-zero) and
multiplies by the whole table -> lookup is far cheaper")
```

```
E shape: (100, 4) | output shape: (1, 3, 4)
x[0, 0] is exactly row 27 of E: True
lookup == one-hot @ E: True
one-hot route: builds 300 numbers (3 non-zero) and multiplies by the whole table -> lookup is far
cheaper
```

Parameters and memory (handout example and Exercise 4)

```
def table(V, d):
    params = V * d
    return params, params * 4 / 1e6, params * 2 / 1e6        # fp32 MB, fp16 MB (decimal)
for name, V, d in [("handout example", 30_000, 768), ("Exercise 4", 50_000, 512), ("real bert-
base-uncased", 30_522, 768)]:
    p, mb32, mb16 = table(V, d)
    print(f"{name:>22}: E is {V:,} x {d} = {p:,} parameters -> fp32 {mb32:.2f} MB, fp16
{mb16:.2f} MB")
B, n = 8, 128
print(f"Exercise 4: a batch of token IDs of shape ({B}, {n}) becomes an embedded tensor of shape
({B}, {n}, 512)")
print(f"handout example in MiB: {30_000 * 768 * 4 / 2**20:.1f} MiB")
```

```
       handout example: E is 30,000 x 768 = 23,040,000 parameters -> fp32 92.16 MB, fp16 46.08 MB
            Exercise 4: E is 50,000 x 512 = 25,600,000 parameters -> fp32 102.40 MB, fp16 51.20 M
B
real bert-base-uncased: E is 30,522 x 768 = 23,440,896 parameters -> fp32 93.76 MB, fp16 46.88 MB
Exercise 4: a batch of token IDs of shape (8, 128) becomes an embedded tensor of shape (8, 128, 5
12)
handout example in MiB: 87.9 MiB
```

Only the rows that were used get gradients

IDs `[4, 9, 4]`: rows 4 and 9 receive gradient, and **row 4 accumulates two contributions** (it appeared twice).

```
emb = nn.Embedding(12, 3)
out = emb(torch.tensor([4, 9, 4]))
out.sum().backward()                                    # a toy loss: every output element counts
1
rows_with_grad = (emb.weight.grad.abs().sum(1) > 0).nonzero().ravel().tolist()
print("rows with non-zero gradient:", rows_with_grad)
print("gradient of row 4:", emb.weight.grad[4].tolist(), " (two positions -> 2 per element)")
print("gradient of row 9:", emb.weight.grad[9].tolist(), " (one position  -> 1 per element)")
```

```
rows with non-zero gradient: [4, 9]
gradient of row 4: [2.0, 2.0, 2.0]  (two positions -> 2 per element)
gradient of row 9: [1.0, 1.0, 1.0]  (one position  -> 1 per element)
```

That's the handout's $\partial L/\partial E[4] = \partial L/\partial x_1 + \partial L/\partial x_3$ in action. It's also why rare tokens learn slowly: their rows are only updated when they appear.

Static vs contextual: row "bank" of E is the same in "river bank" and "bank loan". The **later Transformer layers** produce different, contextual vectors (see the CV course's attention notebook for the "bank" example).

6 · Positions: why attention needs them

6a · Proof: content-only attention ignores word order

```
torch.manual_seed(0)
d = 8
words = {"dog": 0, "bites": 1, "man": 2}
E = nn.Embedding(3, d)
attn = nn.MultiheadAttention(embed_dim=d, num_heads=1, batch_first=True)

def encode(sentence, pos=None):
    x = E(torch.tensor([[words[w] for w in sentence.split()]]))
    return x if pos is None else x + pos[: x.shape[1]]

with torch.no_grad():
    for label, pos in [("WITHOUT positions", None), ("WITH positions", torch.randn(3, d))]:
        a, b = encode("dog bites man", pos), encode("man bites dog", pos)
        out_a, _ = attn(a, a, a); out_b, _ = attn(b, b, b)
        # compare the output vector for each WORD (dog, bites, man) across the two sentences
        same_per_word = torch.allclose(out_a[0, [0, 1, 2]], out_b[0, [2, 1, 0]], atol=1e-6)
        print(f"{label}: outputs for each word identical in both sentences? {same_per_word}   "
              f"| sentence summaries (mean) identical? {torch.allclose(out_a.mean(1),
out_b.mean(1), atol=1e-6)}")
```

```
WITHOUT positions: outputs for each word identical in both sentences? True   | sentence summaries
(mean) identical? True
WITH positions: outputs for each word identical in both sentences? False   | sentence summaries
(mean) identical? False
```

Without positions, each word gets exactly the same output vector whether the sentence is "dog bites man" or "man bites dog". The outputs are just **reordered** with the input (**permutation equivariance**), and any order-free summary is identical. The model literally can't tell who bit whom. **With position vectors added**, the outputs differ.

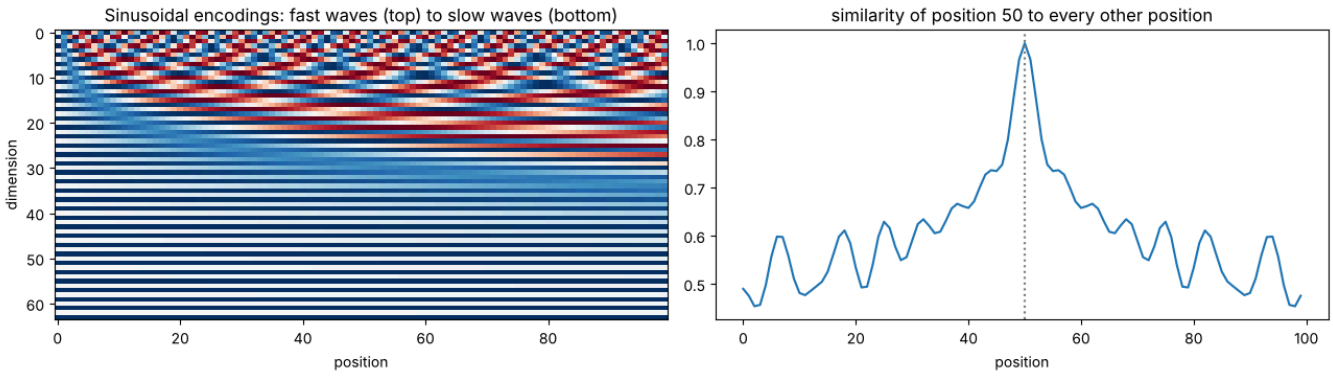
6b · Sinusoidal positional encodings

PE(pos, 2i) = sin(pos / 10000^(2i/d)), PE(pos, 2i+1) = cos(pos / 10000^(2i/d))

```
def sinusoidal(max_len, d):
    pe = np.zeros((max_len, d))
    pos = np.arange(max_len)[:, None]
    i = np.arange(0, d, 2)[None, :]
    pe[:, 0::2] = np.sin(pos / 10000 ** (i / d))
    pe[:, 1::2] = np.cos(pos / 10000 ** (i / d))
    return pe

print("handout example, d = 4, position 1:", sinusoidal(2, 4)[1].round(5), " (expected [0.8415,
0.5403, 0.0100, 0.99995])")
pe = sinusoidal(100, 64)
fig, axes = plt.subplots(1, 2, figsize=(13, 3.8))
axes[0].imshow(pe.T, aspect="auto", cmap="RdBu"); axes[0].set_xlabel("position");
axes[0].set_ylabel("dimension")
axes[0].set_title("Sinusoidal encodings: fast waves (top) to slow waves (bottom)")
sim = pe @ pe.T
axes[1].plot(sim[50] / sim[50, 50]); axes[1].axvline(50, color="grey", ls=":")
axes[1].set_title("similarity of position 50 to every other position");
axes[1].set_xlabel("position")
plt.tight_layout(); plt.show()
```

```
handout example, d = 4, position 1: [0.84147 0.5403  0.01    0.99995]  (expected [0.8415, 0.5403,
0.0100, 0.99995])
```



- Each pair of dimensions is a wave of a different frequency, so every position gets a unique multi-scale "barcode". **Nearby positions have similar codes** (right plot), which helps the model reason about distance.
- They're **fixed** (no parameters). **Learned** absolute position embeddings (a trainable L_max × d table, as in BERT) are the alternative: flexible, but undefined beyond L_max. Modern models often use **RoPE** or relative biases.
- **Exercise 5:**
  - The same token at positions 2 and 10 has the **same token embedding**; only the position part differs.
  - Token and position vectors must have the **same width d**, because they're added element by element.
  - Content-only attention just **permutes its outputs** when the inputs are permuted.

✅ Self-test (handout)

1. Why not one token per possible word? *(open-ended vocabulary: huge table and unknown words)*
2. V doubles, d fixed: embedding parameters? *(double: V × d)*
3. IDs [7, 2, 7]: which rows get gradient? *(7 and 2; row 7 accumulates two contributions)*
4. What is missing from content-only self-attention? *(word order / position)*
5. BPE vs WordPiece in one line each? *(BPE merges the most frequent pair; WordPiece prefers strongly associated pairs and encodes by greedy longest match with ##)*

DL & GenAI · Cheat Sheet (lectures 04-08 → 04-09 + tokens handout)

History (04-08)

Year	What	Who
1943	first artificial neuron (logic with binary inputs/outputs)	McCulloch & Pitts
1958	perceptron: first **trainable** network (linearly separable only)	Rosenblatt
1969	*Perceptrons*: single-layer limits (XOR) → **AI winter**	Minsky & Papert
1986	**backpropagation** popularised → multi-layer training	Rumelhart, Hinton, Williams
1998	**LeNet-5** CNN: digits, cheques, postcodes	LeCun
2006	**Deep Belief Networks**, layer-wise pre-training → modern DL	Hinton
2012	**AlexNet** wins ImageNet: **GPUs, ReLU, Dropout**	Krizhevsky, Sutskever, Hinton
2014	**GANs** (generator vs discriminator)	Goodfellow
2017	**Transformer**, "Attention Is All You Need"	Google
2020+	foundation models & GenAI: LLMs, multimodal, diffusion (GPT, Llama, Gemini, Claude, Stable Diffusion)	many
- **Success factors:** big datasets · GPUs/TPUs · better algorithms (backprop, ReLU, BatchNorm, Dropout, attention) · open-source frameworks (TensorFlow, PyTorch, Keras) · cloud.		
- **Model evolution:** neuron → perceptron → MLP → CNN → RNN → LSTM → Transformer → foundation models.		
- **"Deep"** = number of **hidden layers** (not deep thinking). Layers learn edges → corners/textures → shapes → parts → object, **automatically**.		

AI ⊃ ML ⊃ DL ⊃ GenAI (05-08)

	AI	ML	DL	GenAI
idea	mimic intelligent behaviour	learn from data	multi-layer neural nets	generate new content
features	n/a	**manual**	**automatic**	automatic, massive scale
data	varies	moderate (1000s)	large (100k+)	massive (web-scale)
architecture	rules, search…	trees, SVM, linear	MLP, CNN, RNN/LSTM	**Transformers**, diffusion
example	rule-based chess engine	spam filter	CNN on X-rays	LLM writing an essay
- Three distinguishing factors: **data, feature engineering, architecture**.				
- Misconceptions: AI ≠ ML · DL isn't "just more layers" (automatic features) · GenAI *is* scaled DL · bigger isn't always better.				

Biological → artificial neuron

dendrites	synapse strength	soma	firing threshold	axon	plasticity
inputs x_i	weights w_i	$\Sigma w_i x_i + b$	activation $f(z)$	output	weight updates (backprop)
- Brain: ~86 billion neurons, ~100 trillion synapses, ~20 W. Artificial: numbers, synchronous steps, exact learning rule, a **drastic simplification** by design.					

Perceptron (06-08)

- **$z = \Sigma w_i x_i + b$, $\hat{y} = step(z)$** (1 if $z \geq 0$). Worked example: x = (2, 3), w = (0.5, 0.8), b = −2 → z = **1.4** → Pass.
- **Update:** $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$, $b \leftarrow b + \eta(y - \hat{y})$, error ∈ {−1, 0, +1}. Worked example: w (0.2, −0.4), b −1, y 1, ŷ 0, η 0.1 → **w (0.4, −0.1), b −0.9**.
- η too small = slow, too big = overshoot/diverge (gradient descent). Plain perceptron from zero weights: η doesn't change the epoch count.
- **Linearly separable only:** AND ✓, OR ✓, **XOR ✗** (best single line 75%) → MLP with a hidden layer (+ backprop) solves it.

Activation functions (07-08, 11-08)

f	formula	range	slope	use
Step	1 if $z \geq 0$ else 0	$\{0, 1\}$	0 (no gradient)	classic perceptron
Linear	z	$(-\infty, \infty)$	1	**regression output**
Sigmoid	$1/(1 + e^{-z})$	$(0, 1)$	$\sigma(1 - \sigma) \leq$ **0.25**	**binary output**, gates
tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	$(-1, 1)$, **zero-centred**	$1 - \tanh^2 \leq 1$	RNN/LSTM states
ReLU	$\max(0, z)$	$[0, \infty)$	1 / 0	**hidden layers** (+ He init)
Leaky ReLU	z or αz (α fixed, e.g. 0.01)	$(-\infty, \infty)$	1 / α	dying-ReLU concern
PReLU	z or az (**a learned**)	$(-\infty, \infty)$	1 / a	CNNs, adaptive slope
- No activation → stacked linear layers = **one linear layer**.				
- **Vanishing gradient:** small slopes (sigmoid/tanh saturation) multiplied over layers → early layers stop learning.				
- **Dying ReLU:** slope **exactly 0** for negative input → the neuron never updates. Leaky/PReLU keep a non-zero slope.				

More activations (13-08)

f	formula	note
ELU	z, or $\alpha(e^{z} - 1)$ for $z < 0$	smooth, bounded below by $-\alpha$
GELU	$z \cdot \Phi(z)$	soft gate; **Transformers** (BERT/GPT); min ≈ -0.17
Swish/SiLU	$z \cdot \sigma(z)$	smooth, non-monotonic; min ≈ -0.28
Softmax	$e^{z_i}/\Sigma e^{z_j}$ on a vector	sums to 1; **single-label multi-class output**; subtract max(z) for stability
- (2.5, 1.2, 0.3) → **0.723 / 0.197 / 0.080** (handout says 0.07: rounding slip). Multi-label → **independent sigmoids**, not softmax.		

f	formula	note
- **Output:** binary → sigmoid · multi-class → softmax · regression → linear · positive → softplus · bounded → scaled sigmoid/tanh.		
- **Hidden:** ReLU default · dying → Leaky/PReLU · Transformer → GELU · bounded zero-centred → tanh · **validate, don't pick "newest"**.		

Loss functions (18-08)

problem	loss	output	labels
regression	**MSE** $(1/n)\Sigma(y - \hat{y})^2$	linear	numbers
regression + outliers	**MAE** $(1/n)\Sigma\lvert y - \hat{y}\rvert$	linear	numbers
binary	**BCE** $-[y \log p + (1 - y) \log(1 - p)]$	sigmoid	0/1
multi-class	**CCE** $-\Sigma\, y_k \log p_k = -\log p_correct$	softmax	one-hot
multi-class	**sparse CCE** (same loss)	softmax	integer
- Example (100, 200, 300) vs (90, 220, 270): **MSE 466.67, MAE 20**. Errors 2 vs 10: ×5 → ×25 under MSE.			
- BCE for y = 1: p 0.99 → 0.010, 0.1 → 2.303, 0.01 → 4.605 (confident and wrong = huge). CCE p = 0.82 → 0.198.			
- Why not MSE for classification: saturated sigmoid → tiny gradient (−0.013 vs BCE −0.99 at logit −5). BCE + sigmoid gradient = **p − y**.			
- **"Sparse" = the label format**, not the predictions. PyTorch `CrossEntropyLoss` takes **logits** (don't add Softmax).			

Working of an MLP (20-08)

- input → **Z = XW + b** → **A = f(Z)** → … → ŷ → **loss** → **backprop** (chain rule) → **W ← W − η·∂L/∂W** → repeat.
- Softmax + CE backprop: $\boldsymbol{\delta^2 = \hat{Y} - Y}$, $\partial W^2 = A^{1T}\delta^2$, $\delta^1 = (\delta^2 W^{2T}) \odot f'(Z^1)$, $\partial W^1 = X^T\delta^1$ (verified: numerical 10^{-11}, autograd 10^{-17}).
- Examples: z = 2(0.5) + 3(0.2) + 1(−0.4) + 0.1 = **1.3**; Y [1, 0], Ŷ [0.7, 0.3] → L = **0.357**; w 2 − 0.1·0.5 = **1.95**.
- **Batch** = group of examples · **step** = one batch → one update · **epoch** = full pass. 50,000 / 128 → 391 steps/epoch; 50,000 / 100 × 10 epochs = **5,000 updates**.
- Train (fit weights) · validation (tune, early stop) · test (once). Underfit = poor everywhere; overfit = train ≫ validation.
- **Backprop computes gradients; the optimizer (SGD, Adam) uses them.**

Weight initialization (21-08)

	Xavier/Glorot	He/Kaiming
for	sigmoid, tanh	ReLU, Leaky, PReLU
normal Var	2/(n_in + n_out)	2/n_in
uniform ±a	√(6/(n_in + n_out))	√(6/n_in)
n_in 100, n_out 50	Var 0.0133, σ 0.115	Var 0.02, σ 0.141
- **All-equal/zero weights → symmetry**: neurons stay identical (measured: 8 → 1 distinct). Biases = 0 is fine.		
- Too small → **vanishing**; too large → **exploding**/saturation. Uniform Var = $a^2/3$ (hence the 6).		
- PyTorch default for Linear: U(±1/√n_in), Var 1/(3n_in) (6× smaller than He). `nn.init.kaiming_normal_(w, nonlinearity="relu")`.		

Normalization & regularization (04-09)

- **Normalization = optimisation** (scale). **Regularization = generalisation** (overfitting). Complementary.
- **Standardise** $(x - \mu)/\sigma$ · **min-max** $(x - \min)/(\max - \min)$ (outlier-sensitive) · pixels/255 · fit statistics on **train only** (no leakage).
- **BatchNorm:** batch μ, σ^2 → $\hat{x} = (x - \mu)/\sqrt{(\sigma^2 + \varepsilon)}$ → $\gamma\hat{x} + \beta$ · running statistics at inference · CNNs.
- **LayerNorm:** per-sample statistics over features · batch-independent · Transformers. Instance norm → style transfer; Group norm → small batches.
- **Overfitting:** training loss ↓ while validation loss ↑ (underfit = both poor).
- **L1** $\lambda\Sigma|w|$ → **sparsity** · **L2** $(\lambda/2)\Sigma w^2$ → shrinkage (gradient + λw) · **Elastic Net** = both.
- **Dropout** p: zero activations randomly in training, scale survivors by $1/(1 - p)$, off at inference.
- **Early stopping:** best validation epoch + patience · **augmentation:** label-preserving variations · **weight decay:** $(1 - \eta\lambda)w - \eta\nabla L$; with Adam use **AdamW** (decoupled).
- Guide: different scales → standardise · CNN → BatchNorm · Transformer → LayerNorm · overfit → L2 / dropout / early stopping · sparse → L1.

Text → tokens → vectors (handout)

- Pipeline: text → **tokens** → IDs → **embedding rows** (E is V × d) → **+ position vectors** → Transformer.
- Words: huge vocabulary, unknowns · characters/bytes: long sequences · **subwords**: balance. Attention cost ∝ $\mathbf{n^2 d}$ (2× tokens → 4× work).
- **BPE:** merge the most frequent adjacent pair, repeat (low/lower/newest/widest → es, est, est, lo, low, …).
- **WordPiece:** score count(ab)/(count(a)·count(b)) (0.06 beats 0.005); encode by **greedy longest match**, `##` = continuation.

- Embedding: lookup = one-hot × E (lookup is cheaper). Params **V × d**: 30,000 × 768 = **23.04M** (92.16 MB fp32); 50,000 × 512 = 25.6M (102.4 MB). Only the used rows get gradients ([4, 9, 4] → row 4 gets two).
- Attention is **permutation equivariant** → needs positions. Learned (L_max × d) or sinusoidal: PE(pos, 2i) = sin(pos/10000^{2i/d}), cos for 2i+1. d = 4, pos 1 → [0.8415, 0.5403, 0.0100, 0.99995]. Modern: RoPE.
- BERT: Thiruvananthapuram → 7 pieces; the same sentence costs 6 tokens in English, 13 in Spanish (**fairness**).